



UNEMI
UNIVERSIDAD ESTATAL DE MILAGRO

Compendio del autor

COMPUTACIÓN MÓVIL

UNIDAD 1
INTRODUCCIÓN A LA COMPUTACIÓN MÓVIL

ÍNDICE

Contenido

Unidad 1: INTRODUCCIÓN A LA COMPUTACIÓN MÓVIL.....	3
<i>Tema 1: Generalidades sobre app móviles</i>	<i>3</i>
<i>Objetivo:</i>	<i>3</i>
<i>Introducción:</i>	<i>3</i>
Información de los subtemas.....	4
<i>Subtema 1: ¿Qué es una app móvil?.....</i>	<i>4</i>
<i>Subtema 2: Características de una app móvil</i>	<i>6</i>
<i>Subtema 3: Tipos de apps</i>	<i>10</i>
<i>Subtema 4: Beneficios de las apps móviles.....</i>	<i>15</i>
Preguntas de Comprensión de la Unidad	18
Material complementario	19
Bibliografía.....	20

Unidad 1: INTRODUCCIÓN A LA COMPUTACIÓN MÓVIL

Tema 1: Generalidades sobre app móviles

Objetivo:

Identificar y aplicar los conceptos sobre programación móvil.

Introducción:

Las aplicaciones móviles, también conocidas como apps, han transformado radicalmente la forma en que interactuamos con la tecnología en nuestra vida cotidiana. Estas herramientas de software, diseñadas para ejecutarse en dispositivos móviles como teléfonos inteligentes y tabletas, ofrecen una amplia gama de funcionalidades que abarcan desde entretenimiento y productividad hasta comunicación y servicios especializados. Con su rápida adopción y crecimiento exponencial, las aplicaciones móviles se han convertido en una parte integral de la experiencia digital moderna, brindando a los usuarios acceso instantáneo a información, entretenimiento y servicios, en cualquier momento y lugar.

Información de los subtemas

Subtema 1: ¿Qué es una app móvil?

Origen

Se sabe que las primeras aplicaciones comenzaron a aparecer a finales de los años 90. No se trataba de apps para teléfonos inteligentes, sino para teléfonos analógicos. ¡Sí! Estas también se consideran aplicaciones. La agenda, juegos como el famoso Snake, el Tetris, los editores de tonos de llamada y herramientas para personalizar el teléfono cumplían funciones muy básicas en comparación con las aplicaciones actuales. Sin embargo, en su momento representaron un gran avance en la manera en que percibíamos los teléfonos celulares antiguos (conocidos como "bloques") y abrieron un mercado enorme, cuya competencia sigue siendo feroz, permitiéndonos disfrutar de herramientas cada vez más prácticas, útiles e increíbles. (Softcorp, s.f)

La tecnología EDGE marcó un antes y un después, ya que la posibilidad de conectarse a Internet disparó las oportunidades al máximo, atrayendo a visionarios y nuevos inversores. En esa época, aún existían grandes restricciones por parte de los fabricantes, que eran los propietarios de los sistemas operativos que venían preinstalados en los dispositivos, lo que impedía que desarrolladores externos pudieran incorporar nuevos elementos. Pero tarde o temprano, esto se haría realidad, y no faltaba mucho para que sucediera. (Softcorp, s.f)

En 2007, Apple realizó una jugada maestra que transformó por completo nuestra percepción de los dispositivos móviles y las aplicaciones, las cuales en ese momento se consideraban distantes, poco prácticas y nada relevantes. El iPhone, además de ser una gran innovación, ofreció una plataforma para descargar aplicaciones de desarrolladores externos sin las estrictas restricciones impuestas por los fabricantes en años anteriores. A través de su App Store, este lanzamiento hizo realidad el sueño de muchos desarrolladores que deseaban ofrecer aplicaciones sin las limitaciones de los modelos anteriores. (Softcorp, s.f)

Ese mismo año, Android lanzó el HTC Dream, que presentó una alternativa al App Store de Apple. Fue una apuesta arriesgada, inicialmente con solo 50 aplicaciones, pero que con el tiempo ha crecido hasta contar con más de un millón de aplicaciones en la actualidad. En marzo de 2012, Google renombró "Android Market" a "Google Play", que es como lo conocemos hoy. (Softcorp, s.f)

¿Qué son?

Una aplicación móvil, o app, es un tipo de software creado para funcionar en dispositivos móviles, como smartphones o tabletas. Aunque estas aplicaciones suelen ser pequeñas y con funciones limitadas, logran ofrecer servicios y experiencias de alta calidad a los usuarios. (Herazo, s.f)

A diferencia de las aplicaciones de escritorio, las apps móviles no forman parte de sistemas de software integrados. Cada app móvil brinda una funcionalidad específica y limitada, como un juego, una calculadora o un navegador web móvil. (Herazo, s.f)

Debido a las restricciones de hardware de los primeros dispositivos móviles, las aplicaciones evitaban la multifuncionalidad. Sin embargo, a pesar de que los dispositivos actuales son mucho más avanzados, las apps móviles siguen enfocándose en funciones específicas. Esto permite a los usuarios elegir exactamente qué funciones desean tener en sus dispositivos. (Herazo, s.f)

Las aplicaciones móviles, comúnmente conocidas como apps (abreviatura de "application" en inglés), son herramientas de software desarrolladas en diversos lenguajes de programación para dispositivos como teléfonos inteligentes y tabletas. Estas aplicaciones se destacan por ser útiles, dinámicas y fáciles de instalar. (Quiroz, 2022)

Por lo general, la mayoría de las aplicaciones móviles disponibles para los usuarios requieren una conexión estable a Internet para funcionar. Su descarga se realiza principalmente a través de las grandes tiendas virtuales de los gestores de sistemas operativos, como Android e iOS. (Quiroz, 2022)

Una aplicación móvil, comúnmente conocida como app, es un programa informático diseñado para ejecutarse en teléfonos inteligentes, tabletas y otros dispositivos móviles. Estas aplicaciones permiten a los usuarios realizar una amplia variedad de tareas, ya sea profesionales, de ocio, educativas o de acceso a servicios, entre otras, facilitando así diversas gestiones o actividades. (xpertosolutions, 2017)

Por lo general, las aplicaciones móviles están disponibles a través de plataformas de distribución operadas por las compañías propietarias de los sistemas operativos móviles, como Android, iOS, BlackBerry OS y Windows Phone, entre otros. Estas aplicaciones pueden ser gratuitas o de pago, donde generalmente el 20-30 % del costo se destina al distribuidor, y el resto va para el desarrollador. El término "app" se popularizó rápidamente, tanto que en 2010 fue elegida como Palabra del Año por la American Dialect Society. (xpertosolutions, 2017)

Subtema 2: Características de una app móvil

Interfaz de usuario bien diseñada

Las aplicaciones móviles enfrentan una alta tasa de abandono, por lo que es esencial crear una primera impresión sólida para retener a los usuarios. Esta impresión inicial se basa en una interfaz de usuario intuitiva y atractiva, que es clave al desarrollar una aplicación. (Microsoft, s.f)

El diseño de la interfaz de usuario abarca tanto su aspecto visual como su funcionalidad real. Aunque una aplicación pueda ofrecer valor significativo, perderá usuarios rápidamente si su interfaz no es intuitiva, ya que los usuarios pueden optar por no invertir tiempo en comprenderla. Además, si la interfaz carece de atractivo visual, puede ser difícil atraer a los usuarios, lo que dificulta su adopción generalizada. (Microsoft, s.f)

Dado que la mayoría de las personas utilizan aplicaciones en dispositivos móviles, la interfaz de usuario debe adaptarse a pantallas táctiles pequeñas. Esto implica simplificar el diseño de la aplicación eliminando funciones no esenciales para evitar una apariencia sobrecargada, y garantizar la coherencia del diseño en todas las plataformas y tamaños de dispositivos. (Microsoft, s.f)

Es fundamental mantener la coherencia en el diseño de la tipografía, los botones, los iconos y otros elementos de la marca para ofrecer una experiencia unificada al usuario y mejorar la legibilidad. Asimismo, la estructura de la aplicación debe ser coherente, priorizando visualmente el contenido más importante para facilitar la navegación y mejorar la experiencia del usuario. (Microsoft, s.f)

Tiempo de carga rápido

Un aspecto fundamental de una aplicación de calidad es su capacidad para cargar rápidamente y responder de manera eficiente. Esto no solo mejora la experiencia del usuario, sino que también ayuda a retener usuarios y aumentar las conversiones. (Microsoft, s.f)

Una aplicación móvil bien diseñada no debería tomar más de cinco segundos en cargarse, y lo ideal sería que lo hiciera en dos segundos. Los usuarios valoran la estabilidad, la confiabilidad y la velocidad en las aplicaciones, y tienden a eliminar aquellas que tienen tiempos de carga prolongados o que sufren fallos con frecuencia. Las causas comunes de la lentitud en las aplicaciones incluyen servidores sobrecargados, exceso de datos, versiones de software desactualizadas, código extenso y conexiones cifradas poco optimizadas. (Microsoft, s.f)

Para desarrollar una aplicación móvil rápida y receptiva, es importante implementar técnicas como el almacenamiento en caché del navegador, utilizar redes de entrega de contenido eficientes (CDN) y comprimir datos como imágenes, videos y audio. Además, es crucial mantener la aplicación actualizada y monitorear su rendimiento de manera continua para detectar y corregir errores y fallos. Esto garantiza que la aplicación funcione sin problemas y se mantenga al día con las últimas actualizaciones del sistema operativo, evitando problemas de rendimiento y optimizando la eficiencia general. (Microsoft, s.f)

Excelente soporte al usuario

Si está desarrollando una aplicación móvil para ser utilizada por los empleados de una empresa, es esencial garantizar un nivel adecuado de soporte al usuario. (Microsoft, s.f)

Una manera efectiva de ofrecer este respaldo es mediante una herramienta de comunicación integrada en la aplicación, como un servicio de chat en vivo. Esta funcionalidad permite a los usuarios dar retroalimentación, plantear preguntas y resolver problemas de manera sencilla. En ausencia de un servicio de atención al cliente dedicado, los chatbots basados en inteligencia artificial pueden ser una alternativa valiosa, ya que ofrecen una experiencia más personalizada al cliente. Además, opciones de autoayuda como una sección de preguntas frecuentes pueden ser útiles para que los usuarios encuentren rápidamente soluciones a problemas comunes. (Microsoft, s.f)

La experiencia de usuario de una aplicación también depende en gran medida de la navegación y la accesibilidad. Incluir características como una barra de búsqueda, información emergente con detalles sobre herramientas, accesos directos y pestañas de navegación en la interfaz de usuario facilita el uso de la aplicación y promueve su adopción en toda la organización. (Microsoft, s.f)

Integraciones incorporadas

Cuando desarrolla una aplicación, es esencial que pueda unir todos sus datos y enlazarlos con las otras plataformas que utiliza en su empresa. Por lo tanto, las aplicaciones móviles efectivas cuentan con integraciones integradas, las cuales son fundamentales para su éxito. (Microsoft, s.f)

Las funcionalidades de conectividad facilitan la sincronización de la información necesaria para obtener una comprensión más completa de los clientes. Al centralizar todo, también se reduce el riesgo de errores debido a la duplicación de datos. La unificación de los datos ayuda a los equipos (desde ventas y servicio hasta marketing) a colaborar de manera más efectiva y a superar los

compartimentos estancos de información. Esto agiliza el proceso de toma de decisiones, mejora la transparencia en toda la organización y ayuda a los equipos a trabajar de manera más eficiente. (Microsoft, s.f)

Además, otras características de conectividad, como la mensajería dentro de la aplicación o la integración del servicio al cliente, pueden agilizar la retroalimentación, mejorar la comunicación en toda la organización y ayudar a los equipos a resolver problemas comerciales de manera más rápida. (Microsoft, s.f)

Actualizaciones periódicas de la APP

Es crucial tener en cuenta que una aplicación móvil empresarial requerirá un ciclo constante de desarrollo y, por ende, de actualizaciones regulares. Asegúrate de contar con un equipo capacitado para proporcionar ese mantenimiento y desarrollar nuevas características que impulsen el crecimiento de la aplicación. Todo el contenido ofrecido a través de una aplicación móvil empresarial debe mantenerse actualizado y relevante para los usuarios, de lo contrario, perderá su valor con el tiempo. La incorporación de nuevas correcciones, funciones, desarrollos, servicios y otras mejoras hará que tu producto sea más valioso y, en consecuencia, tu aplicación también lo será. (Bluumi, s.f)

Funcionamiento offline de la APP

Es importante considerar al planificar el uso y la funcionalidad de una aplicación móvil empresarial. Aunque es comprensible que la aplicación dependa del acceso y uso de datos a través de Internet, también puede resultar útil ofrecer ciertas funcionalidades o contenido que estén disponibles sin conexión. Es necesario tener en cuenta que los usuarios no siempre tendrán acceso a Internet, aunque estas situaciones sean poco comunes. Sin embargo, es importante anticiparse a esta posibilidad según las necesidades y objetivos de tu negocio y de la aplicación. (Bluumi, s.f)

Protección sólida de datos

La seguridad es una característica esencial de cualquier aplicación, no solo una ventaja adicional. Una violación de seguridad puede exponer datos confidenciales, como información personal y financiera, poniendo en riesgo tanto a los clientes como a la empresa. Además de los costos asociados con la

limpieza y recuperación de datos, una brecha de seguridad puede ocasionar la pérdida de clientes y dañar la reputación de la marca. (Microsoft, s.f)

Por esta razón, la seguridad debe ser una prioridad principal durante el desarrollo de la aplicación. Al comenzar el proceso de desarrollo, es crucial implementar prácticas de seguridad recomendadas, como diseñar código seguro, cifrar datos, utilizar solo API autorizadas y requerir autenticación multifactor. Además, es fundamental realizar pruebas de seguridad periódicas y estar al tanto de las nuevas amenazas para detectar y abordar posibles vulnerabilidades. (Microsoft, s.f)

La protección de datos no termina una vez que se desarrolla la aplicación; es un proceso continuo. Realizar pruebas de seguridad periódicas ayuda a identificar y abordar posibles brechas en la protección de datos, lo que garantiza la seguridad de la información confidencial y fortalece la confianza del cliente en la marca. (Microsoft, s.f)

Subtema 3: Tipos de apps

APLICACIONES NATIVAS

Estas aplicaciones se desarrollan específicamente para un único sistema operativo móvil, de ahí el nombre "nativas", ya que están diseñadas para funcionar en una plataforma o dispositivo particular. La mayoría de las aplicaciones móviles contemporáneas se crean para sistemas operativos como Android o iOS. En resumen, una aplicación diseñada para Android no se puede instalar ni utilizar en un iPhone, y lo mismo ocurre a la inversa. (Herazo, s.f)

Como su nombre indica, estas aplicaciones nativas están desarrolladas específicamente para un sistema operativo. Por eso algunas aplicaciones indican "disponible solo en" seguido del nombre del sistema. Esto es común entre las dos grandes plataformas: Android e iOS. A menudo, estas compañías proporcionan las mismas funcionalidades en una aplicación, pero solo son compatibles con el sistema operativo del dispositivo. Esto da lugar a la creación de varias aplicaciones similares, pero cada una con su propia identidad. (ISIL, s.f)

Las aplicaciones nativas son desarrolladas para un sistema operativo móvil específico, como iOS o Android, utilizando el lenguaje de programación propio de cada plataforma. Esto implica que una app nativa creada para Android no puede utilizarse en un dispositivo iOS, y viceversa. Este es el tipo de aplicación móvil más común. Para que funcionen, deben descargarse e instalarse desde las tiendas de aplicaciones, como la App Store o Google Play. (Nunez, 2022)

Ventajas

- Ofrecen el mejor rendimiento. Las aplicaciones nativas son las más rápidas y superan en rendimiento a otros tipos de apps, ya que están optimizadas específicamente para el hardware y el sistema operativo del dispositivo. (Nunez, 2022)
- Acceso total e integración con las funciones de hardware del dispositivo. Las apps nativas permiten utilizar al máximo las características móviles: cámara, micrófono, lector de huellas biométrico, sensores, redes inalámbricas (Wi-Fi, Bluetooth, etc.). (Nunez, 2022)
- Pueden operar sin conexión a Internet (modo offline) si han sido diseñadas para ello. (Nunez, 2022)

Desventajas

- Altos costos de desarrollo. Para tener nuestra aplicación disponible en ambos sistemas operativos, se requieren dos líneas de desarrollo separadas, ya que el código usado en un sistema no es reutilizable en el otro. (Nunez, 2022)
- Complejidad en el desarrollo. Se necesitan equipos especializados en el lenguaje específico de cada plataforma, como Kotlin para Android y Swift para iOS. (Nunez, 2022)
- Mayor tiempo de desarrollo. Entre 4 y 6 meses. (Nunez, 2022)

Ejemplos

WhatsApp, Facebook, Twitter, Netflix, Spotify, Pokemon Go, Shazam.

APLICACIONES WEB

Las aplicaciones web son programas de software que funcionan de manera similar a las aplicaciones móviles nativas y operan en dispositivos móviles. No obstante, hay diferencias significativas entre las aplicaciones nativas y las aplicaciones web. Principalmente, las aplicaciones web se ejecutan a través de navegadores y generalmente están desarrolladas en CSS, HTML5 o JavaScript. (Herazo, s.f)

Estas aplicaciones redirigen al usuario a una URL y luego les ofrecen la opción de instalar la aplicación, creando simplemente un marcador en su página. Por este motivo, requieren un mínimo de memoria del dispositivo. (Herazo, s.f)

Como todas las bases de datos personales se almacenan en el servidor, los usuarios solo pueden utilizar la aplicación si tienen una conexión a Internet. Este es el principal inconveniente de las aplicaciones web: siempre necesitan una buena conexión a Internet para funcionar adecuadamente. De lo contrario, se corre el riesgo de ofrecer una experiencia de usuario (UX) deficiente. (Herazo, s.f)

Además, los desarrolladores no disponen de tantas API funcionales, salvo algunas funciones populares como la geolocalización. El rendimiento de la aplicación también dependerá del navegador y de la calidad de la conexión a la red. (Herazo, s.f)

Las aplicaciones web se distinguen por ser multiplataforma, es decir, puedes utilizarlas tanto en tu ordenador (a través del navegador) como en tu teléfono móvil, adaptándose al dispositivo que estés usando. (ISIL, s.f)

Estas aplicaciones están desarrolladas en HTML y CSS. Una ventaja evidente es que no ocupan espacio en la memoria de tu móvil, ya que no necesitas

descargarlas, solo acceder a ellas desde cualquier navegador como Google Chrome, Safari o Internet Explorer. Además, cumplen las mismas funciones que las aplicaciones nativas, aunque también pueden presentar fallos. (ISIL, s.f)

Las aplicaciones web o web apps son básicamente sitios web diseñados específicamente para navegadores móviles. A diferencia de las aplicaciones nativas o híbridas, no requieren ser descargadas, ya que se accede a ellas a través de un navegador web. (Nunez, 2022)

Utilizan las mismas tecnologías de desarrollo que un sitio web, como HTML, CSS o JavaScript. Así, se podría decir que son sitios web con apariencia de aplicaciones, lo que implica ciertas limitaciones. Sin embargo, con la introducción de HTML5, algunas de estas limitaciones se han superado, permitiendo el acceso a funciones del móvil como la geolocalización y las cámaras. (Nunez, 2022)

Ventajas

- Funcionalidad multiplataforma, permitiendo un desarrollo único. (Nunez, 2022)
- Desarrollo sencillo, utilizando tecnologías bien conocidas. (Nunez, 2022)
- Menor tiempo y costo de desarrollo. (Nunez, 2022)

Desventajas

- Acceso restringido a las funcionalidades del dispositivo. (Nunez, 2022)
- No es posible publicarlas en las tiendas de aplicaciones. (Nunez, 2022)
- La experiencia de usuario varía según el navegador utilizado. (Nunez, 2022)
- Es necesario tener conexión a Internet, incluso si se ha diseñado un modo offline, para acceder a actualizaciones o utilizar la aplicación por primera vez. (Nunez, 2022)

Ejemplos

Google Docs, Microsoft Office Online, Pixlr, Spotify (web), Trello, Netflix (web).

APLICACIONES HÍBRIDAS

Estas aplicaciones se desarrollan utilizando tecnologías web como JavaScript, CSS y HTML5. ¿Por qué se les llama híbridas? Las aplicaciones híbridas funcionan esencialmente como aplicaciones web envueltas en un contenedor nativo. (Herazo, s.f)

Las aplicaciones híbridas son fáciles y rápidas de desarrollar, lo cual es una ventaja evidente. Además, se obtiene una única base de código para todas las

plataformas, lo que reduce los costos de mantenimiento y simplifica el proceso de actualización. (Herazo, s.f)

Los desarrolladores también pueden utilizar diversas API para funciones como el giroscopio o la geolocalización. Sin embargo, las aplicaciones híbridas pueden tener deficiencias en cuanto a velocidad y rendimiento. Además, pueden surgir problemas de diseño, ya que la aplicación podría no verse igual en diferentes plataformas. (Herazo, s.f)

Las aplicaciones híbridas emplean la misma codificación, como HTML5, CSS o JavaScript, para ambos sistemas operativos. Esto resulta mucho más rentable en términos de costos, ya que puede representar la mitad de la inversión necesaria en comparación con las aplicaciones nativas, y se desarrolla en menos tiempo. (ISIL, s.f)

Sin embargo, la desventaja de estas aplicaciones es que no ofrecen la misma personalización que las aplicaciones nativas, un factor a considerar si estamos pensando en crear nuestra propia aplicación. (ISIL, s.f)

Las aplicaciones híbridas o multiplataforma combinan aspectos tanto de las aplicaciones nativas como de las aplicaciones web. Estas aplicaciones se crean utilizando tecnologías web como HTML, CSS y JavaScript, pero se empaquetan de manera que puedan instalarse en dispositivos móviles como cualquier otra aplicación nativa. Esto significa que podemos desarrollar una sola aplicación que funcione en varias plataformas. (Nunez, 2022)

React Native ha surgido como el marco de trabajo más popular en este campo. Permite a los desarrolladores crear aplicaciones nativas para Android e iOS utilizando JavaScript y React, lo que acelera el proceso de desarrollo y proporciona un rendimiento similar al de las aplicaciones nativas. (Nunez, 2022)

Ventajas

- Menor inversión debido al uso de lenguajes de programación más comunes, lo que significa una mayor disponibilidad de profesionales en el mercado. (Nunez, 2022)
- Compatible con múltiples plataformas, lo que permite un único proceso de desarrollo. (Nunez, 2022)
- Acceso a algunas características específicas del dispositivo móvil. (Nunez, 2022)
- Tiempos de desarrollo reducidos, con un período estimado de tres meses. (Nunez, 2022)
- Posibilidad de distribuir en tiendas de aplicaciones (App Store y Google Play), aprovechando las oportunidades de monetización y aumentando la visibilidad. (Nunez, 2022)

Desventajas

- Rendimiento inferior en comparación con las aplicaciones nativas. Suelen tener un tamaño considerable y pueden ser más lentas. (Nunez, 2022)
- Limitaciones en el acceso a las funciones del dispositivo. (Nunez, 2022)

Ejemplos

Amazon, Instagram, Uber, Gmail, Evernote.

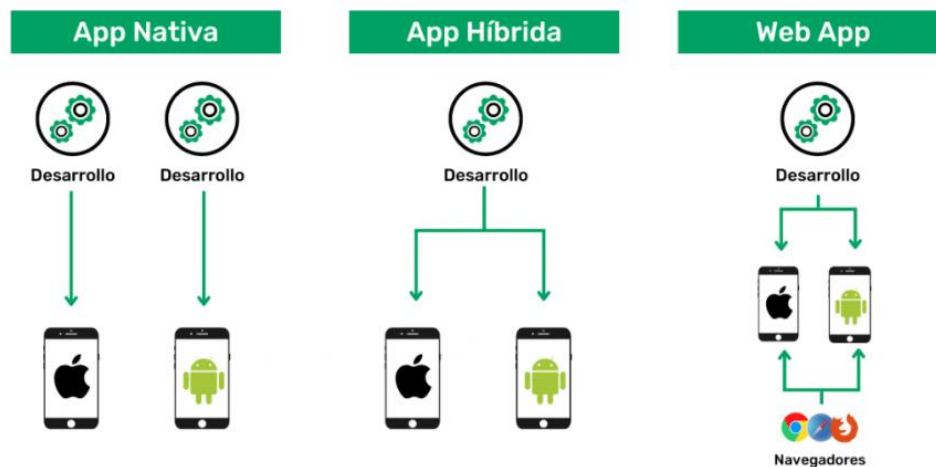


Figura 1: Tipos de apps. Fuente: Nunez, 2022. Recuperado de <https://emma.io/blog/tipos-aplicaciones-caracteristicas-ejemplos/>

Subtema 4: Beneficios de las apps móviles

A continuación, se expone algunos beneficios de las apps móviles para empresas.

Accesibilidad y comodidad para los clientes

Una de las principales ventajas de contar con una aplicación móvil radica en la accesibilidad y conveniencia que brinda a los clientes. A través de la aplicación, los usuarios pueden acceder a la información de la empresa en cualquier momento y lugar, así como realizar compras o solicitar servicios de manera directa. (De Dios, 2023)

Esta funcionalidad ofrece una notable comodidad, ya que los usuarios no están limitados a un ordenador de escritorio o portátil para acceder a los servicios ofrecidos por la empresa. En su lugar, tienen la capacidad de realizar transacciones de forma rápida y sencilla directamente desde sus dispositivos móviles. (De Dios, 2023)

Fidelización de clientes

Otro punto a favor de contar con una aplicación móvil es su capacidad para fortalecer la lealtad de los clientes mediante las notificaciones push. Estas notificaciones son útiles para informar sobre promociones exclusivas, eventos especiales y novedades en productos o servicios. (De Dios, 2023)

Además, las aplicaciones móviles pueden ser personalizadas para adaptarse a las preferencias y comportamientos de compra de los usuarios. Al analizar los datos de los usuarios, las empresas pueden ofrecer productos y servicios que se ajusten a sus necesidades individuales, lo que contribuye a fortalecer la relación y la fidelidad del cliente. (De Dios, 2023)

Aumento de la visibilidad y el alcance de la marca

Crear una aplicación móvil puede potenciar la presencia y el alcance de una marca. Al estar disponible en los resultados de búsqueda de las tiendas de aplicaciones, las empresas pueden alcanzar a una audiencia nueva de manera efectiva. (De Dios, 2023)

Mejora de la experiencia del cliente

Si las aplicaciones móviles son desarrolladas con una interfaz amigable y ajustada a las preferencias del usuario, también contribuyen a enriquecer la experiencia del cliente. Además, pueden ofrecer características adicionales como la capacidad de rastrear los pedidos en tiempo real y brindar retroalimentación a la empresa. (De Dios, 2023)

Aumento de la eficiencia operativa

Al ofrecer la opción de realizar compras directamente desde la aplicación, las compañías pueden disminuir los gastos relacionados con los procedimientos de transacción. Asimismo, las apps móviles tienen la capacidad de automatizar numerosos procesos, lo que podría disminuir la dependencia de mano de obra y aumentar la eficacia. (De Dios, 2023)

Recopilación de datos y análisis

Contar con una aplicación móvil también posibilita a las empresas recolectar datos y examinar los patrones de compra de sus clientes. Al aprovechar estos datos, las compañías pueden ajustar sus productos y servicios para satisfacer de manera más efectiva las demandas de los consumidores. Además, la información obtenida facilita la comprensión de las acciones de los clientes y la identificación de áreas para mejorar los procesos y servicios ofrecidos. (De Dios, 2023)

Mejora de la imagen de la empresa

Las empresas que crean sus propias aplicaciones móviles pueden ser percibidas como más contemporáneas e innovadoras en comparación con aquellas que no lo hacen. Además, estas aplicaciones pueden ayudarles a distinguirse de la competencia y a sobresalir en su industria. (De Dios, 2023)

Comunicación efectiva con los clientes

Las compañías tienen la opción de emplear notificaciones push para enviar comunicaciones significativas y oportunos a sus clientes de manera instantánea. Además, pueden integrar características adicionales como chat en vivo o asistencia técnica en línea para enriquecer la experiencia del cliente. (De Dios, 2023)

Generación de ingresos adicionales

Las aplicaciones también pueden generar ingresos para la empresa al ofrecer opciones de compra dentro de la aplicación, suscripciones y otros servicios premium. Asimismo, pueden ser una herramienta eficaz para impulsar las ventas al ofrecer a los consumidores una forma más accesible y conveniente de realizar transacciones. (De Dios, 2023)

Monetización adaptada al modelo de negocio

Dependiendo del producto o servicio ofrecido, es esencial determinar el modelo de monetización más apropiado. Dos enfoques fundamentales son el modelo Premium y el modelo Freemium. (De Dios, 2023)

- Modelo Premium: Los usuarios realizan un único pago al descargar la aplicación desde la tienda correspondiente, lo que implica que la app debe ofrecer un valor significativo al cliente. Este modelo es común en aplicaciones de videojuegos o aquellas con contenido y funcionalidades exclusivas. A menudo, se requiere comunicar las ventajas y la singularidad del producto a través de diversos canales para justificar el costo. (De Dios, 2023)
- Modelo Freemium: La descarga de estas aplicaciones es gratuita, y su objetivo principal es maximizar el compromiso del usuario. Este modelo se observa en muchas aplicaciones como Spotify, Netflix, plataformas de comercio electrónico, aplicaciones de noticias, entre otras. (De Dios, 2023)

Preguntas de Comprensión de la Unidad

1. ¿Cómo se diferencian las aplicaciones web o web apps de las aplicaciones nativas o híbridas, y cómo se accede a ellas?

Las aplicaciones web o web apps son esencialmente sitios web optimizados para navegadores móviles. A diferencia de las aplicaciones nativas o híbridas, no necesitan ser descargadas, ya que se accede a ellas directamente a través de un navegador web.

2. ¿Qué característica define a las aplicaciones híbridas o multiplataforma y cómo se desarrollan?

Las aplicaciones híbridas o multiplataforma combinan aspectos de las aplicaciones nativas y las aplicaciones web. Se desarrollan utilizando tecnologías web como HTML, CSS y JavaScript, pero se empaquetan de manera que puedan instalarse en dispositivos móviles como cualquier otra aplicación nativa. Esto permite desarrollar una sola aplicación que funcione en varias plataformas.

3. ¿Qué ventajas pueden obtener las empresas al desarrollar sus propias aplicaciones móviles?

Al desarrollar sus propias aplicaciones móviles, las empresas pueden ser percibidas como más contemporáneas e innovadoras en comparación con aquellas que no lo hacen. Además, estas aplicaciones pueden ayudarles a distinguirse de la competencia y a sobresalir en su industria.

4. ¿Cuál es el modelo de monetización comúnmente utilizado en aplicaciones de videojuegos y aquellas con contenido exclusivo?

El modelo de monetización comúnmente utilizado en aplicaciones de videojuegos y aquellas con contenido exclusivo es el modelo Premium, donde los usuarios realizan un único pago al descargar la aplicación desde la tienda correspondiente. Este modelo requiere que la aplicación ofrezca un valor significativo al cliente, y a menudo se necesita comunicar las ventajas y singularidades del producto a través de diversos canales para justificar el costo.

Material complementario

Los siguientes recursos complementarios son sugerencias para que se pueda ampliar la información sobre el tema trabajado, como parte de su proceso de aprendizaje autónomo:

Videos de apoyo:

- <https://www.youtube.com/watch?v=H8tykt3pKTU>
- <https://www.youtube.com/watch?v=kAaJpo2VZec>
- <https://www.youtube.com/watch?v=RudOLuV43Vk>
- <https://www.youtube.com/watch?v=L6U5pvC3VnA>

Bibliografía

- » Herazo, L. (s.f). ¿QUÉ ES UNA APLICACIÓN MÓVIL?. Recuperado de <https://anincubator.com/que-es-una-aplicacion-movil/>
- » Softcorp. (s.f). Definición y cómo funcionan las aplicaciones móviles. Recuperado de <https://servisoftcorp.com/definicion-y-como-funcionan-las-aplicaciones-moviles/>
- » Quiroz, A. (2022). ¿Qué es una aplicación móvil y para qué sirve?. Recuperado de <https://www.b2chat.io/blog/marketing/aplicacion-movil-que-para-que-sirve/>
- » Xpertosolutions. (2017). ¿Qué es una Aplicación Móvil?. Recuperado de <https://www.xpertosolutions.com/x/noticia/item/que-es-una-aplicacion-movil>
- » Microsoft. (s.f). ¿Qué hace que una aplicación móvil sea buena?. Recuperado de <https://powerapps.microsoft.com/es-mx/what-makes-a-good-app/>
- » Bluumi. (s.f). 10 características de una aplicación móvil de empresa de éxito. Recuperado de <https://bluumi.net/10-caracteristicas-una-aplicacion-movil-de-empresa-exito/>
- » ISIL. (s.f). Principales tipos de aplicaciones móviles. Recuperado de <https://isil.pe/blog/tecnologia/tipos-aplicaciones-moviles/>
- » Nunez, L. (2022). TIPOS DE APLICACIONES, CARACTERÍSTICAS, EJEMPLOS Y COMPARATIVA. Recuperado de <https://emma.io/blog/tipos-aplicaciones-caracteristicas-ejemplos/>
- » De Dios, M. (2023). Los beneficios que aportan las aplicaciones móviles a las empresas. Recuperado de <https://www.wearemarketing.com/es/blog/aplicaciones-moviles-pros-y-contras.html>



UNEMI
UNIVERSIDAD ESTATAL DE MILAGRO

Compendio del autor

COMPUTACIÓN MÓVIL

UNIDAD 1
INTRODUCCIÓN A LA COMPUTACIÓN MÓVIL

ÍNDICE

Contenido

Unidad 1: INTRODUCCIÓN A LA COMPUTACIÓN MÓVIL.....	3
<i>Tema 2: Diseño de app móviles.....</i>	<i>3</i>
<i>Objetivo:</i>	<i>3</i>
<i>Introducción:</i>	<i>3</i>
Información de los subtemas.....	4
<i>Subtema 1: El proceso de diseño y desarrollo de una app</i>	<i>4</i>
<i>Subtema 2: Categorías de aplicaciones</i>	<i>9</i>
<i>Subtema 3: Wireframes.....</i>	<i>12</i>
<i>Subtema 4: UX y principios de UX.....</i>	<i>15</i>
Preguntas de Comprensión de la Unidad	18
Material complementario	20
Bibliografía.....	21

Unidad 1: INTRODUCCIÓN A LA COMPUTACIÓN MÓVIL

Tema 2: Diseño de app móviles

Objetivo:

Identifica y aplica los conceptos sobre programación móvil, así como conocer plataformas para la creación de GUIs

Introducción:

El diseño de aplicaciones móviles ha experimentado una evolución significativa en los últimos años, en línea con el crecimiento exponencial del uso de dispositivos móviles en todo el mundo. Las aplicaciones móviles se han convertido en una parte integral de la vida diaria de las personas, sirviendo como herramientas para la comunicación, el entretenimiento, la productividad, las compras y mucho más.

En este contexto, el diseño de aplicaciones móviles se ha convertido en un campo multidisciplinario que combina principios de diseño de interfaz de usuario (UI), experiencia de usuario (UX), diseño visual, y comprensión técnica de las plataformas móviles. Los diseñadores de aplicaciones móviles no solo se centran en crear interfaces visualmente atractivas, sino que también se esfuerzan por garantizar que la experiencia de usuario sea intuitiva, eficiente y satisfactoria.

Desde la concepción de la idea hasta la implementación final, el diseño de aplicaciones móviles implica un proceso iterativo que involucra investigación de mercado, definición de usuarios y casos de uso, creación de prototipos, pruebas de usabilidad y refinamiento continuo. Los diseñadores deben comprender las necesidades y expectativas de los usuarios, así como las tendencias actuales en diseño de aplicaciones móviles, para crear productos que destaquen en un mercado cada vez más competitivo.

Información de los subtemas

Subtema 1: El proceso de diseño y desarrollo de una app

Actualmente, la mayoría de las personas utiliza aplicaciones en su vida diaria. Estas apps sirven para diversas actividades, y cada usuario tiene sus preferidas para mensajear, comprar, pagar, reservar hoteles, hacer ejercicio, meditar, entre muchas otras tareas. Sin embargo, pocos son conscientes de que, antes de que una app llegue a sus manos y les facilite la vida, ha pasado por un extenso proceso de diseño que convierte una idea en una realidad funcional. (Estefan, s.f)

El diseño y desarrollo de aplicaciones incluye desde la generación de la idea inicial hasta la evaluación después de su lanzamiento en las tiendas. A lo largo de las distintas fases, los diseñadores y desarrolladores colaboran de manera simultánea y coordinada la mayor parte del tiempo. (KuboSAS, 2023)

Importancia del diseño en el desarrollo de una app

Aunque muchos solo piensan en colores y tipografías al hablar del diseño de una app, este abarca mucho más. La gran responsabilidad del diseño radica en estructurar la funcionalidad de la aplicación, organizar la información y asegurar una transición fluida entre pantallas. (Estefan, s.f)

El diseño de una app es tan crucial como su desarrollo para alcanzar el éxito en el mercado. Para lograr un desarrollo adecuado, el diseño es indispensable y actúa como un soporte esencial durante todo el proceso de creación. Además, al ser la cara visible de la app, el diseño hace atractivo el código subyacente. (Estefan, s.f)

Un aspecto fundamental del diseño es estructurar la navegación y definir la línea visual de los diferentes elementos y pantallas. (Estefan, s.f)

En resumen, un diseño excelente permite que una aplicación tenga información bien estructurada, ofrezca una navegación y funcionalidad correctas, y que su apariencia sea atractiva para el usuario y coherente con el estilo de la marca. La combinación de todos estos elementos es clave para una experiencia de usuario única. (Estefan, s.f)

¿Cuál es el proceso de diseño de una app?

El proceso de diseño de aplicaciones móviles consta de varias etapas e iteraciones. Se inicia con un plan escalonado, pero también surgen contingencias para realizar mejoras y ajustes durante las distintas fases de prueba. (Estefan, s.f)

Además, se deben considerar las expectativas del cliente y los recursos disponibles. Este último aspecto establece los límites del desarrollo, con el objetivo de satisfacer excelentemente alguna necesidad específica de los usuarios. (Estefan, s.f)

El diseño de aplicaciones móviles comienza con una idea y culmina con su publicación en las tiendas de aplicaciones (y continúa luego con el análisis de su uso). Durante todo este proceso, diseñadores y desarrolladores trabajan de manera conjunta y sincronizada, con tareas claramente asignadas y definidas. (Estefan, s.f)

Etapas del proceso de diseño de una app

Conceptualización

Esta es la fase de ideación, donde se lleva a cabo una investigación de mercado para identificar y entender las necesidades y problemas reales de los usuarios. Luego, se verifica la viabilidad del concepto y se procede a la planificación. (Estefan, s.f)

El resultado de esta fase es una idea de aplicación que considera las necesidades y problemas de los usuarios. Esta idea se basa en una investigación preliminar y en la verificación de la viabilidad del concepto. (KuboSAS, 2023)

- Ideación
- Investigación
- Formalización de la idea

Definición

Es el momento de documentar minuciosamente quién será el usuario objetivo y cuál será la funcionalidad de la aplicación. Este proceso ayudará a definir el alcance del proyecto, determinar los niveles de complejidad en el diseño requeridos y estimar los tiempos de desarrollo necesarios. (Estefan, s.f)

En esta etapa del proceso, se describe detalladamente a los usuarios para quienes se diseñará la aplicación, utilizando metodologías como "Personas" y "Viaje del usuario". Además, se establecen las bases de la funcionalidad, lo que

determinará el alcance del proyecto y la complejidad del diseño y desarrollo de la aplicación. (KuboSAS, 2023)

- Definición de usuarios
- Definición funcional

Diseño

Esta etapa, junto con el desarrollo, es una de las más cruciales y conlleva una significativa carga horaria. Generalmente, se divide en las siguientes sub-etapas: wireframes, prototipo, pruebas y diseño visual. (Estefan, s.f)

En esta fase, se materializan conceptos y definiciones a través de wireframes inicialmente, seguidos por la creación de prototipos para pruebas. Una vez completados estos pasos, se procede a la aplicación de los estilos visuales. (Estefan, s.f)

En la etapa de diseño, se concretan los conceptos y definiciones previos, comenzando con wireframes que facilitan la creación de los primeros prototipos para ser probados con usuarios. Luego, se desarrolla un diseño visual finalizado que se proporciona al desarrollador en forma de archivos separados y pantallas modelo para la programación del código. (KuboSAS, 2023)

- Wireframes
- Prototipos
- Pruebas con usuarios
- Diseño visual

Desarrollo

En esta fase, se centra en dos aspectos fundamentales: la codificación del software de la aplicación y la corrección de errores funcionales y de diseño que puedan surgir después de las pruebas. Esto asegura el correcto funcionamiento de la aplicación. Además, el desarrollador establece la estructura en la que operará la aplicación. (Estefan, s.f)

El desarrollador se responsabiliza de transformar los diseños en funcionalidades reales y de establecer la base sobre la cual la aplicación operará. Después de completar la versión inicial, dedica tiempo considerable a identificar y corregir errores funcionales, asegurando así el correcto desempeño de la aplicación antes de su lanzamiento en las tiendas de aplicaciones. (KuboSAS, 2023)

Publicación

Una vez que la aplicación ha sido probada y completada, es el momento de lanzarla en las tiendas oficiales para su descarga. La elección de las tiendas dependerá del sistema operativo para el que se haya desarrollado la aplicación. Una vez completado este proceso, la aplicación estará disponible en el mercado, lista para que los usuarios la descarguen y la utilicen. (Estefan, s.f)

Una vez que la aplicación ha sido lanzada en las tiendas digitales, se inicia un seguimiento continuo mediante análisis de datos, estadísticas y comentarios de los usuarios. Este seguimiento permite evaluar el rendimiento y comportamiento de la aplicación, identificar errores y áreas de mejora, y preparar actualizaciones para futuras versiones. (KuboSAS, 2023)



Figura 1: Etapas del diseño de una app. Fuente: KuboSAS. 2023. Recuperado de <https://medium.com/@kubo-sas/cu%C3%A1l-es-el-proceso-de-dise%C3%B1o-y-desarrollo-de-apps-c874aa19872f>

Diseño de experiencia de usuario

Durante la fase de diseño de la experiencia de usuario (UX), el enfoque principal está en el usuario y sus interacciones con la aplicación. Aquí se definen la estructura de contenidos, los procesos y las interacciones que conformarán la experiencia general de la app. (Estefan, s.f)

Esta etapa también incluye el diseño visual de las pantallas, donde se debe integrar la identidad de la marca con las tendencias actuales de diseño de aplicaciones. Los diseños de UX se materializan mediante wireframes y prototipos, que sirven como base para el trabajo del desarrollador y permiten validar las decisiones de diseño. (Estefan, s.f)

El primer paso implica diseñar el flujo del usuario a través del "user journey", considerando todas las interacciones desde el descubrimiento de la aplicación hasta su uso habitual. Luego, se crea un mapa digital que detalla todas las acciones y tareas dentro de la app, plasmado en un prototipo de baja fidelidad que se somete a pruebas de usuarios para refinamiento. (Estefan, s.f)

A partir de los resultados de las pruebas, se realizan ajustes en el diseño, tanto en la estructura como en las interacciones, hasta obtener un prototipo que represente fielmente el producto final, aunque aún no esté completamente funcional. (Estefan, s.f)

Diseño de interfaz de usuario

La fase de diseño de la interfaz de usuario (UI) comienza una vez que se han definido la estructura, el contenido y las funcionalidades de manera avanzada. En esta etapa, el objetivo principal es garantizar que el usuario pueda utilizar la aplicación de manera intuitiva, sin necesidad de realizar un esfuerzo cognitivo excesivo. (Estefan, s.f)

El diseño de la interfaz está influenciado principalmente por el público objetivo y las tendencias actuales en diseño visual. Por lo general, el diseño UI avanza en paralelo con el diseño UX, y los diseñadores se centran en crear elementos visuales coherentes (UI-Kit). (Estefan, s.f)

Inicialmente, se definen elementos genéricos como colores, estilos gráficos, tipografías, entre otros. Posteriormente, se desarrolla un catálogo completo de elementos gráficos que incluyen botones, animaciones, formularios, módulos de contenido, tamaños de imagen, entre otros recursos visuales. (Estefan, s.f)

Una vez que el prototipo de la estructura ha sido validado, los diseñadores aplican los lineamientos del UI-Kit en el diseño de todas las pantallas, incorporando animaciones, interacciones, imágenes y otros elementos visuales. El objetivo final es crear una interfaz sencilla y fácil de usar, ya que la satisfacción del usuario depende en gran medida de la experiencia de usuario. Para lograr esto, es fundamental tener en cuenta al usuario a lo largo de todo el proceso de diseño, creando así la identidad visual de la aplicación. (Estefan, s.f)

Subtema 2: Categorías de aplicaciones

Aplicaciones de juegos

La categoría más popular de aplicaciones móviles son los juegos, y es sorprendente la cantidad de usuarios que los instalan en sus dispositivos. Las empresas están dedicando cada vez más tiempo y recursos a la creación de juegos y versiones móviles de los juegos clásicos, ya que este mercado es extremadamente lucrativo. (Herazo, s.f)

Según un estudio reciente, los juegos móviles representan el 33% de todas las descargas de aplicaciones, el 74% del gasto de los consumidores y el 10% de todo el tiempo dedicado al uso de aplicaciones. Juegos exitosos como Candy Crush Saga o Angry Birds son reconocidos a nivel mundial. (Herazo, s.f)

Aplicaciones empresariales o de productividad

Las aplicaciones empresariales o de productividad son una parte significativa del mercado actual, ya que cada vez más personas confían en sus dispositivos móviles para realizar tareas complejas mientras están en movimiento. Estas aplicaciones permiten a los usuarios llevar a cabo una variedad de funciones, como reservar boletos, enviar correos electrónicos o monitorear su progreso laboral, todo desde la comodidad de sus teléfonos inteligentes o tabletas. (Herazo, s.f)

Diseñadas para aumentar la eficiencia y reducir costos, las aplicaciones comerciales ofrecen una amplia gama de funcionalidades que van desde la compra de suministros de oficina hasta la contratación de personal gerencial, lo que facilita la gestión de diversas tareas comerciales de manera rápida y conveniente. (Herazo, s.f)

Aplicaciones educativas

Las aplicaciones educativas abarcan aquellas herramientas móviles diseñadas para facilitar a los usuarios la adquisición de nuevas habilidades y conocimientos. Por ejemplo, aplicaciones como Duolingo, que se centran en la enseñanza de idiomas, han ganado una gran popularidad debido a su flexibilidad y capacidad de adaptación a las necesidades individuales de los usuarios. (Herazo, s.f)

Además, las aplicaciones de juegos educativos son especialmente útiles para los niños, ya que combinan el aprendizaje con la diversión. Estas aplicaciones no solo son populares entre los estudiantes, sino también entre los profesores,

quienes las utilizan para mejorar su proceso de enseñanza y ampliar su propio conocimiento en diversas áreas. (Herazo, s.f)

Aplicaciones de estilo de vida

Esta amplia categoría de aplicaciones engloba diversas áreas como compras, moda, probadores virtuales, entrenamiento, citas y programas de dieta. Su enfoque principal está dirigido a diferentes aspectos del estilo de vida personal. (Herazo, s.f)

Aplicaciones de comercio móvil

Las aplicaciones de compras más conocidas, como Amazon o eBay, proporcionan a los usuarios móviles la misma experiencia que sus versiones de escritorio. Estas aplicaciones ofrecen a los clientes un acceso práctico a productos y opciones de pago sin problemas, garantizando así una experiencia de compra óptima. (Herazo, s.f)

Aplicaciones de entretenimiento

Estas aplicaciones ofrecen a los usuarios la posibilidad de transmitir contenido de video, buscar eventos, chatear o ver contenido en línea. Ejemplos destacados son las aplicaciones de redes sociales como Facebook o Instagram. Asimismo, las aplicaciones de transmisión de video como Netflix o Amazon Prime Video gozan de una enorme popularidad entre los usuarios a nivel mundial. (Herazo, s.f)

Estas aplicaciones suelen fomentar la participación de los usuarios al notificarles sobre actualizaciones y nuevos contenidos añadidos. (Herazo, s.f)

Aplicaciones de utilidad

A menudo pasan desapercibidas debido a su naturaleza práctica y funcional. De hecho, las aplicaciones de utilidad suelen tener sesiones de usuario más breves, ya que las personas las utilizan para realizar tareas específicas y luego continúan con sus actividades. Entre los tipos más populares de aplicaciones de utilidad se encuentran los lectores de códigos de barras, los rastreadores y las aplicaciones de atención médica. (Herazo, s.f)

Aplicaciones de viaje

Estas aplicaciones tienen como objetivo principal facilitar los viajes para los usuarios. Transforman los teléfonos inteligentes o tabletas en diarios de viaje y guías que ayudan a los usuarios a descubrir todo lo que necesitan saber sobre el lugar que están visitando. (Herazo, s.f)

Subtema 3: Wireframes

¿Qué es un prototipo?

Un prototipo de aplicación constituye una representación preliminar de una aplicación o página web antes de su desarrollo completo. Funciona como un esquema inicial del proyecto, posibilitando la realización de pruebas, la recepción de comentarios y la implementación de mejoras. (Tangram, 2020)

Estos prototipos son empleados para introducir ideas, dirigir el diseño de la experiencia del usuario, identificar áreas que requieren mayor atención y facilitar la visualización de la interacción entre diferentes componentes del sistema. (Tangram, 2020)

En consecuencia, se convierten en herramientas fundamentales para los diseñadores de experiencia de usuario (UX) durante el proceso de desarrollo de aplicaciones, permitiéndoles validar conceptos rápidamente sin la necesidad de dedicar tiempo a la codificación completa de las ideas. (Tangram, 2020)

Crear una aplicación es un desafío considerable que implica una cuidadosa planificación y una exhaustiva investigación para garantizar el desarrollo de un producto funcional. (Tangram, 2020)

El proceso de desarrollo de una aplicación sin la creación previa de un prototipo puede resultar en errores significativos, pérdida de tiempo y costos adicionales. Los prototipos no solo son cruciales durante el desarrollo, sino que también son útiles para generar interés entre posibles inversores o clientes. (Tangram, 2020)

Un prototipo de aplicación es una versión simplificada que comunica la idea principal y el propósito del proyecto de manera clara y concisa. La construcción de un prototipo antes de comprometer recursos en el desarrollo permite a los desarrolladores probar sus ideas y comprender qué enfoques funcionan mejor en la práctica, en contraposición a la teoría. (Tangram, 2020)

Sketch

Pensemos en el sketch como el primer esbozo que se realiza para plasmar el proyecto digital que tenemos en mente. Son los trazos iniciales, las primeras ideas que se plasman en papel, en un tablero o incluso en una servilleta. (Morales, 2014)

El sketch captura las ideas generales del proyecto y plantea algunas preguntas clave:

¿Qué contenido y servicios queremos incluir en el proyecto?

¿Cómo será la estructura de navegación?

¿Dónde estarán ubicadas las funciones de ayuda para los usuarios?

¿Deberíamos integrar servicios de redes sociales?

El sketch no requiere un trabajo conceptual exhaustivo. Aquí lo que importa es la creatividad, la experiencia y la visión del diseñador. (Morales, 2014)

Los prototipos de baja fidelidad se caracterizan por su rápida y informal elaboración, a menudo realizada con lápiz y papel o mediante programas que imitan estas funciones de manera digital. Estos prototipos son útiles para obtener una comprensión rápida y general del proyecto. (del Olmo, 2018)

Wireframe

Para entender el concepto, imaginemos una representación básica (en tonos de gris) del proyecto, donde se detallan con mayor precisión (Morales, 2014):

- Áreas de contenido
- Uso de elementos HTML (estructura semántica)
- Funcionalidades de navegación y ayuda
- Flujos de navegación (cómo se conectan las unidades de información)

Es importante tener en cuenta que los wireframes no incluyen elementos gráficos, como tipografías, colores o cualquier otro componente relacionado. En otras palabras, no proporcionan un adelanto de estilos visuales ni de iconografía. (Morales, 2014)

El wireframe marca el "primer paso significativo" en el diseño del proyecto digital. Aquí es crucial tener un entendimiento claro de los servicios, contenidos y público objetivo. (Morales, 2014)

Estos prototipos son más detallados e integran un inventario completo de contenidos, representando el esquema y contenido de cada página con todos sus elementos (enlaces, listas, formularios, etiquetas, etc.). Además, incluyen notas que proporcionan explicaciones relevantes. (del Olmo, 2018)

Mockup

El mockup representa una fase más avanzada del diseño gráfico y comunicativo del proyecto, ofreciendo una visualización más detallada de (Morales, 2014):

- Contenidos, que pueden ser textos simulados.
- Paleta de colores, basada en la identidad institucional, la misión y el público objetivo del proyecto.
- Estilos CSS.
- Dimensiones de áreas de contenido y servicios.
- Iconografía.

Es importante destacar que el mockup incorpora elementos del sketch y del wireframe, siendo cada uno una progresión del anterior en el proceso de diseño. (Morales, 2014)

Los prototipos de alta fidelidad se caracterizan por ofrecer un nivel avanzado de detalle en el proceso interactivo de una o varias tareas del proyecto web o de la aplicación. Esto permite al cliente tener una idea más precisa de cómo será el proyecto antes de su implementación. (del Olmo, 2018)

Prototipo

Finalmente, el prototipo representa un nivel de detalle avanzado del proyecto digital. Aquí se pueden identificar y operar elementos como sistemas de navegación, paleta de colores, iconografía, aplicaciones de declaraciones CSS y la experiencia del usuario en su totalidad, incluyendo servicios de ayuda, búsqueda e interacción, entre otros. (Morales, 2014)

Este producto constituye el paso previo al desarrollo final y la presentación del proyecto. El prototipo es fundamental para identificar, a través de pruebas con usuarios beta, las posibles dificultades del proyecto antes de su implementación definitiva. (Morales, 2014)

Subtema 4: UX y principios de UX

User experience o UX se refiere a la percepción y sensaciones que experimenta el público al interactuar con tus productos, servicios o cualquier aspecto relacionado con tu empresa. Su propósito es proporcionar soluciones agradables, intuitivas, atractivas y eficientes a los usuarios, con el objetivo de garantizar una experiencia satisfactoria con tu marca. (Zendesk, 2024)

Comprender el concepto de user experience y aplicar las mejores prácticas de UX en tu empresa puede ser una estrategia altamente efectiva para mejorar la satisfacción de tus clientes. Es importante tener en cuenta que el 88% de los consumidores en línea afirma que no regresaría a un sitio web donde haya tenido una mala experiencia de usuario. (Zendesk, 2024)

Beneficios

- Satisfacción de las necesidades de los clientes mediante soluciones más integrales y accesibles. (Zendesk, 2024)
- Mayor competitividad en el segmento de mercado para la empresa. (Zendesk, 2024)
- Aumento en la satisfacción del cliente. (Zendesk, 2024)
- Incremento en las conversiones y, por consiguiente, en las ventas. (Zendesk, 2024)
- Mejora en la retención de clientes y mayor capacidad para atraer nuevos. (Zendesk, 2024)

Principios

Centrarse en el usuario

La experiencia de usuario se fundamenta en colocar al usuario en el centro de todo. Consideramos que los productos serán utilizados por personas (aunque cada vez más productos se orientan también a máquinas), y tanto los aspectos visuales como los funcionales deben satisfacer sus necesidades. (Torres, 2021)

Este enfoque nos lleva a pensar en las interacciones con el producto, en las tareas que se llevarán a cabo y en cómo abordar los puntos problemáticos o áreas de dificultad. (Torres, 2021)

Priorizar al usuario implica no solo diseñar soluciones, sino también observar y escuchar de manera continua sus acciones. Esto nos permite identificar áreas de mejora, como simplificar un proceso de tres pasos a dos o colocar elementos dentro del campo visual para facilitar las tareas. Para lograrlo, se requiere

realizar investigaciones de UX para comprender quiénes son los usuarios y cómo utilizan el producto. (Torres, 2021)

Coherencia

Con este principio, aseguramos que la experiencia de usuario de nuestros productos sea consistente, independientemente del dispositivo utilizado. Aunque cada dispositivo tenga sus propias particularidades, la experiencia al acceder a nuestras clases es esencialmente uniforme. (Torres, 2021)

Aunque en una tableta el listado de clases se encuentre en la parte derecha, mientras que en el celular esté debajo del vídeo, la experiencia es prácticamente idéntica: los botones son los mismos, la interacción con el vídeo es uniforme y el diseño visual es coherente en todos los aspectos. Colores, formas, iconos... todo está diseñado de manera coherente. La falta de coherencia en la experiencia puede provocar problemas para tu producto, como tasas de conversión bajas y altas tasas de abandono. (Torres, 2021)

Jerarquía

Este principio es fundamental pero desafiante de aplicar. Al igual que organizamos nuestras tareas diarias por prioridad, también debemos priorizar y jerarquizar los elementos de nuestro diseño, otorgando mayor énfasis a lo más importante. (Torres, 2021)

La jerarquía nos permite resaltar contenidos importantes mediante el uso de colores llamativos o tamaños de fuente más grandes. Sin embargo, otro factor crucial es el orden. (Torres, 2021)

Los usuarios que utilizamos el alfabeto latino estamos acostumbrados a leer de arriba hacia abajo y de izquierda a derecha, por lo que generalmente escaneamos la información antes de profundizar en ella. (Torres, 2021)

Accesibilidad

El producto debe ser accesible para todos, independientemente de sus habilidades o condiciones. Este aspecto es especialmente crucial, ya que garantiza que tu aplicación sea fácil de usar sin importar las limitaciones físicas de los usuarios. Este principio asegura que tu contenido tenga suficiente contraste para que las personas con deficiencias en la percepción del color puedan interpretar la información de manera adecuada. (Torres, 2021)

Asimismo, permite que las personas con discapacidades de movilidad puedan interactuar con el producto utilizando métodos de interacción alternativos al ratón, como la voz. Es importante tener en cuenta que cada vez más países cuentan con regulaciones en este ámbito, por lo que es probable que tu producto deba cumplir con ciertos estándares mínimos de accesibilidad para personas con discapacidades visuales, auditivas o físicas. (Torres, 2021)

Además, la accesibilidad también abarca el aspecto lingüístico, por lo que es fundamental considerar este aspecto para garantizar la inclusión de todos los usuarios, independientemente de su idioma. La accesibilidad no solo es una obligación legal, sino también un valor que promueve la inclusión y la equidad. (Torres, 2021)

Usabilidad

Este principio final se centra en la usabilidad, es decir, en la capacidad del producto para ser utilizado de manera fácil y eficiente. Para lograr esto, es necesario aplicar todos los principios anteriores, de modo que cualquier usuario pueda encontrar nuestro producto adecuado para las tareas que necesita realizar de manera simple y rápida. (Torres, 2021)

Al definir claramente a las personas a las que se dirige nuestro producto, podemos crear una aplicación que sea accesible, con una información debidamente organizada y coherente, sin importar cómo el usuario elija interactuar con ella. Esto implica empatizar con el usuario, ponerse en su lugar y comprender sus necesidades y expectativas. (Torres, 2021)

Preguntas de Comprensión de la Unidad

1. **¿Qué tipo de funciones pueden llevar a cabo los usuarios a través de las aplicaciones empresariales o de productividad, y cómo han evolucionado estas aplicaciones en el mercado actual?**

Las aplicaciones empresariales o de productividad ofrecen a los usuarios la capacidad de realizar una variedad de funciones, como reservar boletos, enviar correos electrónicos o monitorear su progreso laboral, todo desde sus teléfonos inteligentes o tabletas. Estas aplicaciones son una parte significativa del mercado actual, ya que cada vez más personas confían en sus dispositivos móviles para llevar a cabo tareas complejas mientras están en movimiento. Con la evolución del mercado, estas aplicaciones han mejorado su accesibilidad y funcionalidad para adaptarse a las necesidades cambiantes de los usuarios.

2. **¿Cuáles son algunas características comunes de las aplicaciones de entretenimiento y qué ejemplos conocidos existen?**

Las aplicaciones de entretenimiento permiten a los usuarios transmitir contenido de video, buscar eventos, chatear o ver contenido en línea. Ejemplos notables incluyen aplicaciones de redes sociales como Facebook o Instagram, así como aplicaciones de transmisión de video como Netflix o Amazon Prime Video, que son muy populares entre los usuarios a nivel mundial.

3. **¿Qué elementos no se incluyen en los wireframes y cuál es su importancia en el proceso de diseño digital?**

Los wireframes no incluyen elementos gráficos como tipografías, colores o cualquier otro componente visual. Su objetivo principal es establecer la estructura y el esquema de contenido de un proyecto digital sin entrar en detalles visuales. Esto es crucial porque marca el primer paso significativo en el diseño del proyecto, donde se debe tener un entendimiento claro de los servicios, contenidos y público objetivo antes de avanzar en el proceso de diseño.

4. **¿Cuál es el enfoque principal en el diseño de la experiencia de usuario (UX)?**

El enfoque principal en el diseño de la experiencia de usuario (UX) es centrar al usuario como la máxima prioridad. Esto implica considerar que los productos serán utilizados por personas, aunque también haya una

creciente tendencia hacia productos orientados a máquinas. Tanto los aspectos visuales como los funcionales deben satisfacer las necesidades del usuario para garantizar una experiencia positiva y efectiva.

5. ¿Cómo afecta la coherencia en el diseño visual de una aplicación a la experiencia del usuario en diferentes dispositivos?

La coherencia en el diseño visual de una aplicación asegura una experiencia de usuario uniforme, incluso en diferentes dispositivos. Aunque la disposición de elementos pueda variar según el dispositivo, como en el caso de las tabletas y los teléfonos celulares, donde el listado de clases puede ubicarse en diferentes lugares, la experiencia sigue siendo prácticamente idéntica. La uniformidad en los botones, la interacción con el contenido y el diseño visual coherente en colores, formas e iconos son elementos clave. La falta de coherencia puede generar problemas como tasas de conversión bajas y altas tasas de abandono del producto.

6. ¿Qué posibilidad brinda el diseño accesible de una aplicación para personas con discapacidades de movilidad?

El diseño accesible de una aplicación permite que las personas con discapacidades de movilidad puedan interactuar con el producto utilizando métodos de interacción alternativos al ratón, como la voz. Además, con la creciente regulación en muchos países, es probable que los productos deban cumplir con estándares mínimos de accesibilidad para personas con discapacidades visuales, auditivas o físicas.

Material complementario

Los siguientes recursos complementarios son sugerencias para que se pueda ampliar la información sobre el tema trabajado, como parte de su proceso de aprendizaje autónomo:

Videos de apoyo:

- https://www.youtube.com/watch?v=yqt_MEOGQEE
- <https://www.youtube.com/watch?v=TxoVkS4ewK8>
- <https://www.youtube.com/watch?v=zUZSSkRQXQQ>
- <https://www.youtube.com/watch?v=IYW0zxVCTao>
- <https://www.youtube.com/watch?v=HYUCtX8VuE0>

Bibliografía

- » KuboSAS. (2023). ¿CUÁL ES EL PROCESO DE DISEÑO Y DESARROLLO DE APPS?. Recuperado de <https://medium.com/@kubos-sas/cu%C3%A1l-es-el-proceso-de-dise%C3%B1o-y-desarrollo-de-apps-c874aa19872f>
- » Estefan, C. (s.f). El proceso de diseño de una app: de la ideación a la acción. Recuperado de <https://thinkupsoft.com/blog/es/el-proceso-de-diseno-de-una-app/>
- » Herazo, L. (s.f). ¿QUÉ ES UNA APLICACIÓN MÓVIL?. Recuperado de <https://anincubator.com/que-es-una-aplicacion-movil/>
- » Tangram. (2020). Prototipo De Una Aplicación ¿Qué Es y Para Qué Sirve?. Recuperado de <https://tangramconsulting.es/noticias/prototipo-de-una-aplicacion-que-es-y-para-que-sirve>
- » Morales, J. (2014). DIFERENCIAS ENTRE SKETCH, WIREFRAME, MOCKUP Y PROTOTIPO. Recuperado de <https://e-lexia.com/blog/diferencias-entre-sketch-wireframe-mockup-y-prototipo/>
- » Del Olmo, G. (2018). ¿Qué son sketches, mockups, wireframes y prototipos?. Recuperado de <https://www.artcreateweb.com/blog/index.php/2018/07/23/que-son-sketchs-mockups-wireframes-y-prototipos/>
- » Zendesk. (2024). ¿Qué es UX y para qué sirve?. Recuperado de <https://www.zendesk.com.mx/blog/ux-que-es/>
- » Torres, D. (2021). 5 principios básicos del UX Design. Recuperado de <https://platzi.com/blog/principios-basicos-ux-design/>



UNEMI
UNIVERSIDAD ESTATAL DE MILAGRO

Compendio del autor

COMPUTACIÓN MÓVIL

UNIDAD 2
MULTIPLATAFORMA DESARROLLO DE APLICACIONES MÓVILES

ÍNDICE

Contenido

Unidad 2: MULTIPLATAFORMA DESARROLLO DE APLICACIONES MÓVILES.....	3
<i>Tema 1: Diseño de app móviles.....</i>	<i>3</i>
<i>Objetivo:</i>	<i>3</i>
<i>Introducción:</i>	<i>3</i>
Información de los subtemas.....	5
<i>Subtema 1: ¿Qué es una app multiplataforma?</i>	<i>5</i>
<i>Subtema 2: Características de una app multiplataforma</i>	<i>7</i>
<i>Subtema 3: Herramientas para el desarrollo de app multiplataforma.....</i>	<i>9</i>
<i>Subtema 4: Ejemplos de apps multiplataforma.....</i>	<i>14</i>
Preguntas de Comprensión de la Unidad	16
Material complementario	18
Bibliografía.....	19

Unidad 2: MULTIPLATAFORMA DESARROLLO DE APLICACIONES MÓVILES

Tema 1: Diseño de app móviles

Objetivo:

Definir los diferentes conceptos para la comprensión de aplicaciones multiplataforma.

Introducción:

Las aplicaciones multiplataforma han transformado la manera en que se desarrollan y despliegan las aplicaciones móviles y web en la actualidad. En lugar de crear versiones separadas para cada sistema operativo como iOS, Android o Windows, las aplicaciones multiplataforma utilizan un único código base que puede ejecutarse en múltiples plataformas.

Este enfoque ofrece varias ventajas significativas. En primer lugar, reduce drásticamente el tiempo y los recursos necesarios para el desarrollo. En lugar de tener equipos de desarrollo separados para cada plataforma, un solo equipo puede trabajar en un único código base que se adapte a todas ellas. Esto también significa que las actualizaciones y correcciones de errores pueden implementarse de manera más rápida y consistente en todas las plataformas, lo que mejora la experiencia del usuario y la eficiencia operativa.

Además, las aplicaciones multiplataforma permiten a las empresas llegar a una audiencia más amplia de manera más efectiva. Pueden lanzar sus productos simultáneamente en múltiples plataformas, alcanzando a usuarios de iOS, Android, Windows y otras plataformas con una sola versión de la aplicación. Esto no solo simplifica la gestión y el soporte, sino que también maximiza el alcance del mercado sin comprometer la calidad ni la funcionalidad.

Tecnologías como React Native, Flutter, Xamarin, Ionic y otras han democratizado el desarrollo multiplataforma al proporcionar herramientas y frameworks poderosos que facilitan la creación de aplicaciones robustas y visualmente atractivas. Estas plataformas ofrecen acceso nativo a características del sistema operativo y hardware, garantizando que las

aplicaciones multiplataforma puedan ofrecer la misma experiencia de usuario fluida y eficiente que las aplicaciones nativas, sin compromisos significativos en rendimiento o funcionalidad.

Información de los subtemas

Subtema 1: ¿Qué es una app multiplataforma?

Las aplicaciones multiplataforma son programas diseñados utilizando un solo lenguaje de programación, lo que facilita su exportación y visualización en cualquier dispositivo, sin importar el sistema operativo. Gracias a este enfoque, solo se requieren ajustes mínimos para adaptarlas completamente a diferentes dispositivos, como teléfonos móviles, computadoras o tabletas. (ABAMOBILE, s.f)

Desarrollar apps multiplataforma es una opción conveniente para los desarrolladores, en contraste con las aplicaciones nativas. Estas últimas se crean de forma específica para cada sistema operativo, utilizando diferentes lenguajes de programación para cada uno. Aunque las apps nativas ofrecen un rendimiento superior y un alto grado de personalización, es crucial analizar cada proyecto y idea para determinar la opción más adecuada. (ABAMOBILE, s.f)

El desarrollo de aplicaciones multiplataforma puede realizarse de dos formas:

- Por un lado, mediante el uso de lenguajes de desarrollo web como HTML5, CSS o JavaScript. En este enfoque, se desarrolla una aplicación como si fuera una página web, pero con la capacidad de adaptarse a cualquier dispositivo. (ABAMOBILE, s.f)
- Por otro lado, también es posible crear aplicaciones móviles multiplataforma utilizando herramientas de rendering a nativo. En este caso, se emplean frameworks como Flutter o React Native, los cuales generan código nativo para cada sistema operativo. Esto permite que la experiencia del usuario sea similar a la de una aplicación nativa. (ABAMOBILE, s.f)

Una aplicación multiplataforma es aquella que puede funcionar en diversas plataformas. Este tipo de aplicaciones suele ser más sencillo de desarrollar y mantener en comparación con las nativas, ya que se crean utilizando una única base de código. (UFV, s.f)

Para los desarrolladores, las aplicaciones multiplataforma ofrecen una ventaja significativa, ya que les permiten desarrollar una aplicación una sola vez y luego implementarla en múltiples dispositivos. Esto resulta en un ahorro de tiempo y dinero tanto en costos de desarrollo como de mantenimiento. (UFV, s.f)

En la actualidad, existe una amplia gama de herramientas que facilitan considerablemente a los desarrolladores la creación de estas aplicaciones. Sin

estas herramientas, las aplicaciones tendrían que desarrollarse como si fueran páginas web, aplicaciones híbridas o aplicaciones nativas independientes. (UFV, s.f)

Los dispositivos móviles constituyen más de la mitad del tráfico online mundial, mientras que la otra mitad corresponde a la navegación en ordenadores y tablets. Estas estadísticas subrayan la importancia de que las aplicaciones online se lancen con un enfoque multiplataforma. (UFV, s.f)

Usuarios, clientes y empleados buscan la comodidad de poder utilizar aplicaciones de diversas maneras. Desean la libertad de acceder a una aplicación en cualquier momento: durante sus desplazamientos al trabajo, caminando o en el jardín. Asimismo, desean disfrutar de una experiencia óptima en pantalla grande cuando están usando su ordenador. (UFV, s.f)

El desarrollo multiplataforma implica crear una aplicación que pueda distribuirse en varias plataformas, como dispositivos móviles, tabletas u ordenadores. Existen diversas bibliotecas de desarrollo de software que facilitan este proceso, como Ionic, React Native y Flutter. Estas herramientas permiten codificar una vez y desplegar en cualquier plataforma. Manteniendo una única base de código, los equipos de desarrollo pueden reducir significativamente el tiempo de creación. (UFV, s.f)

Tradicionalmente, el desarrollo de aplicaciones se hacía de manera personalizada para cada plataforma, lo que implicaba escribir el código de la aplicación en el lenguaje nativo de iOS para lanzar una aplicación en esta plataforma. (UFV, s.f)

Aunque las aplicaciones nativas se siguen construyendo de esta manera y hay razones para continuar haciéndolo, en términos de velocidad y relación calidad-precio, el desarrollo multiplataforma es una opción superior. (UFV, s.f)

Subtema 2: Características de una app multiplataforma

Las aplicaciones multiplataforma han ganado popularidad en el ámbito del desarrollo de software gracias a su capacidad para operar en diversos sistemas operativos y dispositivos. Estas aplicaciones son altamente versátiles y poseen una serie de características distintivas que resultan atractivas tanto para desarrolladores como para usuarios. (Tangram, 2020)

Versatilidad en Plataformas

Las aplicaciones multiplataforma se distinguen por su habilidad para operar sin inconvenientes en varios sistemas operativos, como Android, iOS, Windows y macOS. Esta versatilidad marca un avance importante en el desarrollo de software, al eliminar la necesidad de crear aplicaciones únicas para cada plataforma específica. (Tangram, 2020)

Eliminación de la Barrera del Sistema Operativo

En el pasado, los desarrolladores enfrentaban desafíos importantes al crear aplicaciones para diversas plataformas operativas. Cada sistema tenía sus propias normativas y requisitos únicos, lo que requería equipos de desarrollo separados para cada uno. (Tangram, 2020)

Esta situación generaba fragmentación y complicaba el proceso, pues los desarrolladores debían dominar diferentes lenguajes de programación y ajustar continuamente sus aplicaciones para cumplir con los estándares de cada plataforma. (Tangram, 2020)

Ahorro de Tiempo y Recursos

Con las aplicaciones multiplataforma, ha habido un cambio significativo en este escenario. Los desarrolladores pueden ahora escribir un solo código base que funcione en múltiples plataformas, evitando así la necesidad de empezar desde cero para cada sistema operativo. (Tangram, 2020)

Este enfoque no solo acorta el tiempo de desarrollo, sino que también optimiza el uso de recursos. Los equipos de desarrollo pueden centrarse en mejorar y perfeccionar un único código base en lugar de distribuir sus esfuerzos entre múltiples códigos específicos para cada plataforma. (Tangram, 2020)

Ampliación del Alcance del Usuario

La capacidad de funcionar en múltiples plataformas no solo es beneficiosa para los desarrolladores, sino también para los usuarios finales. Al eliminar las barreras entre los sistemas operativos, las aplicaciones multiplataforma pueden alcanzar a una audiencia más amplia. (Tangram, 2020)

Los usuarios de Android, iOS, Windows y macOS pueden acceder y disfrutar de las mismas características y funcionalidades, sin importar el dispositivo que estén utilizando. Esto proporciona una experiencia de usuario coherente y unificada, lo que contribuye a aumentar la satisfacción del usuario y la adopción de la aplicación. (Tangram, 2020)

Fomentando la Innovación

La capacidad de funcionar en múltiples plataformas también impulsa la innovación en el desarrollo de aplicaciones. Los desarrolladores pueden concentrarse en crear características innovadoras en lugar de enfrentarse a las complejidades técnicas de cada plataforma. Esto facilita un enfoque más creativo y experimental en el diseño y la funcionalidad de la aplicación, lo que conduce a la creación de aplicaciones más diversas y enriquecedoras para los usuarios. (Tangram, 2020)

Ventajas

- Las aplicaciones multiplataforma operan en todos los sistemas operativos de manera inherente. (Palos, s.f)
- Representan un ahorro significativo en los costes de desarrollo. (Palos, s.f)
- El tiempo de desarrollo es menor en comparación con la creación de aplicaciones nativas para cada plataforma. (Palos, s.f)
- Ofrecen las mismas funcionalidades que las aplicaciones nativas. (Palos, s.f)
- Facilitan el acceso directo mediante la descarga de la aplicación en el dispositivo. (Palos, s.f)
- Se integran perfectamente con el hardware y el software de cada dispositivo, garantizando una interfaz de usuario uniforme. (Palos, s.f)
- Utilizan lenguajes de programación ampliamente reconocidos. (Palos, s.f)
- Posibilitan un lanzamiento más rápido al mercado. (Palos, s.f)

Subtema 3: Herramientas para el desarrollo de app multiplataforma

La fuerza motriz detrás del desarrollo multiplataforma es la creación de aplicaciones que no estén restringidas a un entorno digital específico, con el objetivo de ampliar su alcance a una audiencia más amplia. Esto se opone al enfoque de desarrollo de aplicaciones nativas, que implica crear diferentes versiones de una aplicación para cada plataforma. (Jacinto, 2023)

Inicialmente, el desarrollo de aplicaciones móviles para varios entornos implicaba la creación de un backend separado para cada plataforma o la implementación de un backend compatible con múltiples plataformas. Esto resultaba en procesos lentos y costosos, lo que llevó a las organizaciones a crear aplicaciones nativas para cada plataforma, las cuales no se podían reutilizar en otras. (Jacinto, 2023)

React Native

Desde su lanzamiento en 2015 por Facebook, React ha experimentado un crecimiento continuo y ha consolidado su posición en el mundo del desarrollo de software. En la actualidad, es raro encontrar una empresa de desarrollo de software que no utilice React Native. (Jacinto, 2023)

React se ha convertido en la elección preferida para la creación de aplicaciones dinámicas e innovadoras. Su capacidad para crear prototipos rápidamente y su alta velocidad inicial lo hacen muy popular. Además, con características fáciles de aprender, ha ganado la confianza de grandes empresas como Facebook y Uber, estableciéndose como una herramienta de desarrollo de aplicaciones móviles líder en la industria. (Jacinto, 2023)

Características destacadas:

- Desarrollo multiplataforma para iOS y Android
- Utilización de elementos nativos
- Integración perfecta de terceros, incluidos módulos nativos y JavaScript
- Ejecución de código en tiempo real a través de recarga en caliente y en vivo
- Enfoque centrado en la interfaz de usuario. (Jacinto, 2023)

Flutter

Flutter, un kit de desarrollo de software de código abierto desarrollado por Google, se basa en el lenguaje de programación Dart. Su asequibilidad, rápida

capacidad de desarrollo y conjunto completo de funciones lo hacen adecuado para grandes empresas y startups por igual. (Jacinto, 2023)

Esta herramienta multiplataforma es excelente para crear aplicaciones para Android, iOS, Linux, Mac y Windows. Flutter emplea un modelo de programación reactiva de última generación que permite crear interfaces de usuario altamente eficientes y receptivas en dispositivos móviles. (Jacinto, 2023)

Características destacadas:

- Interfaz de usuario personalizable mediante widgets expresivos
- Recarga en caliente para una actualización rápida de la interfaz de usuario
- Detección de errores integrada durante el desarrollo
- Utilización de Swift, Java y Objective-C para funciones nativas
- Compatible con Google Firebase para escalabilidad del backend. (Jacinto, 2023)

Xamarin

Xamarin, una herramienta de desarrollo de aplicaciones multiplataforma creada por Microsoft, permite a los desarrolladores utilizar C# y el sólido marco .NET para crear aplicaciones nativas en Android, iOS y Windows. (Jacinto, 2023)

Con Xamarin, se reduce el tiempo de comercialización y la incidencia de errores al utilizar una única base de código para varias plataformas. Similar a Flutter, esta herramienta ahorra tiempo y esfuerzo al evitar el desarrollo de aplicaciones nativas por separado para cada plataforma mediante el uso de lenguajes nativos. (Jacinto, 2023)

Características destacadas:

- Acceso completo a la API nativa a través del framework Mono
- Codificación simplificada con IntelliSense específico para cada plataforma
- Ciclo simplificado de creación, prueba y despliegue con C++
- Incorpora indexación de aplicaciones y enlaces profundos
- Creación de bibliotecas compartidas de alto rendimiento. (Jacinto, 2023)

Ionic

Ionic es un marco de trabajo excepcional para el desarrollo de aplicaciones móviles. Está construido con tecnologías web como CSS, HTML5 y SASS, lo que lo hace ideal para crear aplicaciones híbridas. Además, gracias a su completo SDK de código abierto, los desarrolladores pueden enviar actualizaciones directamente a los usuarios. (Jacinto, 2023)

Es fácil de utilizar y se puede integrar con AngularJS para crear aplicaciones avanzadas. Ionic simplifica el desarrollo de aplicaciones móviles con una biblioteca de componentes, herramientas y gestos optimizados para dispositivos móviles. La interfaz de línea de comandos ofrece funciones prácticas como registro, emuladores y recarga en vivo. (Jacinto, 2023)

Características destacadas:

- Desarrollo de aplicaciones móviles para las principales tiendas de aplicaciones con una única base de código
- Emulación de interfaz de usuario nativa con integración de SDK
- Integración perfecta con herramientas como Sentry, LogRocket, etc.
- Compatible con HTML, JavaScript y otras tecnologías
- Ofrece una base de código unificada, componentes de interfaz de usuario y acceso nativo completo. (Jacinto, 2023)

Appery

Appery.io es una plataforma de desarrollo móvil de bajo código que simplifica la creación de aplicaciones móviles híbridas, aplicaciones en línea y aplicaciones web progresivas (PWA). Esta plataforma incluye un servidor de compilación que genera código HTML5, jQuery Mobile, AngularJS, Bootstrap y Apache Cordova. Además, ofrece un generador de código para desarrollar aplicaciones para iOS, Android, Windows Phone y HTML5. (Jacinto, 2023)

Appery.io proporciona una amplia gama de funciones, incluido alojamiento, una base de datos MongoDB, notificaciones push, código de servidor JavaScript y un proxy seguro. También ofrece compatibilidad con alojamiento HTML en su propia nube y con proveedores de alojamiento de terceros. El constructor de aplicaciones incluye pestañas para configurar la aplicación, definir el modelo y almacenamiento, desarrollar páginas, diálogos, plantillas, estilos, CSS, servicios, JavaScript y componentes personalizados. (Jacinto, 2023)

Características destacadas:

- Funciones de arrastrar y soltar para una creación intuitiva de aplicaciones
- Desarrollo de aplicaciones móviles sin necesidad de programación
- Pruebas de compatibilidad para garantizar un rendimiento óptimo en diferentes dispositivos
- Funcionalidades de depuración integradas para facilitar la identificación y solución de errores. (Jacinto, 2023)

Sencha

Sencha ofrece a los desarrolladores marcos multiplataforma en Java y JavaScript para crear, sincronizar y actualizar aplicaciones web para múltiples dispositivos de manera simultánea. Es una herramienta completa de desarrollo de aplicaciones móviles que cuenta con más de 100 componentes de interfaz de usuario probados que se pueden incorporar en las aplicaciones, así como con pruebas integrales y tablas de visualización de datos para monitorear las métricas de la aplicación. La función "Themer" permite a los desarrolladores crear temas reutilizables basados en temas de Ext React, Angular, JS e iOS. (back4app, s.f)

Unity

Unity es reconocido como un potente motor de juegos multiplataforma que ofrece a los desarrolladores las herramientas necesarias para crear aplicaciones y juegos 3D con gráficos avanzados. Con capacidad para exportar a 17 plataformas diferentes, incluyendo web, Windows, Xbox, iOS, Android, Linux, PlayStation, entre otras, Unity brinda una amplia flexibilidad en el desarrollo. (back4app, s.f)

Además de sus capacidades de desarrollo, Unity ofrece un rastreador de métricas que permite recopilar análisis importantes sobre los usuarios y compartir la aplicación en redes sociales. Para los desarrolladores novatos, Unity Connect proporciona una valiosa función que les permite conectarse con otros desarrolladores de Unity para obtener ayuda y orientación en sus implementaciones. (back4app, s.f)

Alpha Anywhere

Alpha Anywhere se destaca como una de las potentes herramientas de desarrollo de aplicaciones multiplataforma a nivel empresarial, compatible con los principales sistemas operativos móviles y de escritorio. Esto permite a los desarrolladores y organizaciones introducir nuevas soluciones SaaS en el mercado de manera rápida y rentable. (back4app, s.f)

Como una plataforma de bajo código, Alpha Anywhere ofrece las herramientas y recursos necesarios para crear aplicaciones basadas en datos, validar entradas de datos, generar informes personalizados, enviar informes por correo electrónico, entre otras funciones. Una de las características más solicitadas de Alpha Anywhere es su capacidad de implementación, ya sea en las instalaciones, en la nube o para acceso a través de dispositivos móviles, según las necesidades específicas. (back4app, s.f)

Firestore

Firestore, la principal oferta de Google, es una plataforma de backend como servicio que ofrece a los desarrolladores todas las herramientas y servicios necesarios para crear aplicaciones de alta calidad sin la necesidad de programar en el servidor. (back4app, s.f)

Es extremadamente versátil y puede adaptarse a una variedad de escenarios para cumplir con diferentes requisitos. Respaldado por Google, se integra sin problemas con varios servicios de Google y de terceros, proporcionando a los desarrolladores funcionalidades adicionales como alojamiento de sitios web estáticos y dinámicos, seguimiento de métricas, entre otros. (back4app, s.f)

Subtema 4: Ejemplos de apps multiplataforma

Facebook

Hace unos años, fue un destacado ejemplo de una aplicación multiplataforma, aunque con el tiempo optaron por desarrollar aplicaciones nativas para cada tipo de dispositivo. Este gigante de las redes sociales tenía una aplicación que era accesible desde dispositivos móviles o tabletas con sistemas operativos iOS o Android, así como desde cualquier tipo de dispositivo, independientemente de su tamaño de pantalla. (Palos, s.f)

Netflix

Esta plataforma de streaming de vídeo también dispone de una aplicación multiplataforma que permite ver nuestras series favoritas en todos los sistemas operativos. (Palos, s.f)

Hangouts

Google es conocido por su experiencia en el desarrollo de aplicaciones multiplataforma, y esta aplicación de mensajería es un excelente ejemplo de ello. Permite mantener conversaciones desde cualquier lugar utilizando una única aplicación que funciona en todos los dispositivos. (Palos, s.f)

Evernote

Es posible que esta aplicación no te resulte familiar, pero si eres aficionado a hacer listas y tomar notas, probablemente la conozcas. Es otro ejemplo de una aplicación multiplataforma que te permite acceder a ella desde cualquier dispositivo. (Océano Atlántico, s.f)

Instagram

Instagram, una popular red social, ha ganado una gran popularidad entre los jóvenes al ofrecer la posibilidad de compartir fotos con otros usuarios y recibir comentarios o "me gusta" de seguidores. Los usuarios pueden agregar etiquetas o hashtags a sus fotos para clasificarlas fácilmente y facilitar las búsquedas por temas específicos. (Soto, 2020)

Inicialmente lanzada exclusivamente para iPhone, Instagram debutó en la App Store de Apple en octubre de 2010. No fue hasta abril de 2012 que la aplicación se lanzó para Android, logrando más de un millón de descargas en sus primeras 24 horas. El éxito fue tan notable que Facebook mostró interés en la aplicación y finalmente la adquirió. (Soto, 2020)

Slack

Slack es una plataforma de mensajería diseñada para empresas que facilita la conexión entre personas y la gestión de información crucial. Transforma la comunicación organizacional al reunir a los equipos y mejorar su colaboración de manera unificada. (slack, s.f)

Pinterest

Pinterest es una red social centrada en compartir elementos visuales organizados en tableros virtuales, llamados "tablones", donde los usuarios pueden publicar imágenes y vídeos sobre diversos temas de interés. Los usuarios crean perfiles personales para compartir y organizar sus tablones, que funcionan como paneles de corcho digital. Otros usuarios pueden interactuar con el contenido guardándolo, conocido como "pinearlo", o participar en tablones colaborativos administrados por varios usuarios. Es una plataforma útil para recopilar ideas y organizar inspiración en áreas como proyectos, decoración del hogar y moda, entre otros. (Armetrics, s.f)

Preguntas de Comprensión de la Unidad

1. ¿Qué implica el desarrollo multiplataforma según el texto proporcionado?

El desarrollo multiplataforma implica crear una aplicación que pueda ser distribuida en múltiples plataformas como dispositivos móviles, tabletas y ordenadores. Herramientas como Ionic, React Native y Flutter facilitan este proceso al permitir a los desarrolladores codificar una vez y desplegar en cualquier plataforma. Esto se logra manteniendo una única base de código, lo que resulta en una reducción significativa del tiempo de desarrollo.

2. ¿Cómo han cambiado las aplicaciones multiplataforma el enfoque de desarrollo de los desarrolladores según el texto?

Con las aplicaciones multiplataforma, los desarrolladores pueden ahora escribir un solo código base que funcione en múltiples plataformas, evitando así la necesidad de comenzar desde cero para cada sistema operativo. Esto ha permitido un cambio significativo al simplificar el proceso de desarrollo y mejorar la eficiencia en la creación de aplicaciones para diferentes entornos digitales.

3. ¿Cómo fomenta la capacidad de funcionar en múltiples plataformas la innovación en el desarrollo de aplicaciones según el texto?

La capacidad de funcionar en múltiples plataformas permite a los desarrolladores concentrarse en crear características innovadoras en lugar de lidiar con las complejidades técnicas de cada plataforma. Esto facilita un enfoque más creativo y experimental en el diseño y la funcionalidad de la aplicación, resultando en la creación de aplicaciones más diversas y enriquecedoras para los usuarios.

4. ¿Por qué React ha ganado popularidad en el desarrollo de aplicaciones según el texto proporcionado?

React se ha convertido en la elección preferida para el desarrollo de aplicaciones debido a su capacidad para crear prototipos rápidamente, su alta velocidad inicial y características fáciles de aprender. Estas ventajas han ganado la confianza de grandes empresas como Facebook y Uber, estableciéndolo como una herramienta líder en la industria del desarrollo de aplicaciones móviles.

5. ¿Qué ofrece Firebase a los desarrolladores según el texto proporcionado?

Firestore proporciona una plataforma de backend como servicio que incluye herramientas y servicios completos para que los desarrolladores creen aplicaciones de alta calidad sin necesidad de programar en el servidor.

Material complementario

Los siguientes recursos complementarios son sugerencias para que se pueda ampliar la información sobre el tema trabajado, como parte de su proceso de aprendizaje autónomo:

Videos de apoyo:

- <https://www.youtube.com/watch?v=rRAfUcV7KpA>
- https://www.youtube.com/watch?v=ayK-B_1YBDc
- <https://www.youtube.com/watch?v=bRqbHpXklPU>
- <https://www.youtube.com/watch?v=MLu6kpOnB8o>
- <https://www.youtube.com/watch?v=i1uoJCsAxWc>

Bibliografía de apoyo:

- <https://reactnative.dev/docs/getting-started>
- <https://docs.flutter.dev/>
- <https://ionicframework.com/docs>

Bibliografía

- » ABAMOBILE. (s.f). Apps multiplataforma. Qué son y sus características. Recuperado de <https://abamobile.com/web/apps-multiplataforma-que-son-y-caracteristicas/>
- » Universidad Francisco de Vitoria. (s.f). ¿QUÉ ES UNA APLICACIÓN MULTIPLATAFORMA?. Recuperado de <https://www.ufv.es/cetys/blog/que-es-una-aplicacion-multiplataforma/>
- » Universidad Francisco de Vitoria. (s.f). APLICACIONES MULTIPLATAFORMA: LO QUE NECESITAS SABER. Recuperado de <https://www.ufv.es/cetys/blog/aplicaciones-multiplataforma-lo-que-necesitas-saber/>
- » Tangram. (2020). Características de las aplicaciones multiplataforma. Recuperado de <https://tangramconsulting.es/noticias/caracteristicas-de-las-aplicaciones-multiplataforma>
- » Palos, A. (s.f). Desarrollo de aplicaciones Multiplataforma: Conoce todas las posibilidades. Recuperado de <https://scoreapps.com/blog/desarrollo-aplicaciones-multiplataforma/>
- » Jacinto, A. (2023). LAS MEJORES HERRAMIENTAS DE DESARROLLO DE APLICACIONES MÓVILES QUE NECESITAS EN 2023. Recuperado de <https://www.startechup.com/es/blog/mobile-app-development-tools-in-2023/>
- » back4app. (s.f). Las 10 mejores herramientas de desarrollo de aplicaciones multiplataforma. Recuperado de <https://blog.back4app.com/es/las-10-mejores-herramientas-de-desarrollo-de-aplicaciones-multiplataforma/>
- » Océano Atlántico. (s.f). 4 ejemplos para saber qué es una aplicación multiplataforma. Recuperado de <https://fp.oceanoatlantico.org/noticias/que-es-una-aplicacion-multiplataforma/>
- » Soto, J. (2020). ¿Qué es Instagram y para qué sirve?. Recuperado de <https://www.geeknetic.es/Instagram/que-es-y-para-que-sirve>
- » Slack. (s.f). ¿Qué es Slack?. Recuperado de <https://slack.com/intl/es-es/help/articles/115004071768-%C2%BFQu%C3%A9-es-Slack>
- » Arimetrics. (s.f). Qué es Pinterest. Recuperado de <https://www.arimetrics.com/glosario-digital/pinterest>



UNEMI
UNIVERSIDAD ESTATAL DE MILAGRO

Compendio del autor

COMPUTACIÓN MÓVIL

UNIDAD 2
MULTIPLATAFORMA DESARROLLO DE APLICACIONES MÓVILES

ÍNDICE

Contenido

Unidad 2: MULTIPLATAFORMA DESARROLLO DE APLICACIONES MÓVILES.....	3
<i>Tema 2: Plataformas y entornos de desarrollo móvil.....</i>	<i>3</i>
<i>Objetivo:</i>	<i>3</i>
<i>Introducción:</i>	<i>3</i>
Información de los subtemas.....	4
<i>Subtema 1: Lenguajes para desarrollo de app móviles</i>	<i>4</i>
<i>Subtema 2: Android, IOs y Windows Phone</i>	<i>10</i>
<i>Subtema 3: Plataformas de desarrollo móvil</i>	<i>15</i>
<i>Subtema 4: Entornos de desarrollo móvil</i>	<i>18</i>
Preguntas de Comprensión de la Unidad	21
Material complementario	23
Bibliografía.....	24

Unidad 2: MULTIPLATAFORMA DESARROLLO DE APLICACIONES MÓVILES

Tema 2: Plataformas y entornos de desarrollo móvil

Objetivo:

Define los diferentes conceptos para la comprensión de aplicaciones multiplataforma y las diferentes herramientas de entorno de Desarrollo.

Introducción:

En el universo del desarrollo móvil, las plataformas y entornos de desarrollo desempeñan un papel fundamental al facilitar la creación de aplicaciones para dispositivos móviles como smartphones y tablets. Estas plataformas no solo proporcionan herramientas y recursos técnicos necesarios para construir aplicaciones, sino que también establecen los estándares y directrices que los desarrolladores deben seguir para garantizar la compatibilidad, seguridad y experiencia del usuario final.

Las dos principales plataformas móviles, Android de Google y iOS de Apple, cada una con sus propias características y requisitos, dominan el mercado. Para desarrollar aplicaciones para estas plataformas, los desarrolladores utilizan entornos de desarrollo integrados (IDE, por sus siglas en inglés) específicos como Android Studio para Android y Xcode para iOS. Estos entornos proporcionan herramientas avanzadas como editores de código, compiladores, depuradores y simuladores de dispositivos que permiten a los desarrolladores diseñar, desarrollar, probar y depurar sus aplicaciones de manera eficiente.

Además, los entornos de desarrollo móvil permiten a los desarrolladores mantenerse al día con las actualizaciones del sistema operativo. Esto es crucial para asegurarse de que las aplicaciones continúen funcionando sin problemas en nuevas versiones del sistema operativo, evitando problemas de compatibilidad y mejorando la experiencia del usuario final.

Información de los subtemas

Subtema 1: Lenguajes para desarrollo de app móviles

Un lenguaje de programación es un sistema formal que utiliza una serie de símbolos y códigos para que el programador pueda dar instrucciones a una máquina. En el contexto de un smartphone, el programador usa estos lenguajes para controlar el dispositivo. (López, 2021)

De esta manera, el experto en programación se encarga de diseñar y desarrollar las aplicaciones móviles que posteriormente usaremos en nuestra vida cotidiana. Hay una gran variedad de lenguajes de programación para dispositivos móviles, algunos de los cuales son específicos para ciertos sistemas operativos. (López, 2021)

Java

Para empezar, si estás considerando un lenguaje de programación para aplicaciones móviles, es probable que estés pensando en Java. Esto se debe a que, por ejemplo, Android fue desarrollado utilizando este lenguaje. (López, 2021)

Java se distingue por su velocidad, facilidad de uso y amplia gama de posibilidades. Es un excelente lenguaje de programación, ideal tanto para aplicaciones móviles como para el desarrollo de software personalizado. Por ello, hay numerosos ejemplos de aplicaciones desarrolladas en Java, como Twitter, Netflix y Uber, entre otras. (López, 2021)

Java es un lenguaje de programación versátil y muy utilizado, especialmente en el desarrollo de aplicaciones móviles. Desde su lanzamiento en 1995, Java se ha convertido en el principal lenguaje de programación para desarrollar aplicaciones Android, que representan una gran parte de las aplicaciones móviles a nivel mundial. Su popularidad en este ámbito se debe a sus características esenciales, como la programación orientada a objetos, su sintaxis comprensible y sus extensas bibliotecas. Estas cualidades constituyen la base de un entorno de desarrollo de aplicaciones potente que ha influido en la industria moderna de aplicaciones móviles. (Nader, 2023)

Además de su importancia en el desarrollo de aplicaciones Android, Java también se emplea para crear aplicaciones móviles multiplataforma mediante soluciones híbridas como Apache Cordova y Xamarin. Estos marcos permiten a los desarrolladores escribir el código una sola vez y desplegarlo en múltiples

plataformas, lo que aumenta la eficiencia y reduce los costos del desarrollo de aplicaciones móviles. (Nader, 2023)

Ventajas

- Compatibilidad multiplataforma: el principio de Java "escribir una vez, ejecutar en cualquier lugar" permite a los desarrolladores escribir aplicaciones una vez y ejecutarlas en cualquier plataforma que soporte Java Runtime Environment (JRE). Esta característica reduce significativamente el tiempo y los costos asociados con el desarrollo de aplicaciones, especialmente cuando se trata de dirigirse a múltiples plataformas. (Nader, 2023)
- Programación orientada a objetos (POO): Java sigue un paradigma de programación orientada a objetos, lo que simplifica la organización y reutilización del código. Este enfoque permite a los desarrolladores mantener y modificar aplicaciones con un impacto mínimo en el resto del código base. (Nader, 2023)
- Gran ecosistema: el amplio ecosistema de Java incluye numerosas herramientas, marcos y bibliotecas, lo que facilita el desarrollo rápido de aplicaciones con muchas funcionalidades. Además, Java cuenta con una extensa comunidad de desarrolladores que apoya a los usuarios a enfrentar desafíos complejos en el desarrollo de aplicaciones. (Nader, 2023)
- Amplias bibliotecas: Java dispone de un extenso conjunto de bibliotecas y API que simplifican tareas complejas y aceleran el desarrollo de aplicaciones móviles. Estas bibliotecas proporcionan fragmentos de código optimizados y listos para usar, lo que reduce la carga de trabajo de los desarrolladores y acorta el tiempo de desarrollo. (Nader, 2023)

Kotlin

Por otro lado, uno de los lenguajes de programación más utilizados para dispositivos móviles Android es, sin duda, Kotlin. ¿Por qué? La respuesta es simple: Kotlin cuenta con un código muy intuitivo, sencillo y eficaz. (López, 2021)

Fue diseñado para funcionar junto con Java; sin embargo, desde hace algunos años, Google lo ha recomendado para el desarrollo de aplicaciones Android, siendo su lenguaje preferido. Kotlin es moderno y también destaca por su escalabilidad. (López, 2021)

Kotlin es un lenguaje de programación similar a Java, pero con mejoras y características adicionales que lo hacen más fácil y eficiente para los desarrolladores. Al igual que Java, Kotlin es uno de los lenguajes oficiales para desarrollar aplicaciones móviles en Android. (Brutti, 2023)

Fue creado por JetBrains, una empresa de software, y se presentó oficialmente en 2011. Su objetivo principal era simplificar la escritura de código, reducir errores comunes y aumentar la productividad de los programadores. (Brutti, 2023)

Desde su lanzamiento, Kotlin ha ganado una enorme popularidad en la comunidad de desarrollo de software y se ha convertido en un lenguaje ampliamente adoptado, especialmente en el desarrollo de aplicaciones móviles para Android. Su crecimiento y aceptación continúan aumentando constantemente, convirtiéndose en el lenguaje preferido por la mayoría de los desarrolladores para programar en la plataforma Android. (Brutti, 2023)

Ventajas

- Kotlin es un lenguaje de programación de código abierto, lo que significa que puedes utilizarlo en varios proyectos sin restricciones ni necesidad de adquirir una licencia específica. (Brutti, 2023)
- Kotlin tiene una curva de aprendizaje sencilla. Su sintaxis es clara y fácil de entender, lo que lo convierte en un excelente lenguaje para aquellos que están aprendiendo a programar. En comparación con lenguajes como Java, Kotlin tiene una curva de aprendizaje más suave. (Brutti, 2023)
- Con Kotlin, puedes lograr lo mismo que en otros lenguajes escribiendo menos líneas de código. Esto no solo hace que tu código sea más claro y fácil de seguir, sino que también es especialmente beneficioso en proyectos grandes, donde la cantidad de código puede llegar a ser considerable, con cientos o incluso miles de líneas. (Brutti, 2023)
- Kotlin proporciona una mayor productividad al permitirte escribir menos líneas de código en menos tiempo y con menos errores. Esto puede incrementar la eficiencia en el desarrollo de aplicaciones de manera significativa. (Brutti, 2023)

Python

Las aplicaciones móviles desarrolladas con Python se distinguen por su código conciso. Python simplifica el trabajo para programadores e ingenieros informáticos al requerir menos líneas de código que lenguajes como Java, por ejemplo. (López, 2021)

Para desarrollar aplicaciones móviles con Python, es común utilizar frameworks como Kivy o kits de desarrollo de aplicaciones multiplataforma que están diseñados para utilizar Python. (López, 2021)

Python es un lenguaje de programación versátil que facilita la creación de una amplia variedad de aplicaciones, incluidas aquellas para la web, así como

sistemas operativos como Android e iOS. También se utiliza en sectores avanzados de la tecnología, como la inteligencia artificial. (Bambu, 2022)

Python es conocido por ser un lenguaje de propósito general, fácil de aprender y compatible con Windows, MacOS y Linux. Es especialmente adecuado para el desarrollo de aplicaciones móviles debido a su sintaxis clara y poderosa capacidad para resolver problemas. Además, cuenta con una extensa biblioteca que facilita la conexión con APIs y módulos externos. (Bambu, 2022)

Existen frameworks en Python que aceleran el desarrollo de aplicaciones móviles, permitiendo la creación rápida de prototipos y la incorporación gradual de funcionalidades a lo largo del proceso de desarrollo. (Bambu, 2022)

JavaScript

Si estás considerando desarrollar una aplicación multiplataforma, JavaScript podría ser el lenguaje de programación móvil ideal que estás buscando. Es conocido por su rapidez, versatilidad y simplicidad, destacando especialmente por su funcionalidad. Además, JavaScript es efectivo en mejorar la experiencia del usuario mediante su capacidad para crear interactividad en las aplicaciones móviles. (López, 2021)

JavaScript es un lenguaje de programación ampliamente utilizado para desarrollar páginas web dinámicas e interactivas. Es el lenguaje de scripting más popular en la web y es utilizado por los desarrolladores para agregar funcionalidad a sitios web y aplicaciones. (Ortíz, 2023)

Se utiliza para crear interfaces de usuario interactivas, desarrollar aplicaciones en el lado del servidor, establecer conexiones con bases de datos y realizar muchas otras tareas. Su código es fundamental en el desarrollo web moderno y está presente en prácticamente todas las aplicaciones web hoy en día. (Ortíz, 2023)

JavaScript es utilizado para desarrollar aplicaciones móviles a través de frameworks como React Native. Este framework permite a los desarrolladores crear aplicaciones para iOS y Android utilizando JavaScript como el lenguaje principal de programación. (Ortíz, 2023)

Características

- JavaScript es un lenguaje multiplataforma que puede ser utilizado con diversos frameworks y plataformas. Además, puede ser ejecutado en cualquier máquina, independientemente de su sistema operativo. (Ortíz, 2023)

- JavaScript es un lenguaje interpretado y orientado a objetos. Esto significa que permite la definición de tipos de datos y sus operaciones. Cada objeto es distinto, lo cual facilita la organización y prevención de errores en el código. Al ser interpretado, puede ejecutarse directamente en el navegador sin necesidad de ser compilado previamente. (Ortíz, 2023)
- JavaScript es un lenguaje de programación de alto nivel y de tipado débil. Su sintaxis es cercana al lenguaje natural, lo que facilita su comprensión y uso. En cuanto al tipado, JavaScript es débilmente tipado o no tipado, lo que significa que no es necesario que el desarrollador especifique el tipo de una variable al declararla. (Ortíz, 2023)
- JavaScript cuenta con una amplia variedad de librerías y frameworks, como React, Angular o Node.js, que permiten a los desarrolladores crear proyectos de manera más rápida y con menos líneas de código. Estas librerías también facilitan el uso de tecnologías como AJAX o JSON, entre otras. (Ortíz, 2023)
- Además, JavaScript ha evolucionado constantemente para adaptarse a las demandas del mercado, ofreciendo nuevas funcionalidades y herramientas. Por ejemplo, en los últimos años se han introducido nuevas APIs que simplifican tareas como la geolocalización o la funcionalidad de arrastrar y soltar elementos. También, gracias a los web workers, es posible ejecutar funciones en segundo plano sin afectar al rendimiento general de la página. (Ortíz, 2023)
- JavaScript es un lenguaje asíncrono, lo que significa que puede ejecutar múltiples tareas simultáneamente sin afectar el rendimiento de la página. Esta característica es fundamental para desarrollar aplicaciones web interactivas, ya que permite ejecutar procesos en segundo plano mientras el usuario interactúa con otros elementos de la página. (Ortíz, 2023)
- JavaScript es capaz de trabajar con JSON (JavaScript Object Notation), que es un formato ligero utilizado para el intercambio de datos. Permite crear, analizar y manipular objetos de manera eficiente, lo que resulta muy útil en el desarrollo de aplicaciones web dinámicas. Además, JavaScript puede conectarse a bases de datos, ya sea a través de APIs específicas o utilizando frameworks y bibliotecas diseñadas para facilitar esta integración. (Ortíz, 2023)

Swift

Asimismo, un ejemplo destacado de lenguaje de programación para iPhone es Swift. Este lenguaje, creado por Apple, no solo se utiliza para desarrollar aplicaciones para iOS, sino que también es compatible con Windows, Linux y macOS. Swift es un lenguaje de código abierto. (López, 2021)

El lenguaje de programación Swift es el estándar utilizado actualmente para desarrollar aplicaciones digitales en entornos iOS y macOS. Es el lenguaje ideal para programar aplicaciones que deben ser compatibles con los dispositivos de Apple. (Tokio School, s.f)

Características

- La seguridad en Swift se fundamenta en la reducida probabilidad de cometer errores durante su escritura. Gracias a un código más limpio y una estructura de variables menos propensa a incorrecciones, junto con gestiones automáticas, se minimiza la presencia de errores o problemas. (Tokio School, s.f)
- Swift, como lenguaje de programación, está diseñado para reducir al mínimo la posibilidad de errores. Esto implica que el desarrollo de aplicaciones basado en Swift tiende a ser más estable y seguro en comparación con otras opciones. (Tokio School, s.f)
- La velocidad de desarrollo es una característica distintiva y crucial del lenguaje de programación Swift. Swift fue creado para mejorar y en algunos casos reemplazar a lenguajes como Objective-C. No solo lo ha superado en eficiencia, sino que también se considera superior a usar C o C++, en los cuales está fundamentado. Hoy en día, Swift sigue siendo reconocido como el lenguaje de programación más rápido para cualquier desarrollo en iOS. (Tokio School, s.f)
- Una característica destacada de Swift es su continua evolución. Este lenguaje de programación se presenta como una evolución de los lenguajes previos, como Objective-C. Siguiendo esta filosofía, Swift continúa evolucionando constantemente para aprovechar los avances en nuevas tecnologías. Esto permite desarrollar aplicaciones más complejas y funcionales para los usuarios. (Tokio School, s.f)

Subtema 2: Android, IOs y Windows Phone

Android

Cuando mencionamos a Google, hablamos del gigante de Internet no solo por su popular buscador, sino también por la amplia gama de servicios y herramientas que ofrece, siendo además propietario del sistema operativo más utilizado en smartphones y tablets a nivel mundial. Vamos a explorar la historia de Android, su definición exacta y las diferentes versiones del sistema. (Adeva, 2023)

Aunque el término Android puede sonar desconocido para algunas personas, la mayoría sabe si su dispositivo móvil utiliza Android o el sistema iOS de Apple. Estos dos sistemas operativos dominan el mercado de dispositivos móviles en la actualidad, aunque Android también se encuentra en otros tipos de dispositivos electrónicos. (Adeva, 2023)

Android es un sistema operativo móvil diseñado principalmente para dispositivos con pantalla táctil como teléfonos inteligentes y tablets. Sin embargo, también se encuentra en otros dispositivos como relojes inteligentes, televisores e incluso en sistemas multimedia de algunos modelos de automóviles. Desarrollado por Google, se basa en el kernel de Linux y otros softwares de código abierto, facilitando el uso de una amplia variedad de aplicaciones de manera sencilla. (Adeva, 2023)

Inicialmente, Android fue desarrollado por Android Inc., que luego fue adquirida por Google en 2005. El sistema fue presentado en 2007 como parte del avance hacia estándares abiertos en dispositivos móviles. El código fuente principal de Android es conocido como Android Open Source Project (AOSP). Android se destaca por ser el sistema operativo móvil más utilizado a nivel mundial, con una cuota de mercado que superaba el 90% en 2018, posicionándolo considerablemente por encima de su competidor directo, iOS. (Adeva, 2023)

Desde su lanzamiento, Android ha experimentado numerosas actualizaciones a lo largo del tiempo. Cada una de estas actualizaciones, conocidas como versiones del sistema operativo, han corregido errores, introducido nuevas funciones y añadido soporte para tecnologías emergentes. Un aspecto notable de estas versiones es que cada una ha sido nombrada con un nombre en código relacionado con postres o dulces, y han seguido estrictamente un orden alfabético con nombres como "Donut", "Eclair", "Froyo", "Gingerbread", "Honeycomb", "Ice Cream Sandwich", "Jelly Bean", "KitKat", "Lollipop", "Marshmallow", "Nougat", "Oreo", "Pie". (Adeva, 2023)

Aplicaciones Android

Una de las características distintivas del sistema operativo Android es su gran libertad para el desarrollo y distribución de aplicaciones. Android cuenta con una extensa tienda de aplicaciones donde cualquier usuario puede descargar e instalar apps en su smartphone o tablet, y donde los desarrolladores pueden subir sus creaciones para que estén disponibles para todos los usuarios del sistema. (Adeva, 2023)

Google Play es la plataforma o tienda en línea desarrollada por el gigante de las búsquedas, donde se pueden encontrar una amplia variedad de aplicaciones para Android. Esta aplicación está instalada por defecto en todos los dispositivos Android, proporcionando un acceso fácil al catálogo completo de aplicaciones que incluyen música, juegos, noticias, clima, educación, compras, salud, utilidades, mapas, deportes, bancos, entre otros. (Adeva, 2023)

Para usar la tienda de aplicaciones, es necesario asociar una cuenta de Gmail. Sin embargo, debido a la popularidad de Android, existen varias tiendas de aplicaciones alternativas a Google Play donde también se pueden encontrar numerosas herramientas y aplicaciones. (Adeva, 2023)

IOs

iOS, abreviatura de "iPhone Operating System" en inglés, es un sistema operativo propietario desarrollado por Apple Inc. Este sistema está diseñado para dispositivos como smartphones, tablets, televisores y reproductores de música como iPhone, iPad y iPod. (Gabit, s.f)

Al igual que Android, iOS permite la instalación de aplicaciones o apps para ampliar las funcionalidades de los dispositivos. Estas aplicaciones pueden incluir herramientas de oficina, productividad, almacenamiento en la nube, educativas, juegos y mucho más. Desde la perspectiva tecnológica, iOS no es compatible con tecnologías como Java o Flash, lo que puede limitar el acceso a componentes de algunas páginas web y aplicaciones específicas. La instalación de aplicaciones se realiza principalmente a través de la tienda oficial de Apple, conocida como App Store, que ofrece aproximadamente un millón de aplicaciones actualmente. (Gabit, s.f)

En términos de diferencias con Android, la principal distinción es que iOS está restringido a dispositivos fabricados por Apple, mientras que Android puede ser instalado en una amplia variedad de dispositivos de diferentes fabricantes. Además, iOS se destaca por ofrecer un nivel de seguridad más alto, especialmente en lo que respecta a virus o malware. Las aplicaciones en iOS deben pasar por un proceso de validación riguroso y el firmware del sistema está diseñado para ser más robusto en comparación con Android. Sin embargo, los

desarrolladores aficionados pueden enfrentar más obstáculos al publicar aplicaciones o contenidos multimedia en iOS. (Gabit, s.f)

Características

iOS, el sistema operativo de Apple para dispositivos como iPhones, iPads y otros productos de la marca, ofrece características distintivas que lo hacen popular entre los usuarios. (ADnaliza, s.f)

- App Store: iOS cuenta con la App Store, una tienda de aplicaciones con millones de apps disponibles para descargar. Todas estas aplicaciones pasan por un estricto proceso de revisión para garantizar su calidad y seguridad antes de estar disponibles para los usuarios. (ADnaliza, s.f)
- Siri: iOS incluye Siri, un asistente personal inteligente que utiliza el procesamiento de lenguaje natural para realizar diversas tareas y responder preguntas de los usuarios. Siri es conocido por su capacidad de comprensión y su integración con diversas funciones del dispositivo. (ADnaliza, s.f)
- Seguridad y privacidad: iOS se destaca por sus sólidas medidas de seguridad y privacidad. Cada aplicación en iOS debe obtener el permiso explícito del usuario para acceder a información específica, como la ubicación o los contactos. Apple ha implementado varias tecnologías de seguridad para proteger los datos del usuario, lo que lo convierte en una opción popular entre quienes valoran la seguridad de sus datos personales. (ADnaliza, s.f)

Ventajas

iOS de Apple ofrece varias ventajas significativas que contribuyen a su popularidad entre los usuarios:

- Calidad y seguridad de las aplicaciones: Gracias al riguroso proceso de revisión de Apple, las aplicaciones disponibles en la App Store suelen ser seguras y de alta calidad. Este proceso ayuda a mantener un estándar elevado en términos de funcionamiento y seguridad de las aplicaciones disponibles para los usuarios de iOS. (ADnaliza, s.f)
- Sincronización con otros dispositivos de Apple: iOS se integra de manera fluida y eficiente con otros productos de Apple, como Macs, iPads, Apple Watch y Apple TV. Esta integración permite una experiencia de usuario cohesiva y sin interrupciones, facilitando la sincronización de datos, la continuidad entre dispositivos y el uso compartido de contenido entre diferentes dispositivos Apple. (ADnaliza, s.f)
- Actualizaciones regulares: Apple se compromete a proporcionar actualizaciones de software regulares para todos los dispositivos

compatibles con iOS, independientemente del operador de telefonía móvil. Estas actualizaciones no solo introducen nuevas funciones y mejoras de rendimiento, sino que también incluyen parches de seguridad importantes para proteger los dispositivos contra vulnerabilidades conocidas. (ADnaliza, s.f)

Desventajas

iOS, el sistema operativo de Apple, presenta algunas limitaciones que pueden influir en la experiencia del usuario:

- Falta de personalización: En comparación con Android, iOS ofrece menos opciones de personalización para los usuarios. Mientras que Android permite ajustar y modificar varios aspectos del sistema operativo, como widgets en la pantalla de inicio, lanzadores de aplicaciones alternativos y configuraciones más detalladas, iOS tiene una interfaz más uniforme y controlada por Apple. Esto puede resultar frustrante para quienes prefieren tener más control sobre la apariencia y el funcionamiento de su dispositivo móvil. (ADnaliza, s.f)
- Precio de los dispositivos: Los dispositivos iOS, como iPhones y iPads, tienden a ser más caros en comparación con dispositivos que ejecutan otros sistemas operativos. Apple posiciona sus productos en el segmento premium del mercado, lo que puede hacer que los dispositivos iOS sean inaccesibles para algunos consumidores que buscan opciones más económicas. (ADnaliza, s.f)
- Compatibilidad limitada con dispositivos no Apple: iOS está optimizado para funcionar de manera óptima con otros productos de Apple, como Macs, iPads y Apple Watch. Sin embargo, puede haber limitaciones en la compatibilidad con dispositivos y servicios de terceros que no pertenecen al ecosistema de Apple. Esto puede afectar la interoperabilidad y la experiencia del usuario que utiliza dispositivos y servicios no Apple junto con un dispositivo iOS. (ADnaliza, s.f)

Windows Phone

Durante el año 2010, Microsoft lanzó su intento de incursionar en el mercado de teléfonos móviles y competir con gigantes establecidos como Google y Apple. El 11 de octubre de ese año, Microsoft presentó Windows Phone como un nuevo sistema operativo diseñado para ofrecer una experiencia innovadora y distinta en dispositivos móviles. (Silva, 2023)

Para ese entonces, con la llegada del nuevo Windows 8, que presentaba una apariencia más plana y organizaba las aplicaciones en pequeños recuadros de colores, los teléfonos móviles con Windows Phone ofrecían una experiencia

similar. En lugar de tener una pantalla de inicio con íconos de aplicaciones distribuidas, las "ventanas" ocupaban una parte significativa de la pantalla. (Silva, 2023)

Las aplicaciones en Windows Phone podían tener diferentes tamaños para facilitar la organización del espacio en la pantalla de inicio. A diferencia de Android, que permitía agrupar varias aplicaciones en una carpeta, Windows Phone organizaba las aplicaciones en forma de mosaico. Esto significaba que las aplicaciones más importantes podían destacarse en grandes recuadros, mientras que las menos utilizadas estaban en recuadros más pequeños en segundo o tercer plano. (Silva, 2023)

Aunque Windows Phone parecía ganar popularidad en esos años, su sistema operativo no pudo soportar una aplicación que se volvió fundamental: Instagram. Debido a su estructura, fue difícil incorporarla inicialmente a la tienda de aplicaciones y, aunque eventualmente lo logró, el declive de los dispositivos llegó en 2015. Cuatro años después, en 2019, Instagram anunció su retirada de estos teléfonos, al igual que WhatsApp y otras plataformas, debido a la falta de actualizaciones del sistema operativo. (Silva, 2023)

Incluso YouTube, una aplicación esencial para el consumo de contenido de entretenimiento, no tenía soporte en este dispositivo, principalmente debido a un conflicto entre Microsoft y Google. (Silva, 2023)

En general, Windows Phone tenía buenas ideas que fueron implementadas de manera deficiente debido a las limitaciones de los dispositivos disponibles en ese momento. Su apariencia siempre mantuvo similitudes con la de los computadores que ejecutaban el mismo sistema operativo, sin presentar diferencias significativas. (Silva, 2023)

Sin embargo, a pesar de tener buena aceptación entre los usuarios, Microsoft no logró alcanzar la relevancia que Apple y Google tenían (y siguen teniendo) como desarrolladores de sistemas operativos en ese entonces. (Silva, 2023)

Para el último año del Windows Phone, se anunció el fin del desarrollo del sistema operativo, lo que significó que dejarían de haber nuevas actualizaciones y servicios para estos teléfonos. Aquellos que habían adquirido estos dispositivos tuvieron que migrar nuevamente a modelos que utilizaban Android o iOS antes de que todas sus aplicaciones favoritas se volvieran obsoletas. (Silva, 2023)

Subtema 3: Plataformas de desarrollo móvil

El mercado de las aplicaciones móviles está experimentando un crecimiento significativo en la actualidad. Cada vez más usuarios a nivel global utilizan diariamente diversas aplicaciones en sus teléfonos y tabletas, tanto para sus actividades personales como profesionales. (Axarnet, s.f)

Para el desarrollo de aplicaciones móviles, es crucial utilizar plataformas de desarrollo de aplicaciones que se pueden clasificar en diferentes grupos según su naturaleza. (Axarnet, s.f)

Nativas

Las plataformas de desarrollo nativas se refieren a aquellas diseñadas específicamente para cada sistema operativo: iOS, Android o Windows Phone. Cada una utiliza un lenguaje de programación específico: Objective-C para iOS, Java para Android, y .NET para Windows Phone. (Axarnet, s.f)

Estas aplicaciones nativas tienen la ventaja de aprovechar al máximo las funcionalidades de los dispositivos y pueden ejecutarse sin conexión a internet, lo cual es una característica distintiva. (Axarnet, s.f)

Sin embargo, el desarrollo y las actualizaciones de aplicaciones nativas también tienen un costo elevado, lo cual es una desventaja significativa. (Axarnet, s.f)

Híbridas

Las aplicaciones híbridas son aquellas que fusionan características de las aplicaciones nativas y web según las necesidades específicas. Estas aplicaciones se desarrollan utilizando los lenguajes de programación JavaScript, CSS y HTML, similares a las aplicaciones web, lo que les permite adaptarse a cualquier sistema operativo. Además, al igual que las aplicaciones nativas, proporcionan acceso a todas las funcionalidades de los dispositivos. (Axarnet, s.f)

Multiplataforma

Una de las opciones más solicitadas por los desarrolladores debido a su capacidad para disminuir los gastos y acelerar el proceso de desarrollo. Como su nombre indica, estas aplicaciones están diseñadas para ajustarse perfectamente a las diversas plataformas de dispositivos móviles. (Axarnet, s.f)

La principal desventaja que tienen es que los usuarios no pueden aprovechar al máximo todas las funcionalidades de los teléfonos y tabletas con estas aplicaciones. (Axarnet, s.f)

Web / HTML5

Las aplicaciones web se construyen utilizando los lenguajes de programación JavaScript, CSS y HTML, que son tres de los más populares a nivel mundial. (Axarnet, s.f)

Estas aplicaciones son compatibles y se ajustan a todos los sistemas operativos, eliminando la necesidad de desarrollar una aplicación específica para cada SO como ocurre con las aplicaciones nativas. Además, las aplicaciones web se adaptan de manera precisa a los navegadores móviles de los dispositivos. (Axarnet, s.f)

Con HTML5, los programadores y desarrolladores tienen la capacidad de crear aplicaciones basadas en la web que los usuarios pueden ejecutar directamente desde cualquier dispositivo móvil a través del navegador web integrado en el terminal. (Axarnet, s.f)

React Native

React Native es una de las plataformas más populares para el desarrollo de aplicaciones móviles. Permite a los desarrolladores crear aplicaciones para Android e iOS utilizando JavaScript y React. Una de sus ventajas destacadas es la capacidad de desarrollar aplicaciones con un rendimiento casi nativo, aprovechando JavaScript, un lenguaje ampliamente conocido y utilizado. (Axarnet, s.f)

Flutter

La plataforma de desarrollo Flutter, creada por el gigante tecnológico Google, ha experimentado un rápido crecimiento. Flutter utiliza el lenguaje Dart, permitiendo el desarrollo de aplicaciones con un diseño atractivo y un rendimiento excelente. Destaca por su sistema de widgets, que facilita una personalización amplia y sencilla de las interfaces de usuario. (Axarnet, s.f)

Xamarin

Xamarin, propiedad de Microsoft, es una opción destacada para el desarrollo de aplicaciones móviles. Emplea el lenguaje C#, lo cual es ideal para desarrolladores familiarizados con .NET. Una ventaja significativa de Xamarin es su capacidad para compartir código entre las plataformas iOS, Android y Windows, lo que permite ahorrar tiempo y recursos durante el proceso de desarrollo. (Axarnet, s.f)

Ionic

Ionic es una plataforma de desarrollo orientada a crear aplicaciones móviles mediante tecnologías web como HTML, CSS y JavaScript. Este enfoque simplifica la transición de los desarrolladores web hacia el desarrollo móvil. Aunque las aplicaciones no son nativas, ofrecen un rendimiento robusto y admiten plugins nativos para integrar funcionalidades adicionales. (Axarnet, s.f)

PhoneGap

PhoneGap, también llamado Apache Cordova, es una plataforma para desarrollar aplicaciones móviles utilizando tecnologías web como HTML, CSS y JavaScript. Aunque su rendimiento no llega al nivel de una aplicación nativa, su facilidad de uso y la capacidad de trabajar con estos lenguajes hacen que sea muy atractivo para los desarrolladores web. (Axarnet, s.f)

Subtema 4: Entornos de desarrollo móvil

Los entornos de desarrollo de aplicaciones son software integral que proporciona todas las herramientas necesarias para desarrollar una aplicación. Normalmente, incluyen soporte para varios lenguajes de programación, capacidades de compilación para convertir el código en la aplicación final, depuración en tiempo real y pruebas. Además, suelen ofrecer funciones como autocompletado y resaltado de sintaxis para agilizar la escritura de código y reducir errores. (Liderlogo, s.f)

Existen diversos tipos de entornos de desarrollo de aplicaciones, cada uno con capacidades distintas según las necesidades específicas de desarrollo de la aplicación. (Liderlogo, s.f)

Los entornos de desarrollo integrados, conocidos como IDE (por sus siglas en inglés, Integrated Development Environment), son herramientas completas que facilitan el desarrollo de software a los programadores. Estos entornos suelen incluir un editor de código fuente, herramientas automáticas de construcción y un depurador. Los mejores IDE también ofrecen funciones avanzadas como autocompletado inteligente (IntelliSense). Algunos IDEs como NetBeans y Eclipse además integran compiladores e intérpretes. (Liderlogo, s.f)

Los IDEs son fundamentales para aumentar la productividad durante el desarrollo de aplicaciones, ya que proporcionan un espacio organizado para trabajar en proyectos, además de capacidades robustas de depuración. Por ejemplo, IntelliSense mejora la velocidad y precisión al sugerir opciones contextuales mientras se escribe código, lo cual es crucial para desarrollar aplicaciones de alta calidad de manera eficiente y rápida para su lanzamiento al mercado. (Liderlogo, s.f)

Android Studio

Este IDE está construido sobre la base de IntelliJ IDEA, proporcionando un editor de código robusto y una variedad de funciones diseñadas para mejorar la eficiencia durante el desarrollo de aplicaciones. También ofrece un sistema de compilación flexible, un emulador rápido y herramientas para detectar problemas relacionados con la compatibilidad, rendimiento o usabilidad. Además, permite realizar modificaciones en el código y aplicar nuevos recursos a la aplicación sin necesidad de reiniciarla, y es compatible con C++, NDK y la Plataforma Google Cloud. (Nielfa, s.f)

Este software está diseñado específicamente para la creación de aplicaciones móviles en el sistema operativo Android y se ha convertido en una herramienta

esencial para cualquier desarrollador. No obstante, su utilidad va más allá de la creación inicial, ya que permite realizar numerosas acciones cruciales para el desarrollo y actualización continuo de la aplicación. (Nielfa, s.f)

Su interfaz simple e intuitiva facilita la creación y el desarrollo de aplicaciones, sin requerir un gran esfuerzo, ya seas un desarrollador experimentado o un principiante. (Nielfa, s.f)

Ventajas

- Permite realizar compilaciones de manera muy rápida, lo que facilita la detección inmediata de fallos y la implementación de mejoras en la aplicación. También ofrece renderizado de layouts en tiempo real y la posibilidad de utilizar parámetros tools. Además, permite ejecutar la aplicación directamente desde el teléfono móvil en tiempo real. Su potente emulador permite verificar el estado de la aplicación al instante, sin necesidad de utilizar un ordenador. (Nielfa, s.f)
- Permite simular diversos dispositivos y tabletas, permitiendo visualizar todos en un mismo entorno. Esto facilita trabajar con varias aplicaciones simultáneamente y ver las secciones de código necesarias para cada una. (Nielfa, s.f)
- Además, permite crear elementos gráficos para la interfaz de la aplicación sin necesidad de escribir código, lo cual simplifica el diseño visual de la app. (Nielfa, s.f)
- Es el IDE oficial de Android, lo que garantiza su correcto funcionamiento, ya que es utilizado por los mismos desarrolladores del sistema operativo oficial de Android para crear todas sus aplicaciones. (Nielfa, s.f)

Desventajas

- Una de sus principales desventajas es la falta de soporte para el desarrollo con NDK. No obstante, no te preocupes, ya que con el plugin de IntelliJ es posible realizar desarrollos con NDK, obteniendo resultados similares al entorno predeterminado y permitiendo el uso de otros lenguajes de programación. (Nielfa, s.f)
- Otro problema que puede surgir con este software es su alto consumo de recursos y batería. Para que el emulador funcione de manera óptima, es necesario tener un equipo potente, con una gran cantidad de RAM y suficiente espacio en el disco duro, entre otros requisitos básicos. (Nielfa, s.f)

Xcode

Xcode, el entorno de desarrollo integrado (IDE) creado por Apple, es una herramienta fundamental para los desarrolladores que operan dentro del ecosistema Apple. Está diseñado para facilitar la creación de aplicaciones para

dispositivos iOS, macOS, watchOS, tvOS, iPadOS y, más recientemente, visionOS. (Lizana, 2024)

Es un conjunto completo de herramientas que incluye un editor de código, compiladores, depuradores y otras utilidades fundamentales para el desarrollo de software. Su relevancia se debe a su capacidad para crear, mantener, compilar y depurar proyectos, además de proporcionar herramientas que simplifican el desarrollo, como la visualización en tiempo real del código que se está escribiendo. (Lizana, 2024)

Funciones destacadas

- **Editor de código avanzado:** Xcode cuenta con un robusto editor de código compatible con varios lenguajes de programación, como Swift y Objective-C. Este editor incluye funciones inteligentes como el resaltado de sintaxis, autocompletado y corrección de errores. Sin embargo, Apple está promoviendo el uso exclusivo de Swift para mejorar la eficiencia de las aplicaciones. (Lizana, 2024)
- **Simuladores y depuradores:** Xcode ofrece simuladores para dispositivos iOS, macOS, watchOS y tvOS, permitiendo probar y depurar aplicaciones en entornos virtuales antes de su lanzamiento. Esto significa que puedes ejecutar la aplicación en un iPhone virtual con la versión de software deseada para evaluar su interfaz, funcionamiento y detectar posibles errores. (Lizana, 2024)
- **Instruments:** Xcode incluye la herramienta Instruments, que permite a los desarrolladores medir el rendimiento de sus aplicaciones y detectar posibles cuellos de botella. (Lizana, 2024)
- **Integración con SDK y frameworks:** Xcode se integra estrechamente con los SDK (Software Development Kits) y frameworks de Apple, lo que permite a los desarrolladores acceder a las API oficiales diseñadas específicamente para el software en el que están trabajando. Esto mejora el rendimiento y facilita el desarrollo. (Lizana, 2024)

Compatibilidad con nuevas versiones: Xcode permite probar la integración del proyecto con las nuevas versiones de software que Apple lanza. Esto ayuda a garantizar que una aplicación siga funcionando correctamente en un nuevo entorno de software. (Lizana, 2024)

Preguntas de Comprensión de la Unidad

1. ¿Qué son las plataformas de desarrollo nativas y cómo se diferencian en función del sistema operativo?

Las plataformas de desarrollo nativas son aquellas diseñadas específicamente para cada sistema operativo móvil como iOS, Android o Windows Phone. Cada una utiliza un lenguaje de programación propio: Objective-C y Swift para iOS, Java y Kotlin para Android, y .NET para Windows Phone. Estas plataformas permiten a los desarrolladores aprovechar al máximo las funcionalidades del sistema operativo correspondiente para crear aplicaciones optimizadas y de alto rendimiento.

2. ¿Cómo se comparan Android e iOS en términos de instalación y funcionalidad de aplicaciones?

Tanto Android como iOS permiten a los usuarios instalar una amplia variedad de aplicaciones, desde herramientas de oficina y productividad hasta juegos y aplicaciones educativas. Sin embargo, desde el punto de vista tecnológico, iOS no es compatible con tecnologías como Java o Flash, lo que podría limitar el acceso a ciertos componentes de páginas web y aplicaciones que dependen de estas tecnologías específicas.

3. ¿Qué papel desempeña Google en el ámbito de los sistemas operativos móviles?

Google no solo es conocido por su popular motor de búsqueda, sino también por ser propietario del sistema operativo más utilizado en smartphones y tablets a nivel mundial, Android. Este sistema operativo móvil ha evolucionado significativamente desde su lanzamiento inicial y ha marcado una gran influencia en el mercado tecnológico global.

4. ¿Por qué JavaScript es considerado el lenguaje ideal para desarrollar aplicaciones móviles multiplataforma?

JavaScript se destaca como el lenguaje ideal para aplicaciones móviles multiplataforma debido a su rapidez, versatilidad y simplicidad. Conocido por su funcionalidad efectiva, JavaScript permite mejorar la experiencia del usuario mediante la creación de interactividad en las aplicaciones móviles. Su capacidad para ser ejecutado en diversos entornos y sistemas operativos garantiza que las aplicaciones desarrolladas con JavaScript

sean accesibles en una amplia gama de dispositivos móviles, facilitando así la creación de aplicaciones consistentes y eficientes.

5. ¿Qué ventaja ofrece Python en el desarrollo de aplicaciones móviles en comparación con otros lenguajes como Java?

Python destaca en el desarrollo de aplicaciones móviles por su código conciso, lo cual simplifica significativamente el trabajo de programadores e ingenieros informáticos. Con Python, se necesitan menos líneas de código en comparación con lenguajes como Java para lograr funcionalidades similares, lo que acelera el proceso de desarrollo y reduce la posibilidad de errores. Esta característica hace que Python sea especialmente atractivo para el desarrollo rápido de prototipos y aplicaciones que requieren mantenimiento y actualizaciones frecuentes.

Material complementario

Los siguientes recursos complementarios son sugerencias para que se pueda ampliar la información sobre el tema trabajado, como parte de su proceso de aprendizaje autónomo:

Videos de apoyo:

- <https://www.youtube.com/watch?v=2etL7xYhIAE>
- <https://www.youtube.com/watch?v=tvOBTwamsS0>
- <https://www.youtube.com/watch?v=IhTbyjzujAE>
- https://www.youtube.com/watch?v=5uxeW-_6DCY
- <https://www.youtube.com/watch?v=K2u9IPtQpmQ>

Links de apoyo:

- <https://developer.android.com/studio>
- <https://developer.apple.com/xcode/>

Bibliografía

- » Nader, K. (2023). Java para el desarrollo de aplicaciones móviles: mejores prácticas y consejos. Recuperado de <https://appmaster.io/es/blog/java-para-desarrollo-de-aplicaciones-moviles>
- » Brutti, F. (2023). Kotlin: Aprende todo sobre el mejor el lenguaje para aplicaciones móviles en Android. Recuperado de <https://thepower.education/blog/kotlin-aprende-todo>
- » Bambu editorial. (2022). Desarrollo de apps con Python. Recuperado de <https://bambu-mobile.com/desarrollo-de-apps-con-python/>
- » Ortíz, M. (2023). Javascript: ¿Qué Es Y Para Qué Sirve?. Recuperado de <https://stride.com.co/blog/javascript-que-es-para-que-sirve/>
- » Tokio School. (s.f). Qué es Swift: ¡Descúbrelo!. Recuperado de <https://www.tokioschool.com/formaciones/cursos-programacion/swift/que-es/>
- » López, M. (2021). Lenguajes de programación para móvil. Recuperado de <https://immune.institute/blog/lenguajes-de-programacion-para-movil/>
- » Adeva, R. (2023). Qué es Android: todo sobre el sistema operativo de Google. Recuperado de <https://www.adslzone.net/reportajes/software/que-es-android/>
- » Gabit. (s.f). ¿Qué es iOS?. Recuperado de <https://www.gabit.org/gabit/?q=es/que-es-ios>
- » ADnaliza. (s.f). iOS. Recuperado de <https://www.adnaliza.com/diccionario/ios/>
- » Silva, R. (2023). Windows Phone, el celular de Microsoft que cumple 13 años. Recuperado de <https://www.infobae.com/tecno/2023/10/22/windows-phone-el-celular-de-microsoft-que-cumple-13-anos/>
- » Axarnet. (s.f). Plataformas de desarrollo de aplicaciones móviles. Recuperado de <https://axarnet.es/blog/plataformas-desarrollo-aplicaciones-moviles>
- » Liderlogo. (s.f). Entornos de desarrollo de apps: principales características. Recuperado de <https://www.liderlogo.es/disenio-web/entornos-de-desarrollo-de-apps/>
- » Nielfa, J. (s.f). Android Studio: El entorno de desarrollo oficial de Android. Recuperado de <https://scoreapps.com/blog/android-studio/>
- » Lizana, J. (2024). Xcode: qué es, para qué sirve y qué funciones para desarrolladores incluye este IDE de Apple. Recuperado de <https://www.applesfera.com/nuevo/xcode-que-sirve-que-funciones-para-desarrolladores-incluye-este-ide-apple>



UNEMI
UNIVERSIDAD ESTATAL DE MILAGRO

Compendio del autor

COMPUTACIÓN MÓVIL

UNIDAD 3
DESARROLLO DE APLICACIONES MÓVILES

ÍNDICE

Contenido

Unidad 3: DESARROLLO DE APLICACIONES MÓVILES	3
<i>Tema 1: Desarrollo móvil nativo.....</i>	<i>3</i>
<i>Objetivo:</i>	<i>3</i>
<i>Introducción:</i>	<i>3</i>
Información de los subtemas.....	4
<i>Subtema 1: Configuración de entorno de desarrollo Android Studio.....</i>	<i>4</i>
<i>Subtema 2: Creación de una aplicación Android Studio de ejemplo</i>	<i>12</i>
<i>Subtema 3: Entendiendo el ciclo de vida de aplicaciones, actividades y View Layouts de Android</i>	<i>17</i>
<i>Subtema 4: Trabajar con componentes de Android.....</i>	<i>22</i>
Preguntas de Comprensión de la Unidad	24
Material complementario	25
Bibliografía.....	26

Unidad 3: DESARROLLO DE APLICACIONES MÓVILES

Tema 1: Desarrollo móvil nativo

Objetivo:

Revisar aspectos y herramientas para la codificación de aplicaciones móviles utilizando tecnologías de Desarrollo Nativo y Framework.

Introducción:

El desarrollo móvil nativo se refiere a la creación de aplicaciones específicamente diseñadas y desarrolladas para funcionar en un sistema operativo móvil en particular, como Android o iOS. A diferencia de las aplicaciones web o híbridas, las aplicaciones nativas están escritas en lenguajes de programación específicos de la plataforma, como Java o Kotlin para Android, y Objective-C o Swift para iOS.

Este enfoque de desarrollo ofrece numerosas ventajas, como un rendimiento superior, una integración más profunda con las características del dispositivo y un acceso completo a las API y funcionalidades nativas. Además, las aplicaciones nativas suelen ofrecer una experiencia de usuario más fluida y optimizada, ya que están diseñadas para aprovechar al máximo las capacidades y características únicas de cada plataforma móvil.

En este documento, exploraremos los fundamentos del desarrollo móvil nativo, desde los conceptos básicos de la plataforma Android, así como componentes y elementos que conforman esta plataforma.

Información de los subtemas

Subtema 1: Configuración de entorno de desarrollo Android Studio

Generalmente, cualquier aplicación, herramienta, sitio web, o servidor digital que ofrece algún tipo de servicio en internet utiliza lenguajes de programación o entornos de desarrollo específicos. Un ejemplo es Python, un lenguaje ampliamente empleado en el campo de la Inteligencia Artificial (Santaella, 2022).

De la misma manera, ocurre con el sistema operativo Android. Todas las aplicaciones y herramientas creadas para este SO en particular tienen su propio entorno de desarrollo. Este entorno es Android Studio, que ofrece una gran flexibilidad en el desarrollo de características y funcionalidades para las herramientas o aplicaciones de este sistema (Santaella, 2022).

¿Qué características tiene Android Studio?

Android Studio facilita la incorporación de características y funciones muy beneficiosas para las aplicaciones, las cuales se mejoran con el tiempo. Así, obtenemos lo siguiente:

- El sistema de compilación es versátil y compatible con Gradle, lo que permite automatizar las compilaciones de manera flexible y con alto rendimiento. Los scripts de compilación se escriben en Groovy y Kotlin DSL (Santaella, 2022).
- Este entorno está diseñado para ofrecer al usuario una experiencia de trabajo fluida, con muchas funciones prácticas y útiles (Santaella, 2022).
- Además, esta plataforma permite desarrollar aplicaciones para cualquier dispositivo Android (Santaella, 2022).
- Incluye plantillas de compilación que facilitan la implementación rápida de funciones comunes de otras aplicaciones, así como la importación de códigos de muestra (Santaella, 2022).
- Ofrece una amplia variedad de herramientas de prueba con diversos marcos de trabajo (Santaella, 2022).
- Permite modificar fragmentos de código y recursos de una aplicación sin necesidad de reiniciarla (Santaella, 2022).
- Ofrece compatibilidad con servicios en la nube como Google Cloud Platform (Santaella, 2022).
- Admite lenguajes como NDK y C++ (Santaella, 2022).

¿Qué necesito para instalar Android Studio?

Primero debemos tener en cuenta los requerimientos básicos solicitados para la instalación de este entorno.

Para Windows (requisitos mínimos):

- Microsoft® Windows® 8/10/11 de 64 bits (Google Developers, 2024).
- Arquitectura de CPU x86_64; procesador Intel Core de segunda generación o posterior, o CPU AMD compatible con un hipervisor de Windows (Google Developers, 2024).
- 8 GB de RAM o más (Google Developers, 2024).
- 8 GB de espacio disponible en el disco como mínimo (IDE + SDK de Android + Android Emulator) (Google Developers, 2024).
- Resolución de pantalla mínima de 1280 × 800 (Google Developers, 2024).

Para macOS (requisitos mínimos):

- MacOS® 10.14 (Mojave) o versiones posteriores (Google Developers, 2024).
- Chips basados en ARM, o Intel Core de segunda generación (o posterior) compatibles con el framework de hipervisor (Google Developers, 2024).
- 8 GB de RAM o más (Google Developers, 2024).
- 8 GB de espacio disponible en el disco como mínimo (IDE + SDK de Android + Android Emulator) (Google Developers, 2024).
- Resolución de pantalla mínima de 1280 × 800 (Google Developers, 2024).

Para Linux (requisitos mínimos):

- Cualquier distribución de Linux de 64 bits que sea compatible con Gnome, KDE o Unity DE; GNU C Library (glibc) 2.31 o versiones posteriores (Google Developers, 2024).
- Arquitectura de CPU x86_64; procesador Intel Core de segunda generación o posterior, o procesador AMD compatible con AMD Virtualization (AMD-V) y SSSE3 (Google Developers, 2024).
- 8 GB de RAM o más (Google Developers, 2024).
- 8 GB de espacio disponible en el disco como mínimo (IDE + SDK de Android + Android Emulator) (Google Developers, 2024).
- Resolución de pantalla mínima de 1280 × 800 (Google Developers, 2024).

JDK (Java Development Kit)

Según IBM (2021) “Java Development Kit (JDK) es un software para los desarrolladores de Java. Incluye el intérprete Java, clases Java y herramientas de desarrollo Java (JDT): compilador, depurador, desensamblador, visor de applets, generador de archivos de apéndice y generador de documentación.”

De igual manera IBM (2021) menciona que “El JDK le permite escribir aplicaciones que se desarrollan una sola vez y se ejecutan en cualquier lugar de cualquier máquina virtual Java. Las aplicaciones Java desarrolladas con el JDK en un sistema se pueden usar en otro sistema sin tener que cambiar ni recompilar el código. Los archivos de clase Java son portables a cualquier máquina virtual Java estándar.”

Proceso de instalación de Android Studio en Windows

Primero se tiene que descargar el instalador de Android Studio podemos acceder a este instalador por medio de la página oficial de Android Studio, donde podremos obtener dicho instalador.

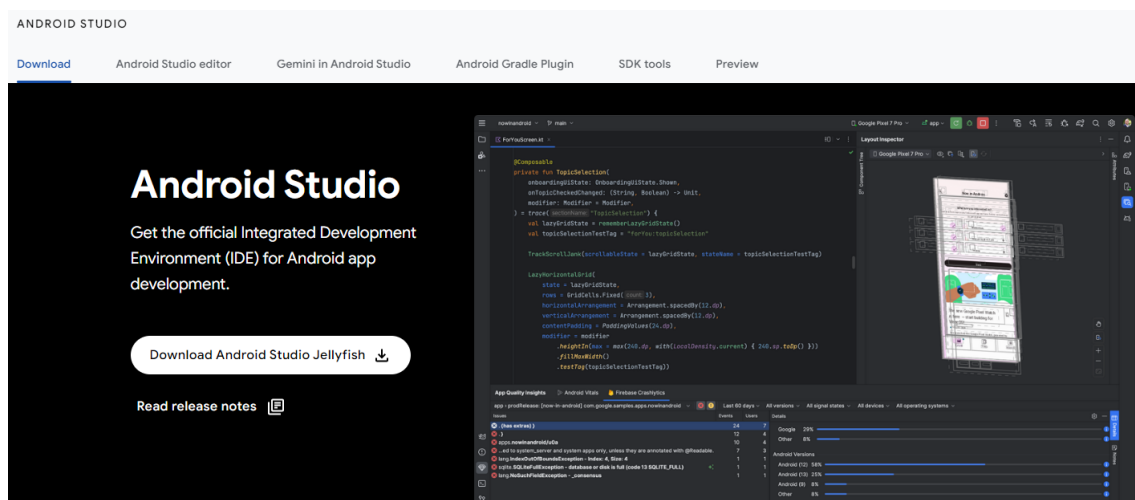


Figura 1: Sitio oficial de descarga de Android Studio. Fuente: Google Developers, 2024. Recuperado de https://developer.android.com/studio?gad_source=1&gclid=Cj0KCQjwGJyyBhCGARIsAK8LVLNc-otpRBA04IEvFObl81fF5xU-ki7dh-9IsdjuFWWX9IK9iaqeOQaAqGIEALw_wcB&gclid=aw.ds

Una vez descargado el instalador procedemos a ejecutarlo como cualquier programa en Windows, a lo cual tendremos el siguiente procedimiento.

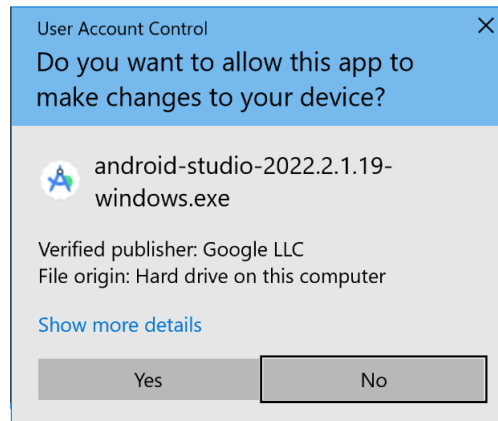


Figura 2: User Account Control Fuente: Google Developers, 2024. Recuperado de <https://developer.android.com/codelabs/basic-android-kotlin-compose-install-android-studio?hl=es-419#2>



Figura 3: Diálogo welcome to Android Studio Setup. Fuente: Google Developers, 2024. Recuperado de <https://developer.android.com/codelabs/basic-android-kotlin-compose-install-android-studio?hl=es-419#2>

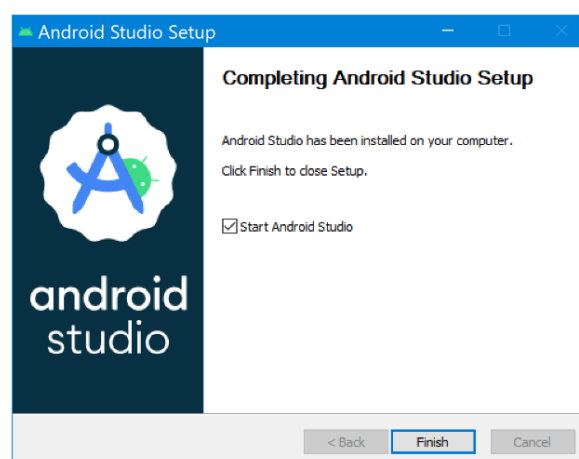


Figura 4: Finalización del proceso de instalación. Fuente: Google Developers, 2024. Recuperado de <https://developer.android.com/codelabs/basic-android-kotlin-compose-install-android-studio?hl=es-419#2>

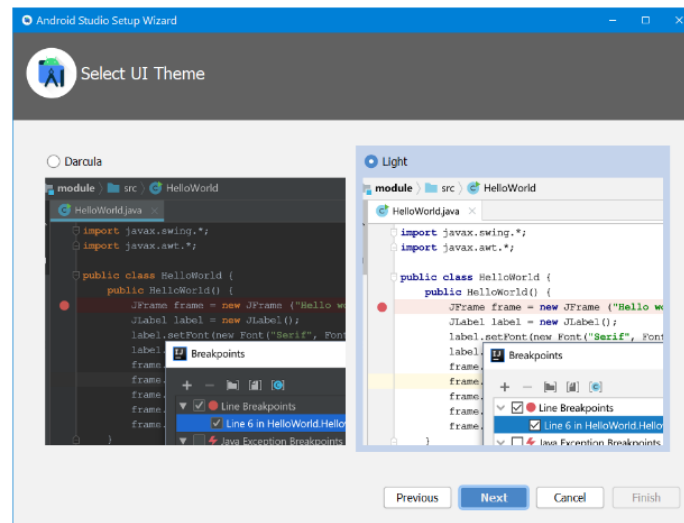


Figura 5: Selección de tema grafico del IDE. Fuente: Google Developers, 2024. Recuperado de <https://developer.android.com/codelabs/basic-android-kotlin-compose-install-android-studio?hl=es-419#2>

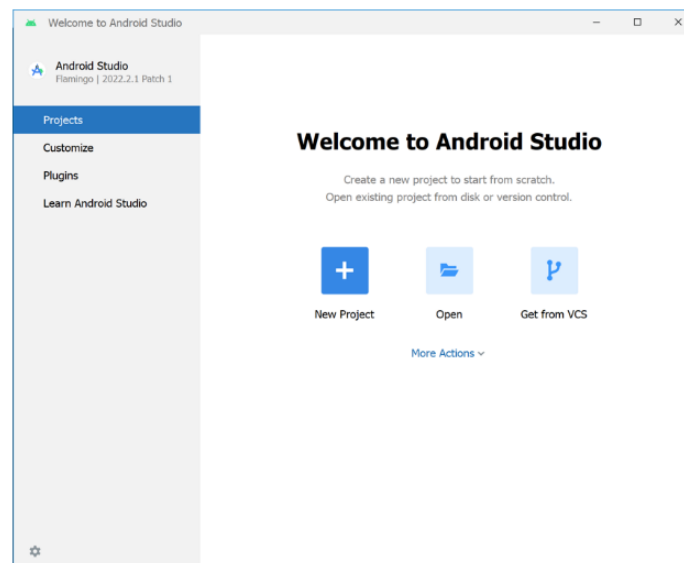


Figura 6: Ventana de Bienvenida de Android Studio. Fuente: Google Developers, 2024. Recuperado de <https://developer.android.com/codelabs/basic-android-kotlin-compose-install-android-studio?hl=es-419#2>

Una vez finalizado el proceso de instalación, podremos ver el área de trabajo de nuestro IDE Android Studio como el siguiente:

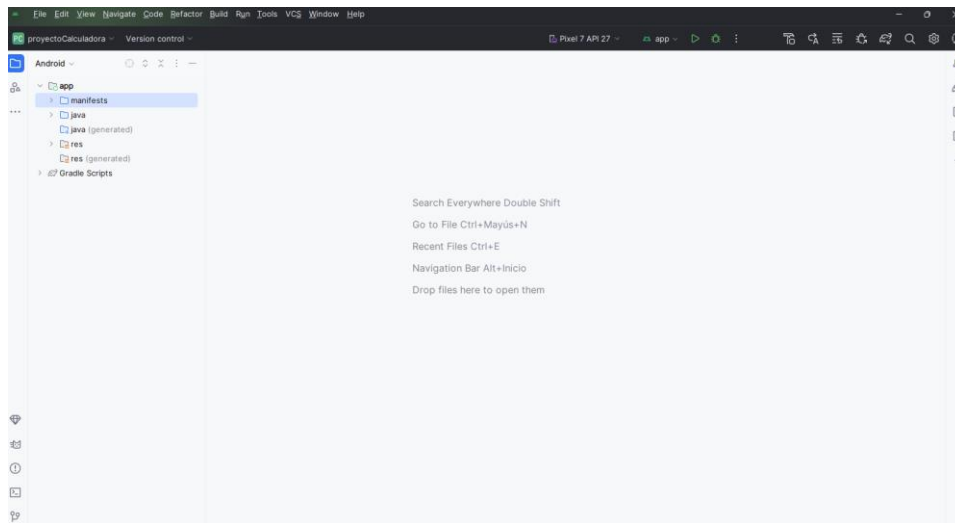


Figura 7: IDE Android Studio

Device Manager

Sobre este elemento Microsoft (2024) menciona “Android Device Manager se usa para crear y configurar dispositivos virtuales Android (AVD) que se ejecutan en Android Emulator. Cada AVD es una configuración del emulador que simula un dispositivo Android físico. Esto permite ejecutar y probar la aplicación en diversas configuraciones que simulan diferentes dispositivos Android físicos.”

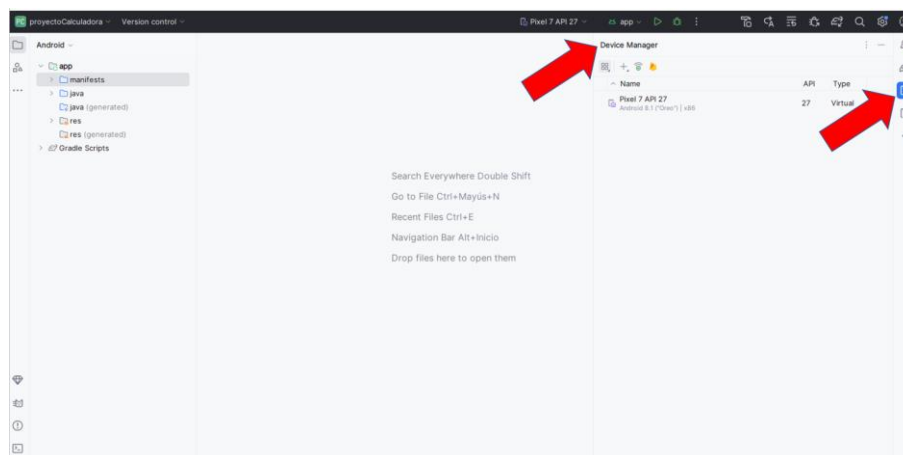


Figura 8: Device Manager

Crear un nuevo dispositivo virtual para usarlo en el emulador de Android Studio.

Para crear un nuevo dispositivo virtual en nuestro entorno Android Studio

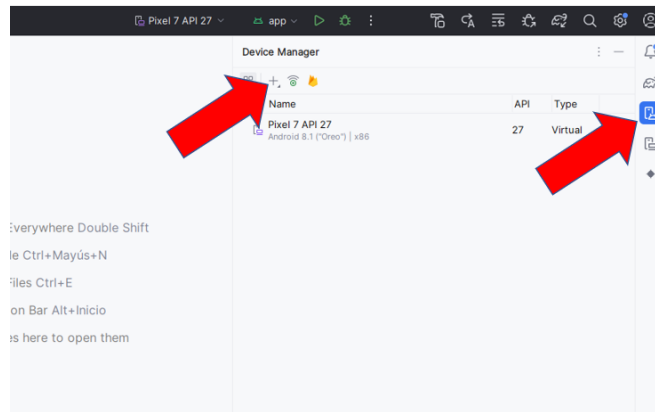


Figura 9: Selección de agregar nuevo dispositivo virtual

Debemos seleccionar la categoría y la versión del teléfono en este caso.

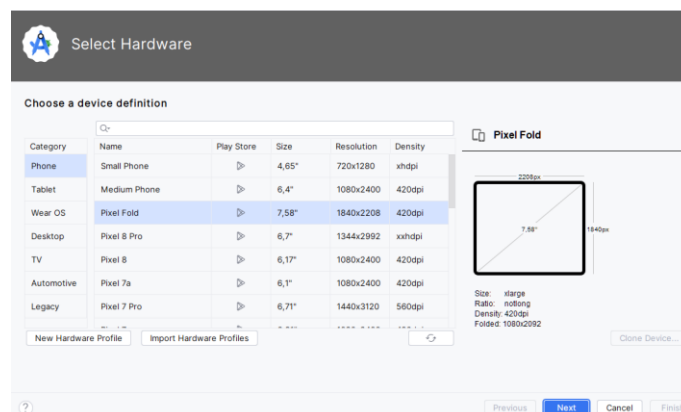


Figura 10: Selección de categoría y versión del teléfono

Posterior seleccionamos la versión del sistema operativo de Android (si no lo tenemos podemos descargar el SO que seleccionemos directamente desde Android Studio).

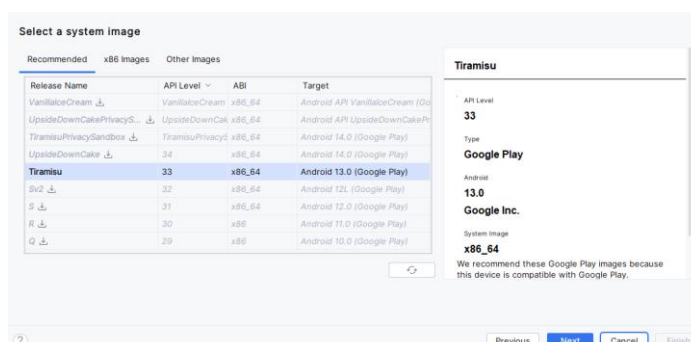


Figura 11: Selección de la versión del sistema operativo

Por último se presenta una ventana de verificación de configuración.

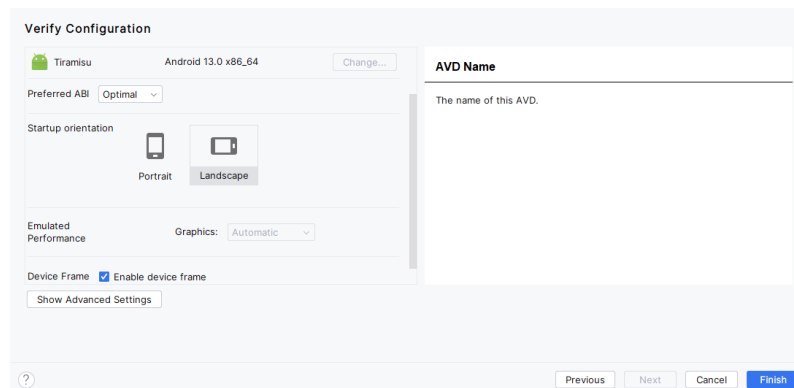


Figura 12: Ventana de verificación

Y finalmente tenemos generado el dispositivo virtual listo para arrancarlo en el emulador de Android Studio.

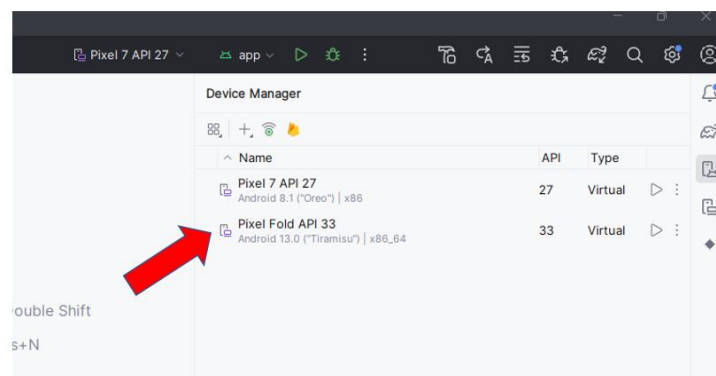


Figura 13: Dispositivo virtual creado.

Subtema 2: Creación de una aplicación Android Studio de ejemplo

Para este ejemplo vamos a desarrollar una aplicación en Android Studio referente a una calculadora, para ello vamos a realizar lo siguiente:

1. Creamos un nuevo proyecto en nuestro IDE Android Studio.

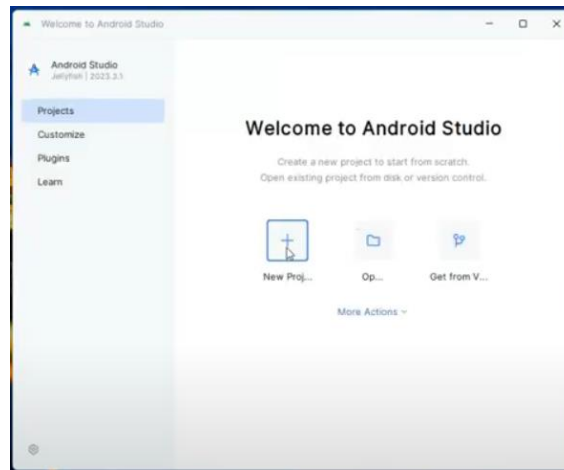


Figura 14: Creación de nuevo proyecto

2. Como vamos a trabajar con el lenguaje Java, seleccionamos la opción de Empty Views Activity.

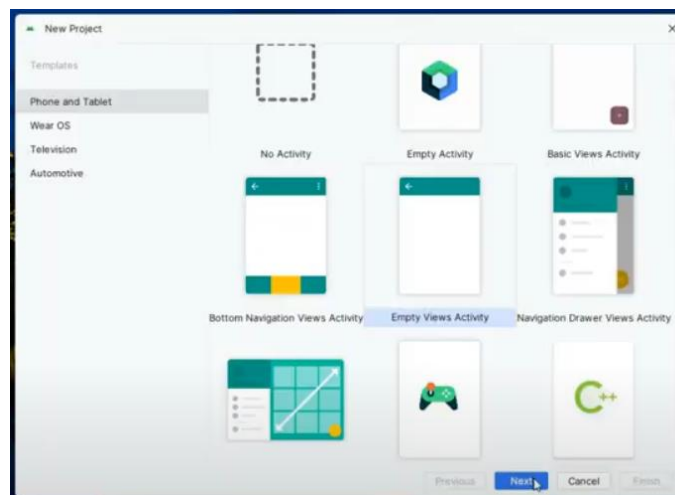


Figura 15: Selección de activity

- Después colocamos el nombre de nuestra aplicación, seleccionamos el lenguaje de programación en este caso es Java y la API de Android que deseemos.

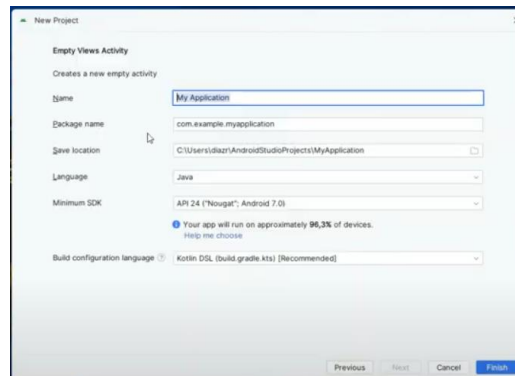


Figura 16: Información de nuestro proyecto

- Después ya tendremos nuestra área de trabajo del proyecto que generamos.

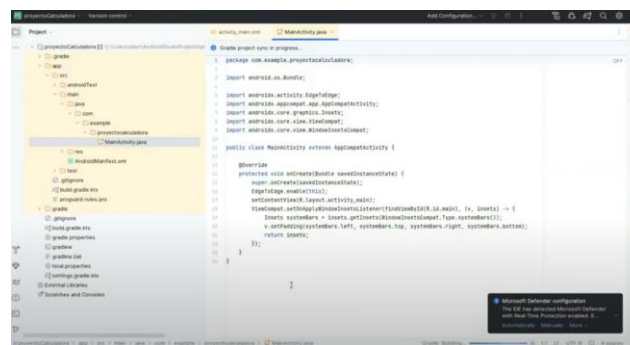


Figura 17: Área de trabajo del proyecto

En el proyecto las interfaces tendrán dos archivos un archivo con extensión XML y otro con extensión Java. El primero nos servirá para definir el diseño de la pantalla y el segundo nos permitirá definir la funcionalidad de los elementos que definamos en el diseño como botones, campos de texto, etc.

Para nuestro ejemplo vamos a crear una interfaz arrastrando elementos a la interfaz grafica, para este caso vamos a necesitar un elemento TextView (presentar el resultado de la operacion), dos elementos editTextView (ingresar los números para realizar la operación) y cuatro botones (para las operaciones básicas suma, resta, multiplicación y división). Tendríamos algo como esto.

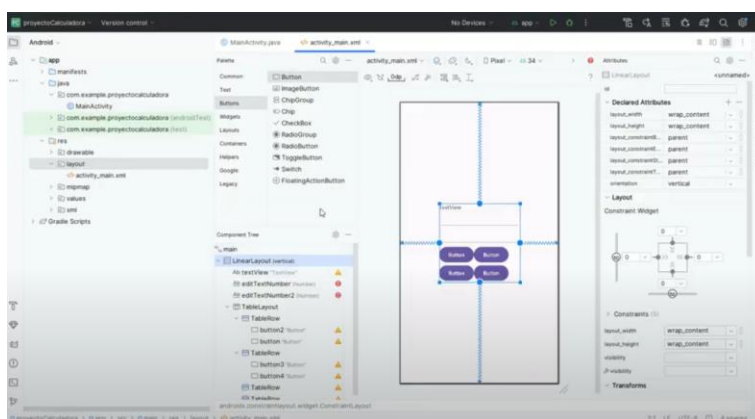


Figura 18: Interfaz de la app de ejemplo

Adicional se puede modificar características de los elementos como por ejemplo el id que nos servirá para poder identificar al elemento en codificación. Así también podemos modificar el texto que se presentara en un inicio.

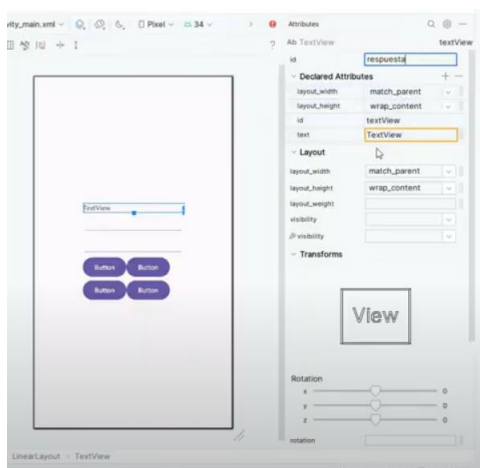


Figura 19: Modificación de características de los elementos

Ahora para probar que la interfaz creada se despliegue en el celular utilizamos el Device manager para trabajar con el emulador del IDE y probar nuestra app.

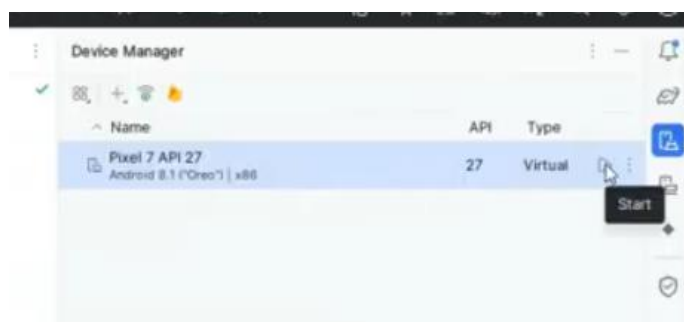


Figura 20: Device manager, encendiendo una imagen del SO Android en el emulador del IDE

Una vez que esta funcionando el emulador de Android ejecutamos la app con el icono de play que tenemos arriba como se ve en la imagen.

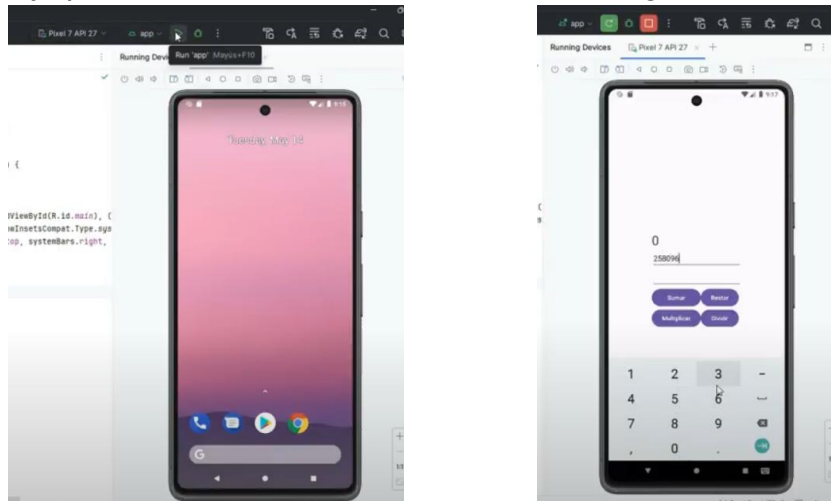


Figura 21: Ejecución de la app en el emulador

Es hora de ir al código, para esto vamos a enlazar los componentes graficos como los textos con variables del código en Java como lo vemos en la siguiente imagen del archivo con extensión Java.

```

MainActivity.java  activity_main.xml
13  public class MainActivity extends AppCompatActivity {
14
15      1 usage
16      private EditText num1;
17      1 usage
18      private EditText num2;
19      1 usage
20      private TextView resp;
21
22      @Override
23      protected void onCreate(Bundle savedInstanceState) {
24          super.onCreate(savedInstanceState);
25          setContentView(R.layout.activity_main);
26          ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v, insets) -> {
27              Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
28              v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
29              return insets;
30          });
31          num1 = findViewById(R.id.text_numero1);
32          num2 = findViewById(R.id.text_numero2);
33          resp = findViewById(R.id.respuesta);
34
35      No usages
36      public void sumar() {
37          |
38      }
39  }
  
```

Figura 22: Código en Java de la app

Como podemos ver también tenemos definida una función sumar() a la cual también debemos enlazarla en este caso a un botón que va a desencadenar esta acción, para ello lo vamos a realizar desde la parte grafica en el botón vamos a buscar en sus características el evento onClick.

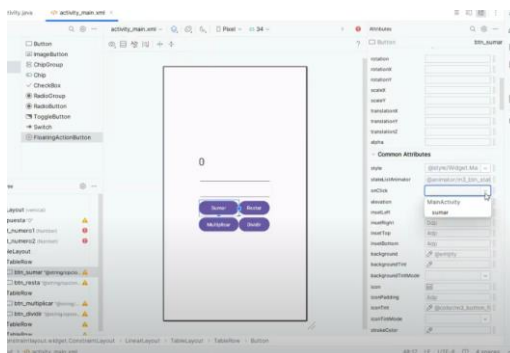


Figura 23: Evento onClick de un botón

Ahora debemos hacer lo mismo para los otros botones y los enlazaremos a otras funciones que crearemos, quedando de la siguiente manera estas funciones.

```

10  int i = findViewById(R.id.text_numero1);
11  int j = findViewById(R.id.text_numero2);
12  int k = findViewById(R.id.text_numero3);
13  int resp = findViewById(R.id.text_respuesta);
14
15  // Sumar
16  public void sumar(View view) {
17      Integer respuestaSuma = Integer.parseInt(i.getText().toString()) + Integer.parseInt(j.getText().toString());
18      resp.setText(respuestaSuma + "");
19  }
20
21  // Restar
22  public void restar(View view) {
23      Integer respuestaResta = Integer.parseInt(i.getText().toString()) - Integer.parseInt(j.getText().toString());
24      resp.setText(respuestaResta + "");
25  }
26
27  // Multiplicar
28  public void multiplicar(View view) {
29      Integer respuestaMulti = Integer.parseInt(i.getText().toString()) * Integer.parseInt(j.getText().toString());
30      resp.setText(respuestaMulti + "");
31  }
32
33  // Dividir
34  public void dividir(View view) {
35      Integer respuestaDivi = Integer.parseInt(i.getText().toString()) / Integer.parseInt(j.getText().toString());
36      resp.setText(respuestaDivi + "");
37  }
  
```

Figura 24: Funciones de las operaciones básicas de la calculadora

Y por último volvemos a ejecutar la app de ejemplo en el emulador para ver su funcionamiento.

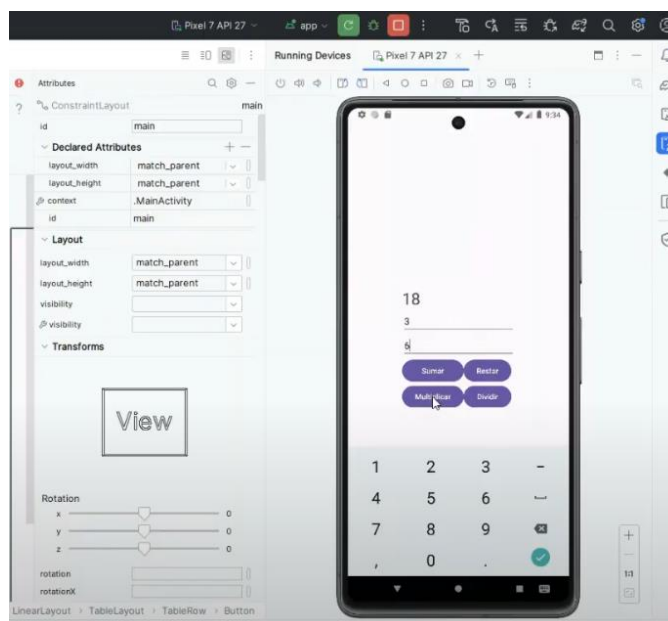


Figura 25: Funcionamiento de la app en el emulador

Subtema 3: Entendiendo el ciclo de vida de aplicaciones, actividades y View Layouts de Android

FASES DEL CICLO DE VIDA DE UNA APP

Las fases de ciclo de vida de una app son las siguientes:

- Planificación
- Desarrollo
- Test
- Lanzamiento
- Monitorización

Planificación. No importa cuánto tiempo dediques a esta fase. Considera que el tiempo invertido aquí te ahorrará tiempo en las etapas posteriores. Es crucial definir claramente los objetivos de la aplicación y las funcionalidades que se deberán implementar para alcanzarlos. (Internet Marketing, s.f)

Desarrollo. Diseñadores y programadores colaboran estrechamente, trabajando codo con codo, para transformar la idea inicial en un producto final que sea coherente y funcional. Los diseñadores se encargan de la estética, la experiencia del usuario y la interfaz, asegurándose de que el producto sea visualmente atractivo y fácil de usar. Mientras tanto, los programadores se concentran en la funcionalidad técnica, escribiendo el código que dará vida a las características y capacidades planificadas. Esta colaboración cercana y continua es esencial para garantizar que todos los aspectos del proyecto se integren perfectamente, resultando en una aplicación que no solo cumpla con los objetivos definidos, sino que también ofrezca una experiencia de usuario satisfactoria y un rendimiento sólido. La sinergia entre diseño y programación permite materializar la visión original de una manera que sea tanto estéticamente agradable como técnicamente robusta. (Internet Marketing, s.f)

Test. Este proceso implica someter la aplicación a pruebas exhaustivas hasta que se detecten errores y fallos. Es crucial que esta fase de pruebas no sea realizada por los propios desarrolladores, sino por personas externas al equipo de desarrollo. Idealmente, estos probadores deberían ser usuarios seleccionados al azar, quienes pueden ofrecer una perspectiva imparcial y representar una variedad de comportamientos y patrones de uso. La diversidad en los probadores asegura que se identifiquen una amplia gama de posibles problemas, desde errores técnicos hasta inconvenientes en la experiencia del

usuario. Al contar con la participación de usuarios reales y no sesgados, se puede obtener una evaluación más precisa de cómo la aplicación funcionará en el mundo real, lo que permite realizar ajustes y mejoras antes de su lanzamiento oficial. (Internet Marketing, s.f)

Lanzamiento. Sobre este punto Internet Marketing (s.f) menciona lo siguiente “en el caso de la versión 1 ten en cuenta que vale más una app publicada que una por publicar. No te quedes en el bucle 1-2-3 más tiempo del estrictamente necesario. La app necesita nacer para desarrollarse. Anota las mejoras para la siguiente versión y lanza la app.”

Monitorización. Muchos desarrolladores y clientes no son conscientes de la importancia de esta etapa del proceso y, en su lugar, intentan adivinar lo que está sucediendo con su aplicación basándose únicamente en el número de descargas. Sin embargo, evaluar el éxito de una app no debería limitarse a esta métrica superficial. Existen numerosas herramientas y métodos que permiten analizar en detalle el comportamiento de los usuarios dentro de la aplicación. Es crucial utilizar analíticas avanzadas para entender cómo los usuarios navegan por la app, qué funciones utilizan más, dónde encuentran dificultades y en qué puntos suelen abandonar la aplicación. Esta información proporciona una visión clara y detallada de la experiencia del usuario, permitiendo identificar áreas de mejora. Paralelamente, es esencial mantener un registro continuo de los errores y bugs que vayan apareciendo. Implementar sistemas de seguimiento y reporte de fallos ayuda a identificar y corregir problemas de manera proactiva, asegurando una mejor estabilidad y rendimiento de la aplicación a largo plazo. Conocer y comprender tanto la conducta de los usuarios como los problemas técnicos es vital para el desarrollo y mantenimiento de una aplicación exitosa. (Internet Marketing, s.f)

¿Qué es una Actividad o Activity?

Una Activity en Android corresponde a una pantalla dentro de nuestra aplicación. En realidad, es un punto de entrada que Android puede cargar en cualquier momento. Se compone de los siguientes elementos (Leiva, 2020):

- **Clase:** Normalmente extiende de AppCompatActivity y es donde definimos el código de lo que queremos que haga la aplicación. (Leiva, 2020)
- **Layout:** Define la apariencia de la vista y el diseño. Se escribe en formato XML, aunque también se puede usar el diseñador visual para simplificar el proceso. (Leiva, 2020)
- **Definición en el AndroidManifest:** Especifica cómo se utilizará la Activity dentro de la aplicación. (Leiva, 2020)

Ciclo de vida de un Activity

En una serie de etapas por las que pasara la pantalla (Activity) en función del estado en el que se encontrara.

onCreate(): Este es el primer paso del ciclo de vida de una Activity. Se invoca inmediatamente después de que la Activity es creada. (Leiva, 2020)

onStart(): Este método se llama cuando la Activity se está iniciando. Se ejecuta siempre después de onCreate(), pero puede ser llamado también cuando la Activity pasa a segundo plano y luego vuelve a ser visible. (Leiva, 2020)

onResume(): Se invoca cuando la Activity está a punto de comenzar a interactuar con el usuario. Siempre se llama después de onStart(), y también puede ser llamado cuando algo aparece sobre la Activity (como un diálogo), pero esta sigue siendo visible. Cuando la Activity recupera el control, se llama de nuevo a onResume(). (Leiva, 2020)

onPause(): Este método es el opuesto de onResume(). Se llama cuando otra actividad aparece delante de la actual, aunque esta siga siendo parcialmente visible. (Leiva, 2020)

onStop(): Este método es el opuesto de onStart(). Se llama cuando la Activity ya no es visible para el usuario, ya sea porque otra Activity ha sido lanzada o la aplicación ha pasado a segundo plano. (Leiva, 2020)

onDestroy(): Este método es el opuesto de onCreate(). Se invoca justo antes de que la Activity sea destruida. Es lo último que se llama antes de que la Activity desaparezca completamente. (Leiva, 2020)

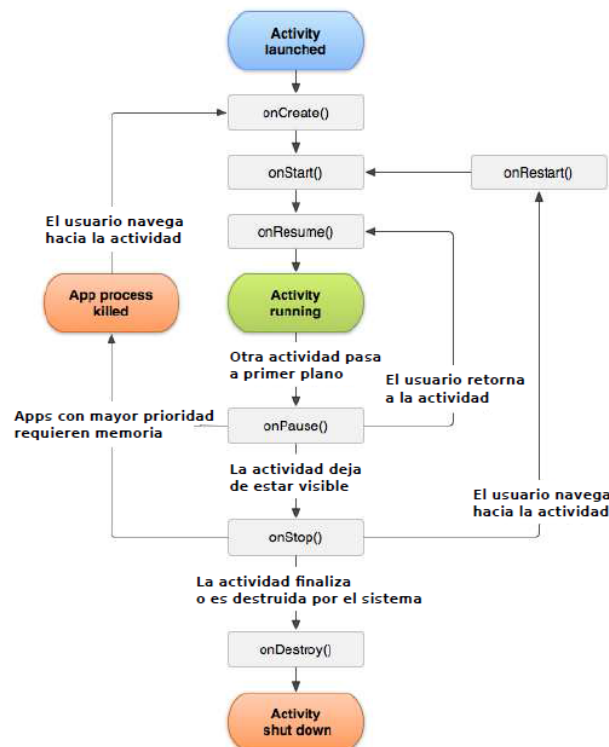


Figura 26: Ciclo de vida de un Activity Fuente: Cánovas, 2014 Recuperado de <https://juanjosecanbus.wordpress.com/2014/09/28/practica-1-2-ciclo-de-vida-de-una-app-de-android-pmm/>

VIEW

Es un componente que permite gestionar la interacción del usuario con la aplicación. Son muy similares a los controles SWING de Java, como etiquetas (Labels), botones (Buttons), campos de texto (TextFields), casillas de verificación (Checkboxes), entre otros. Los Views se organizan dentro de los Layouts para que el usuario pueda entender claramente los objetivos de la actividad. (develou, s.f)

Atributos que posee un View

layout:height Representa la dimensión vertical de un View. Puedes asignarle valores absolutos en dps, dependiendo de las métricas de diseño que manejes, o usar los valores `match_parent` y `wrap_content`. El primero ajusta la dimensión a las medidas del contenedor padre, mientras que el segundo ajusta el tamaño al contenido del View. (develou, s.f)

layout:width Representa la dimensión horizontal de un View. (develou, s.f)

layout:layout_margin Especifica los márgenes del View con respecto a otros componentes. Puedes definir los valores individualmente para los márgenes izquierdo, derecho, superior e inferior, o puedes especificar un mismo valor para todos los márgenes. (develou, s.f)

layout:alignComponent Define la proximidad de los márgenes entre componentes. Por ejemplo, esto permite alinear el margen izquierdo de un botón con el margen izquierdo de otro botón adyacente. Si en algún momento el botón se mueve, su compañero se ajustará en consecuencia. Este atributo destaca la versatilidad de un RelativeLayout. (develou, s.f)

layout:alignParent Este atributo permite especificar que un View estará situado en uno de los lados de su contenedor padre. (develou, s.f)

layout:centerInParent Este atributo permite centrar un View tanto horizontal como verticalmente dentro de su contenedor padre. (develou, s.f)

id Es un identificador que distingue los distintos Views entre sí. Es útil cuando necesitas referenciar los controles en el código Java. Por lo tanto, elige nombres que sean representativos y fáciles de interpretar. (develou, s.f)

Layout

Un layout es un elemento que representa el espacio contenedor para todas las vistas (Views) dentro de una actividad. Define la disposición y el orden de los elementos para permitir la interacción del usuario con la interfaz. Estos layouts se implementan mediante subclases Java que heredan de la clase ViewGroup. (develou, s.f)

Para crearlos, se escribe un documento XML con un tipo de layout como nodo raíz, y luego se añaden los elementos secundarios dentro de él. Android Studio facilita la generación automática del código XML mediante el panel de Diseño. (develou, s.f)

Tipos de Layout

Existen diversos y su empleo dependerá de la necesidad que tenga el usuario, pero en general veamos tres de los más populares.

LinearLayout: develou, (s.f) menciona que “El Linear Layout es el más sencillo. Dentro de él los elementos son ubicados en forma lineal a través de columnas o filas. Posee un atributo que permite modificar su orientación, ya sea para presentar los elementos horizontal o Verticalmente.”

WebView: develou, (s.f) menciona que “Este View fue creado para mostrar el contenido con formato web.”

RelativeLayout: develou, (s.f) menciona que “Este es el layout más recomendado a usar, ya que los componentes dentro de él se pueden organizar de forma relativa entre sí.”

Subtema 4: Trabajar con componentes de Android

Algunos componentes en Android son los siguientes:

- Vista (View)
- Actividad (Activity)
- Fragmentos (Fragments)
- Servicio (Service)
- Intención (Intent)
- Receptor de anuncios (Broadcast Receiver)
- Proveedores de Contenido (Content Provider)

Vista (View)

La vista es un componente fundamental en las aplicaciones Android, ya que engloba todos los elementos visuales de la aplicación, como botones, imágenes, textos, listas desplegables, cuadros de texto, e incluso controles personalizados. Dentro de las vistas, también se encuentran los layouts, que son conjuntos de vistas agrupadas para lograr un diseño específico en la aplicación. (3Androides, s.f)

Actividad (Activity)

Cada pantalla o ventana que compone una aplicación se denomina actividad (Activity), y en conjunto, forman lo que se conoce como la interfaz de usuario. Cada actividad dentro de una aplicación tiene su propio diseño visual definido por su propio layout. (3Androides, s.f)

Fragmentos (Fragments)

Un fragmento es un elemento de una aplicación Android que combina varias vistas para formar un bloque más funcional. Este componente posibilita que las aplicaciones se ajusten de manera más eficiente a diferentes tamaños de pantalla u orientaciones, lo que facilita el diseño responsivo. (3Androides, s.f)

Servicio (Service)

Cuando nos referimos a un servicio como componente de una aplicación en Android, estamos hablando de una función o tarea que se ejecuta de forma oculta para el usuario. Este componente se puede dividir en dos tipos principales (3Androides, s.f):

- **Servicio local:** Estos servicios se ejecutan en el mismo proceso y dispositivo donde se encuentra la aplicación. (3Androides, s.f)

- **Servicio remoto:** Estos servicios se ejecutan en procesos separados y de forma externa al dispositivo donde se encuentra la aplicación. (3Androides, s.f)

Intención (Intent)

El concepto de intención puede resultar un poco complejo al entender cómo se implementa en Android. Se trata de un componente que representa la intención de realizar una acción específica dentro de la aplicación, como realizar una llamada telefónica, enviar un mensaje de texto, transmitir un mensaje de difusión (broadcast), o iniciar un servicio particular. (3Androides, s.f)

Receptor de anuncios (Broadcast Receiver)

El componente Broadcast Receiver se encarga de manejar los avisos que recibe la aplicación, ya sea del propio dispositivo o de otras aplicaciones instaladas. Un ejemplo simple para comprender qué es el componente BroadcastReceiver de Android es un aviso del sistema sobre la batería baja. (3Androides, s.f)

Estos mensajes de Android no están dirigidos a una aplicación específica, sino que son enviados a cualquier componente que esté interesado en recibirlos y gestionarlos. (3Androides, s.f)

Proveedores de Contenido (Content Provider)

Este componente representa el sistema nativo de Android para compartir datos entre aplicaciones en un entorno seguro que garantiza la privacidad e integridad de la información. (3Androides, s.f)

Gracias al Content Provider, es posible acceder a los datos de otras aplicaciones, como la lista de contactos y la información asociada a ellos. (3Androides, s.f)

Preguntas de Comprensión de la Unidad

1. ¿Qué se denomina a cada pantalla o ventana que compone una aplicación en Android, y qué componente define su diseño visual?

Cada pantalla o ventana que compone una aplicación en Android se denomina actividad (Activity), y su diseño visual está definido por su propio layout.

2. ¿Cuál es el papel de las vistas en las aplicaciones Android y cómo se relacionan con los layouts?

Las vistas en las aplicaciones Android son componentes fundamentales que abarcan todos los elementos visuales, como botones, imágenes, textos, listas desplegables, cuadros de texto e incluso controles personalizados. Los layouts, por otro lado, son conjuntos de vistas agrupadas para lograr un diseño específico en la aplicación.

3. ¿Qué es el Java Development Kit (JDK) y qué componentes incluye para los desarrolladores de Java?

El Java Development Kit (JDK) es un software esencial para los desarrolladores de Java. Incluye el intérprete Java, clases Java y una variedad de herramientas de desarrollo Java (JDT), como el compilador, el depurador, el desensamblador, el visor de applets, el generador de archivos de apéndice y el generador de documentación.

4. ¿Para qué se utiliza Android Device Manager y qué permite crear y configurar?

Android Device Manager se utiliza para crear y configurar dispositivos virtuales Android (AVD) que se ejecutan en Android Emulator. Cada AVD es una configuración del emulador que simula un dispositivo Android físico. Esto permite ejecutar y probar la aplicación en diversas configuraciones que simulan diferentes dispositivos Android físicos.

Material complementario

Los siguientes recursos complementarios son sugerencias para que se pueda ampliar la información sobre el tema trabajado, como parte de su proceso de aprendizaje autónomo:

Videos de apoyo:

- https://www.youtube.com/watch?v=BQaxPwZWboA&list=PLNdFk2_brsRdYF0FXDtSaGvluzBNHRbNe
- https://www.youtube.com/watch?v=X5fsU0Oxlg&list=PLlygiKpYTC_6XbaH_E39-cBEfqlosadAq
- <https://www.youtube.com/watch?v=5tfEAKi38DA>
- <https://www.youtube.com/watch?v=kVUCXmRrGQc>

Links de apoyo:

- https://developer.android.com/studio?gad_source=1&gclid=Ci0KCOjwiLGyBhCYARIsAPqTz1-uRTvufIc1Y2Kp7NRi5xrg_mN3MuyY-1jzZcA3stVTCf0lCbfsPOlaAqXNEALw_wcB&gclsrc=aw.ds
- <https://www.oracle.com/java/technologies/downloads/>

Bibliografía

- » Santaella, J. (2022). ¿Qué es Android Studio?. Recuperado de <https://talently.tech/blog/que-es-android-studio/>
- » Google Developers. (2024). Descarga e instala Android Studio. Recuperado de <https://developer.android.com/codelabs/basic-android-kotlin-compose-install-android-studio?hl=es-419#1>
- » IBM. (2021). Java Development Kit. Recuperado de <https://www.ibm.com/docs/es/i/7.3?topic=platform-java-development-kit>
- » Internet Marketing. (s.f). CICLO DE VIDA DE UNA APP. Recuperado de <https://www.imk.es/blog/ciclo-de-vida-de-una-app/>
- » Leiva, A. (2020). Qué es una Activity, el componente más importante de una App Android. Recuperado de <https://devexpert.io/activity-android/>
- » develou. (s.f). Configurar Layouts y Views En Android Studio. Recuperado de <https://www.develou.com/android-layouts-views/#:~:text=Es%20un%20componente%20que%20permite,los%20objetivos%20de%20la%20actividad.>
- » 3Androides. (s.f). Componentes de una aplicación Android. Recuperado de <https://www.3androides.com/actualidad/280-componentes-de-una-aplicacion-android>



UNEMI
UNIVERSIDAD ESTATAL DE MILAGRO

Compendio del autor

COMPUTACIÓN MÓVIL

UNIDAD 3
DESARROLLO DE APLICACIONES MÓVILES

ÍNDICE

Contenido

Unidad 3: DESARROLLO DE APLICACIONES MÓVILES	3
<i>Tema 2: Frameworks de desarrollo móvil</i>	<i>3</i>
<i>Objetivo:</i>	<i>3</i>
<i>Introducción:</i>	<i>3</i>
Información de los subtemas.....	4
<i>Subtema 1: Introducción a Flutter</i>	<i>4</i>
<i>Subtema 2: Flutter interacción con el usuario, estilos y animaciones</i>	<i>8</i>
<i>Subtema 3: Introducción a la gestión del estado y validación de formularios en Flutter</i>	<i>13</i>
<i>Subtema 4: Enfoque para gestionar el estado y patrón Modelo Vista Controlador</i>	<i>17</i>
Preguntas de Comprensión de la Unidad	20
Material complementario	21
Bibliografía.....	22

Unidad 3: DESARROLLO DE APLICACIONES MÓVILES

Tema 2: Frameworks de desarrollo móvil

Objetivo:

Revisar aspectos y herramientas para la codificación de aplicaciones móviles utilizando tecnologías de Desarrollo Nativo y Framework.

Introducción:

Los frameworks de desarrollo móvil son conjuntos de herramientas, bibliotecas y componentes predefinidos que permiten a los desarrolladores crear aplicaciones móviles de manera más eficiente y rápida. Estos frameworks proporcionan una base sólida para construir aplicaciones para dispositivos móviles, abstrayendo muchas de las complejidades del desarrollo y ofreciendo características comunes y funcionalidades integradas.

Estos frameworks suelen ofrecer una variedad de características, como soporte multiplataforma, lo que permite a los desarrolladores escribir código una vez y ejecutarlo en múltiples plataformas, como iOS y Android. Además, ofrecen herramientas para la creación de interfaces de usuario atractivas y receptivas, acceso a funcionalidades del dispositivo, como la cámara o el GPS, y capacidades de prueba y depuración integradas.

Flutter es un marco de desarrollo móvil de código abierto creado por Google, diseñado para ayudar a los desarrolladores a construir aplicaciones móviles hermosas y de alto rendimiento de manera rápida y eficiente. Utiliza el lenguaje de programación Dart y ofrece una amplia gama de widgets personalizables que permiten a los desarrolladores crear interfaces de usuario atractivas y receptivas.

Información de los subtemas

Subtema 1: Introducción a Flutter

Según AWS (s.f) “Flutter es un marco de código abierto desarrollado y compatible con Google. Los desarrolladores de front-end y pila completa utilizan Flutter para crear una interfaz de usuario (IU) de aplicación para varias plataformas con un único código base. Cuando Flutter se lanzó, en 2018, era compatible principalmente con el desarrollo de aplicaciones móviles. Ahora, Flutter es compatible con el desarrollo de aplicaciones en seis plataformas: iOS, Android, web, Windows, MacOS y Linux.”

Por otro lado, Méndez (2019) menciona “Flutter es el kit de herramientas de interfaz de usuario de Google para crear hermosas aplicaciones compiladas de forma nativa para dispositivos móviles, web y de escritorio desde una única base de código, flutter usa el lenguaje de Dart.”

Por su parte Cristancho (2022) indica “Flutter es un framework que permite el desarrollo de un proyecto de programación. Es gratuito y de código abierto, y fue creado por Google en mayo de 2017. Básicamente, permite crear una aplicación móvil nativa con una sola base de código. ¿Qué significa esto? Que puede usar un lenguaje de programación y una base de código para crear dos aplicaciones diferentes (para iOS y Android). Esta es, quizás, la principal ventaja de lo que es Flutter y lo que lo hace súper valioso.”

CARACTERÍSTICAS DE FLUTTER

Rendimiento nativo

Los widgets disponibles en Flutter ya incorporan las diferencias críticas entre varias plataformas, como el desplazamiento, la navegación, los íconos y las fuentes. Esto permite ofrecer un rendimiento nativo tanto en iOS como en Android. (Cristancho, 2022)

Flutter emplea el lenguaje de programación Dart y se compila en código máquina. Este código es comprendido por los dispositivos host, lo que asegura un rendimiento rápido y eficiente. (AWS, s.f)

Desarrollo Rápido

En solo unos segundos, Flutter permite dar vida a una aplicación. La función Hot Reload facilita el uso de un conjunto completo de widgets personalizables para crear interfaces nativas de manera rápida y eficiente, además de acelerar la corrección de errores. Asimismo, los tiempos de recarga son inferiores a un segundo y no se pierde el estado, ya sea en emuladores, simuladores o dispositivos para iOS y Android. (Cristancho, 2022)

UI expresiva y flexible

Flutter permite diseñar rápidamente funcionalidades, enfocándose en ofrecer una experiencia de usuario nativa. Según el sitio web, "la arquitectura en capas proporciona una personalización total, lo que resulta en un renderizado extremadamente rápido y en diseños expresivos y flexibles." El catálogo de widgets incluye elementos visuales, estructurales, de plataforma e interactivos. (Cristancho, 2022)

Rendimiento rápido, consistente y personalizable

En lugar de confiar en herramientas de renderización específicas de la plataforma, Flutter utiliza la biblioteca gráfica de código abierto Skia de Google para renderizar la interfaz de usuario. Esto garantiza una experiencia visual uniforme para los usuarios, independientemente de la plataforma que utilicen para acceder a una aplicación. (AWS, s.f)

Herramientas para desarrolladores

El objetivo de Google es hacer que Flutter sea accesible y sencillo de utilizar. Funciones como la recarga en caliente permiten a los desarrolladores obtener una vista previa de cómo lucen los cambios de código sin perder el estado actual. Además, herramientas como el inspector de widgets simplifican la visualización y la solución de problemas relacionados con los diseños de la interfaz de usuario. (AWS, s.f)

Qué ocupa o qué compone Flutter

- SDK (Software Development Kit): Cristancho (2022) menciona "se trata de una colección de herramientas que permite desarrollar aplicaciones. Esto incluye elementos para compilar código para iOS y Android."
- Framework (Biblioteca de interfaz de usuario basada en widgets): Cristancho (2022) menciona "una colección de elementos de interfaz de

usuario reutilizables (por ejemplo: botones, entradas de texto, controles deslizantes, etc.) que pueden personalizarse según lo que requiera el proyecto.”

¿Qué lenguaje de programación emplea Flutter?

Flutter emplea el lenguaje de programación Dart, de código abierto, que fue también desarrollado por Google. Dart está especialmente optimizado para la creación de interfaces de usuario, y muchos de sus puntos fuertes se aprovechan en Flutter. (AWS, s.f)

Por ejemplo, una funcionalidad destacada de Dart que se utiliza en Flutter es la seguridad en la gestión de nulos. Esta característica permite detectar fácilmente los errores más comunes, conocidos como errores de nulos. Tal aspecto contribuye a reducir el tiempo empleado por los desarrolladores en el mantenimiento del código, brindándoles así más tiempo para concentrarse en la construcción de sus aplicaciones. (AWS, s.f)

Importancia de aprender flutter

Fácil de aprender y usar: Cristancho (2022) menciona “es un framework moderno y sencillo. Puedes crear grandes apps, sin necesidad de tanto código. Si eres desarrollador devops, o vienes de otro rubro tech, seguro notarás la diferencia entre otros lenguajes más complejos y Flutter.”

Compilación rápida y máxima productividad: Cristancho (2022) menciona “Flutter permite cambiar el código y ver los resultados en tiempo real.”

“Todo es un widget”: Cristancho (2022) menciona “el lema de Flutter ofrece numerosas ventajas para el desarrollador.”

Widgets en Flutter

En Flutter, los desarrolladores construyen la interfaz de usuario utilizando widgets. Esto implica que todos los elementos visibles en la pantalla, como ventanas, paneles, botones y textos, están compuestos por widgets. (AWS, s.f)

Los widgets en Flutter se diseñan con el objetivo de facilitar su personalización por parte de los desarrolladores. Este objetivo se logra mediante un enfoque de composición, donde la mayoría de los widgets están formados por otros más pequeños, y los widgets más básicos tienen propósitos específicos. Este enfoque permite a los desarrolladores combinar o modificar widgets para crear otros nuevos. (AWS, s.f)

Flutter representa los widgets utilizando su propio motor gráfico en lugar de depender de los widgets incorporados en la plataforma. Esto asegura una apariencia coherente en una aplicación Flutter en todas las plataformas. Además, este enfoque brinda flexibilidad a los desarrolladores, ya que algunos widgets de Flutter pueden realizar funciones que los widgets específicos de la plataforma no pueden llevar a cabo. (AWS, s.f)

Flutter incluye un extenso conjunto de widgets desde el momento en que se descarga. Este catálogo abarca 14 categorías diferentes, que comprenden estilos generales, widgets de estilo Cupertino (inspirados en iOS) y componentes de Material Design (siguiendo las pautas de diseño de Google). (AWS, s.f)

En Flutter, todo dentro de una aplicación se representa como un widget, desde elementos simples como texto y botones hasta diseños completos de pantalla. Estos widgets se organizan jerárquicamente para su visualización en la pantalla. Los widgets que tienen la capacidad de contener otros widgets se conocen como widgets contenedor. La mayoría de los widgets de diseño son widgets contenedor, a excepción de aquellos que realizan funciones mínimas, como el "Text Widget". (Méndez, 2019)

Se tiene dos tipos de widgets al momento de escribir una aplicación, generalmente se creará nuevos widgets como subclases:

- **Stateless.** Según Méndez (2019) "Un widget sin estado es fácil de entender ya que es un widget que no cambia por ejemplo un simple Button o un Text."
- **Stateful.** Según Méndez (2019) "Un widget con estado es dinámico: por ejemplo, puede cambiar su apariencia en respuesta a eventos desencadenados por las interacciones del usuario o cuando recibe datos. Checkbox , Radio , Slider , InkWell , Form y TextField son ejemplos de widgets con estado. Subclase de widgets con estado StatefulWidget ."

Subtema 2: Flutter interacción con el usuario, estilos y animaciones

Interacción con el usuario

En el mundo actual, los usuarios de aplicaciones móviles son cada vez más exigentes. Esperan experiencias fluidas y una estética impresionante. Como desarrollador, es crucial mantenerse al día con los estándares de la industria y estar preparado para enfrentar este desafío. (Jain, s.f)

Flutter se emplea para desarrollar aplicaciones móviles multiplataforma a partir de un único código base. Se puede usar para crear aplicaciones para cualquier navegador web y diversos sistemas operativos, como Android, iOS, Linux, macOS y Windows. La popularidad de Flutter está creciendo tanto que algunas organizaciones ahora gestionan sus negocios exclusivamente con esta tecnología, desarrollando aplicaciones innovadoras. (Jain, s.f)

En base a esto es importante tomar en consideración lo siguiente:

Elegir la tipografía y la paleta de colores adecuadas

Seleccionar la paleta de colores y la tipografía adecuadas es fundamental para crear interfaces de usuario atractivas con Flutter. Elegir colores y fuentes que evocan emociones y establecen una jerarquía visual puede mejorar significativamente la experiencia general del usuario. (Jain, s.f)

Empiece seleccionando una colección visual de colores, imágenes e inspiración que refleje la apariencia deseada de su aplicación. También es crucial comprender la psicología del color, ya que los colores pueden transmitir diversas emociones. (Jain, s.f)

Además, utilice diferentes tamaños y estilos para dirigir la atención de los usuarios. Destaque la información importante y utilice la información secundaria como apoyo. Esto ayudará a crear una jerarquía visual clara de la información. (Jain, s.f)

Creación de diseños receptivos y consistentes

Para crear una aplicación que luzca bien en diferentes dispositivos y tamaños de pantalla, un diseño bien planificado es fundamental. Adopte un enfoque basado en cuadrículas y divida la pantalla de su aplicación en filas y columnas. Esto facilitará la alineación de los elementos de la interfaz de usuario. (Jain, s.f)

La interfaz de usuario de Flutter cuenta con una amplia variedad de widgets de diseño, como "Row," "Column," y "Container," que pueden asistirlo en estructurar

su aplicación en forma de cuadrícula. Además, asegúrese de mantener un espacio constante entre los elementos de la interfaz de usuario. Puede lograrlo utilizando las propiedades de margen y padding de Flutter. Esto le permitirá crear un diseño visualmente equilibrado. (Jain, s.f)

Además, también puede emplear los widgets "Flexible y extendido ", que permiten que los elementos de la interfaz de usuario ajusten automáticamente su tamaño y posicionamiento de acuerdo con el espacio disponible. (Jain, s.f)

Dominar widgets de Flutter para interacciones intuitivas de usuario

Los widgets son los elementos visuales que constituyen la interfaz de usuario de su aplicación, y Flutter ofrece una amplia biblioteca de widgets. Puede concebir los widgets como bloques de construcción LEGO que puede ensamblar para crear la interfaz de usuario deseada. (Jain, s.f)

Además, Flutter emplea un "Árbol de widgets", que es una estructura jerárquica entre widgets. Puede utilizar este árbol para gestionar la relación padre-hijo entre los widgets. También puede aprovechar el catálogo de widgets de Flutter, que constituye una amplia colección de widgets predefinidos que incluyen diversos elementos de la interfaz de usuario, como botones, campos de texto y más. (Jain, s.f)

Estilos

Flutter ofrece una amplia variedad de widgets predefinidos listos para ser utilizados. Además, brinda la posibilidad de crear widgets personalizados según las necesidades específicas de cada proyecto. (Téllez, 2024)

Widgets de estilo

Dentro de estos podemos nombrar 3 que son Padding, Theme y MediaQuery. (Téllez, 2024)

Theme

A través de este widget, podemos gestionar el tema de toda nuestra aplicación Flutter. Se puede aplicar de manera individual a todos los widgets que utilicemos, y su impacto principalmente afecta a los colores y la tipografía. (Téllez, 2024)

La configuración del tema se realiza desde nuestro archivo main.dart, dentro de la clase MaterialApp, utilizando el atributo "theme", donde se pasa el objeto ThemeData. Este objeto contiene todas las especificaciones necesarias para configurar el tema de todos los widgets hijos dentro de MaterialApp (Figura 1). (Téllez, 2024)


```
@override
Widget build(BuildContext context) {
  return MaterialApp(
    theme: ThemeData(),
    home: const MyHomePage(title: 'Flutter'),
  );
}
```

Figura 1: Configuración del Theme. Fuente: Téllez, J. 2024. Recuperado de <https://medium.com/@jaimetellezb/flutter-widgets-de-estilo-023bb3d7215f>

Los widgets hijos pueden acceder al tema global utilizando el método `Theme.of`, que recibe el contexto actual (`BuildContext`) y devuelve un objeto `ThemeData`. Los componentes de Material suelen depender de las propiedades `colorScheme` y `textTheme`. (Téllez, 2024)

```
// declaración para obtener todos los textTheme
final textTheme = Theme.of(context).textTheme;

// dentro de un Text
Text(
  widget.title,
  style: textTheme.titleLarge,
)
```

Figura 2: Declaración de un widget hijo. Fuente: Téllez, J. 2024. Recuperado de <https://medium.com/@jaimetellezb/flutter-widgets-de-estilo-023bb3d7215f>

Propiedades de ThemeData

- **colorScheme**: es un conjunto de 30 colores basados en Material que se aplican a diferentes componentes. (Téllez, 2024)
- **textTheme**: permite configurar los estilos de texto de la aplicación. (Téllez, 2024)
- **appBarTheme**: se utiliza para personalizar el tema de los AppBar. (Téllez, 2024)
- **bottomNavigationBarTheme**: permite personalizar el tema de los BottomNavigationBar. (Téllez, 2024)
- **brightness**: determina si el fondo del tema será blanco o negro, utilizando las propiedades `Brightness.light` y `Brightness.dark`. (Téllez, 2024)
- **cardColor**: gestiona el color y los estilos del widget Card. (Téllez, 2024)
- **extensions**: se utiliza para agregar nuevos temas personalizados. (Téllez, 2024)
- **typography**: administra el color y la geometría de los textos. (Téllez, 2024)
- **useMaterial3**: una propiedad que se establece en `false` para desactivar los diseños de Material 3 y en `true` para activarlos. En las últimas versiones de Flutter, viene establecida en `true` por defecto. (Téllez, 2024)

Padding

El widget `Padding` nos permite agregar un espacio alrededor de su widget hijo, que se rellena con el color de fondo. (Téllez, 2024)

Propiedades de `Padding`

- **`padding`**. Esta es la propiedad que define los márgenes en píxeles desde los bordes hasta el widget hijo. Recibe un objeto `EdgeInsets` que especifica las cuatro direcciones cardinales: izquierda (`left`), arriba (`top`), derecha (`right`) y abajo (`bottom`). (Téllez, 2024)

```
Card(
  color: colorScheme.primaryContainer,
  child: const Text('Sin Padding'),
),
Card(
  color: colorScheme.primaryContainer,
  child: Padding(
    padding: const EdgeInsets.all(20.0),
    child: Text(
      'Con Padding all',
      style: textTheme.labelLarge,
    ),
  ),
),
Card(
  color: colorScheme.primaryContainer,
  child: Padding(
    padding: const EdgeInsets.symmetric(horizontal: 20.0),
    child: Text(
      'Con Padding symmetric vertical',
      style: textTheme.labelLarge,
    ),
  ),
),
Card(
  color: colorScheme.primaryContainer,
  child: Padding(
    padding: const EdgeInsets.only(top: 20.0, left: 10.0),
    child: Text(
      'Con Padding only bottom left',
      style: textTheme.labelLarge,
    ),
  ),
),
```

Figura 3: Uso del `padding`. Fuente: Téllez, J. 2024. Recuperado de <https://medium.com/@jaimetellezb/flutter-widgets-de-estilo-023bb3d7215f>

MediaQuery

MediaQuery nos permite utilizar el tamaño de la pantalla de manera efectiva en nuestros widgets. Para obtener el tamaño de la pantalla actual, podemos usar `MediaQuery.sizeOf(context)`. También es posible obtener el padding actual con `MediaQuery.paddingOf(context)`. Aunque hay muchas otras propiedades disponibles, las anteriores son las más comunes. Para obtener más información, podemos consultar `MediaQueryData`. (Téllez, 2024)

```
final currentSize = MediaQuery.sizeOf(context);
print(currentSize);
// flutter: Size(430.0, 932.0)

final currentPadding = MediaQuery.paddingOf(context);
print(currentPadding);
// aquí imprime en el orden: left, top, right, bottom
// flutter: EdgeInsets(0.0, 59.0, 0.0, 34.0)
```

Figura 4: Uso de MediaQuery. Fuente: Téllez, J. 2024. Recuperado de <https://medium.com/@jaimetellezb/flutter-widgets-de-estilo-023bb3d7215f>

Animaciones

Flutter clasifica las animaciones basadas en código en dos categorías: implícitas y explícitas. (Escacena, 2022)

Las animaciones **implícitas**, según la documentación de Flutter, se fundamentan en establecer un valor inicial de una propiedad de un widget, definir el valor final y luego Flutter se encarga de ejecutar las animaciones para transicionar entre estos valores. Este proceso, conocido como interpolación, implica calcular los valores de animación entre una posición inicial y una posición final. Las animaciones implícitas son un punto de partida óptimo para adentrarse en las animaciones en Flutter. (Escacena, 2022)

Por otro lado, para determinar si necesitamos una animación **explícita**, podemos considerar estas tres preguntas (Escacena, 2022):

- ¿La animación se repite en un ciclo?
- ¿Los valores de la animación son discontinuos? Por ejemplo, ¿la animación implica cambios abruptos en el tamaño o posición del objeto animado?
- ¿Estamos animando varios widgets de manera coordinada?

Si la respuesta es afirmativa a alguna de estas preguntas, es probable que necesitemos una animación explícita. Estas requieren la implementación de un controlador (`AnimationController`) que define cuándo y cómo se realiza la animación. (Escacena, 2022)

Subtema 3: Introducción a la gestión del estado y validación de formularios en Flutter

Gestión del estado

El estado refleja la situación actual de la aplicación o de sus componentes. Por ejemplo, al hacer clic en un checkbox, se produce un cambio en la situación de la aplicación, lo que implica una modificación en su estado. (Herrera, 2022)

La gestión de estado implica manejar los cambios en los datos que controlan la interfaz de usuario de una aplicación. En Flutter, esto implica definir cómo se almacenan, actualizan y transfieren los datos dentro de la aplicación. (Chiva, s.f)

Estados efímeros

El estado efímero, también conocido como estado de UI o estado local, es aquel que puede ser gestionado por un solo widget. (Herrera, 2022) Por ejemplo:

- La pestaña actual seleccionada de un BottomNavigationBar. (Herrera, 2022)
- La selección de un CheckBox. (Herrera, 2022)
- El estado basado en la activación de un botón. (Herrera, 2022)

Estado de aplicación (Global)

Este estado, en contraste con el efímero, no es limitado en el tiempo y es necesario compartirlo a lo largo de la aplicación, a veces incluso mantenerlo entre sesiones de usuario. Se le conoce como estado compartido. (Herrera, 2022)

Algunos ejemplos incluyen:

- Preferencias elegidas por el usuario. (Herrera, 2022)
- Estado de la autenticación de usuario. (Herrera, 2022)
- Un carrito de compras en la aplicación de comercio electrónico de una empresa. (Herrera, 2022)

Estrategias de Gestión de Estado

StatefulWidget y setState().

- `setState()` es el método utilizado para actualizar la interfaz de usuario cuando cambia el estado de un widget en Flutter. (Chiva, s.f)
- **Uso práctico:** Se recomienda para cambios simples en la interfaz de usuario, como animaciones o actualizaciones de texto. (Chiva, s.f)

- **Ventajas y desventajas:** Es fácil de implementar, pero puede volverse complicado de manejar para estados más complejos o compartidos entre múltiples widgets. (Chiva, s.f)

Provider

- **Uso práctico:** Es útil para gestionar y compartir el estado a través de varios niveles de widgets, especialmente en aplicaciones medianas. (Chiva, s.f)
- **Ventajas y desventajas:** Es más escalable que setState(), pero puede resultar excesivo para estados muy simples debido a la complejidad adicional que introduce. (Chiva, s.f)

Riverpod

- **Uso práctico:** El Provider evolucionado ofrece una gestión de estado aún más modular y reutilizable, lo cual es excelente para aplicaciones grandes y complejas. (Chiva, s.f)
- **Ventajas y desventajas:** Proporciona un alto grado de control y flexibilidad en el manejo del estado, pero puede tener una curva de aprendizaje más pronunciada debido a su mayor complejidad. (Chiva, s.f)

Formularios y validación

Las aplicaciones frecuentemente necesitan que los usuarios ingresen información en campos de texto. Por ejemplo, al desarrollar una aplicación, es posible que necesitemos que los usuarios inicien sesión proporcionando una combinación de dirección de correo electrónico y contraseña. (Alejandro, 2018)

Para garantizar la seguridad y la facilidad de uso de nuestras aplicaciones, es fundamental validar la información proporcionada por el usuario. Si el usuario completa correctamente el formulario, podemos proceder a procesar la información. Sin embargo, si el usuario proporciona información incorrecta, podemos mostrar un mensaje de error amigable que indique dónde se produjo el error para que el usuario pueda corregirlo. (Alejandro, 2018)

Crear Formulario

Inicialmente, necesitaremos un widget Form para manejar la interacción con el formulario. Este widget funciona como un contenedor para agrupar y validar varios campos de formulario simultáneamente. (Alejandro, 2018)

Al crear el formulario, también será necesario asignar una **GlobalKey**. Esta clave proporcionará una identificación única para el formulario en cuestión,

permitiéndonos validar el formulario en etapas posteriores del proceso. (Alejandro, 2018)

```
class MyCustomForm extends StatefulWidget {  
  @override  
  MyCustomFormState createState() {  
    return MyCustomFormState();  
  }  
}  
  
class MyCustomFormState extends State<MyCustomForm> {  
  GlobalKey<MyCustomFormState>  
    final _formKey = GlobalKey<FormState>();  
  
  @override  
  Widget build(BuildContext context) {  
    return Form(  
      key: _formKey,  
      child: // We'll build this out in the next steps!  
    );  
  }  
}
```

Figura 5: Creación de un widget Form con una GlobalKey Fuente: Alejandro, 2018. Recuperado de <https://medium.com/@flutterman/flutter-validaci%C3%B3n-de-formularios-f18ae3e8c60a>

Añadir un campo TextFormField con la lógica correspondiente de validación

Ahora que hemos configurado nuestro formulario, es momento de proporcionar a los usuarios un medio para ingresar texto. Aquí es donde entra en juego el widget **TextFormField**. Este widget muestra un campo de entrada de texto siguiendo los principios de diseño de material, y también maneja la presentación de errores de validación en caso de que ocurran. (Alejandro, 2018)

Al asignar una función **validator** al campo **TextFormField**, estamos estableciendo un mecanismo para validar la información ingresada por el usuario. Esta función debe retornar una cadena que describa el error si la información no es válida. Por otro lado, si la información es correcta, la función debe devolver null. (Alejandro, 2018)

```
TextFormField(  
  validator: (value) {  
    if (value.isEmpty) {  
      return 'Please enter some text';  
    }  
  },  
);
```

Figura 6: Añadir un TextFormField con validación. Fuente: Alejandro, 2018. Recuperado de <https://medium.com/@flutterman/flutter-validaci%C3%B3n-de-formularios-f18ae3e8c60a>

Agregar un botón para validar y enviar la información del formulario

Una vez que hemos establecido un formulario con un campo de texto, es necesario incluir un botón que permita al usuario enviar la información ingresada. Al intentar enviar el formulario, es crucial verificar su validez. En caso de ser válido, se muestra un mensaje de éxito. No obstante, si el campo de texto está vacío, se debe mostrar un mensaje de error correspondiente. (Alejandro, 2018)

```
RaisedButton(  
  onPressed: () {  
    if (_formKey.currentState.validate()) {  
      Scaffold  
        .of(context)  
        .showSnackBar(SnackBar(content: Text('Processing Data')));  
    }  
  },  
  child: Text('Submit'),  
);
```

Figura 7: Agregar botón para enviar la información del formulario y validarla. Fuente: Alejandro, 2018. Recuperado de <https://medium.com/@flutterman/flutter-validaci%C3%B3n-de-formularios-f18ae3e8c60a>

Subtema 4: Enfoque para gestionar el estado y patrón Modelo Vista Controlador

En el ámbito del desarrollo de software, los patrones arquitectónicos son fundamentales para la creación de aplicaciones que sean escalables, fáciles de mantener y sólidas. Uno de estos patrones ampliamente adoptados es el Modelo-Vista-Controlador (MVC), que divide las responsabilidades de una aplicación en tres componentes principales: el modelo, la vista y el controlador. (Rhm Faiz, 2023)

Modelo: Contiene los datos y la lógica de negocio de la aplicación, encapsulando los datos y ofreciendo métodos para su interacción y manipulación. (Rhm Faiz, 2023)

Vista: Es la representación de la interfaz de usuario (UI) de la aplicación, mostrando los datos del modelo y permitiendo la interacción del usuario. (Rhm Faiz, 2023)

Controlador: Actúa como un intermediario entre el Modelo y la Vista, gestionando la entrada del usuario, modificando el modelo según esta entrada y actualizando la Vista en consecuencia. (Rhm Faiz, 2023)

Implementando el MVC en Flutter.

Modelo: En esta parte del patrón MVC, se crea una clase independiente para representar los datos y la lógica empresarial. Por ejemplo, en una aplicación de gestión de tareas simple, podríamos definir una clase Task de la siguiente manera (Figura 8). (Rhm Faiz, 2023)

```
class Task {  
  String title;  
  bool completed;  
  
  Task(this.title, this.completed);  
}
```

Figura 8: Modelo. Fuente: Rhm Faiz 2023. Recuperado de https://medium.com/@Faiz_Rhm/understanding-mvc-architecture-in-flutter-a-comprehensive-guide-with-examples-5d1a372c7eaf

Interfaz de usuario (**Vista**): En Flutter, los widgets representan los elementos de la interfaz de usuario. Es importante crear widgets que se encarguen de mostrar los datos y de capturar la entrada del usuario. Por ejemplo, para nuestra

aplicación de gestión de tareas, podríamos crear un widget TaskListView que muestre una lista de tareas (Figura 9). (Rhm Faiz, 2023)

```
class TaskListView extends StatefulWidget {
  final TaskListController controller;

  const TaskListView({
    super.key,
    required this.controller
  });

  @override
  State<TaskListView> createState() => _TaskListViewState();
}

class _TaskListViewState extends State<TaskListView> {
  @override
  Widget build(BuildContext context) {
    return ListView.builder(
      itemCount: widget.controller.tasks.length,
      itemBuilder: (context, index) {
        final task = widget.controller.tasks[index];
        return ListTile(
          title: Text(task.title),
          leading: Checkbox(
            value: task.completed,
            onChanged: (value) {
              setState(() =>
                widget.controller.toggleTaskCompletion(index)
              );
            },
          ),
        );
      },
    );
  }
}
```

Figura 9: Vista. Fuente: Rhm Faiz 2023. Recuperado de https://medium.com/@Faiz_Rhm/understanding-mvc-architecture-in-flutter-a-comprehensive-guide-with-examples-5d1a372c7eaf

Controlador: Se debe crear una clase independiente encargada de gestionar la entrada del usuario y de actualizar el modelo y la vista en consecuencia. Siguiendo nuestro ejemplo, podríamos crear una clase llamada TaskListController (Figura 10). (Rhm Faiz, 2023)

```
class TaskListController {
  List<Task> tasks = [
    Task('Task 1', false),
    Task('Task 2', true),
    Task('Task 3', false),
  ];

  void toggleTaskCompletion(int index) {
    tasks[index].completed = !tasks[index].completed;
  }
}
```

Figura 10: Controlador. Fuente: Rhm Faiz 2023. Recuperado de https://medium.com/@Faiz_Rhm/understanding-mvc-architecture-in-flutter-a-comprehensive-guide-with-examples-5d1a372c7eaf

Por último, debemos conectar nuestros elementos para crear una aplicación que funcione (Figura 11).

```
void main() {  
  runApp(TaskListApp());  
}  
  
class TaskListApp extends StatelessWidget {  
  final TaskListController controller = TaskListController();  
  
  TaskListApp({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return MaterialApp(  
      home: Scaffold(  
        appBar: AppBar(  
          title: const Text('Task List'),  
        ),  
        body: TaskListView(controller: controller),  
      ),  
    );  
  }  
}
```

Figura 11. Archivo principal. Fuente: Rhm Faiz 2023. Recuperado de https://medium.com/@Faiz_Rhm/understanding-mvc-architecture-in-flutter-a-comprehensive-guide-with-examples-5d1a372c7eaf

Ventajas de usar MVC

- **Separación de responsabilidades:** MVC fomenta una división clara entre datos, interfaz de usuario y lógica, lo que simplifica la comprensión, prueba y mantenimiento del código base. (Rhm Faiz, 2023)
- **Reutilización:** La estructura modular de MVC permite la reutilización de componentes en distintas partes de la aplicación, lo que aumenta la eficiencia del desarrollo. (Rhm Faiz, 2023)

Escalabilidad: Al desacoplar el modelo, la vista y el controlador, se facilita la incorporación de nuevas funciones o la modificación de las existentes sin afectar a los demás componentes, lo que contribuye a la escalabilidad del sistema. (Rhm Faiz, 2023)

Preguntas de Comprensión de la Unidad

1. ¿Qué ventajas ofrece Flutter según su arquitectura y catálogo de widgets?

Flutter ofrece ventajas significativas en el desarrollo de aplicaciones móviles gracias a su arquitectura en capas, que permite una personalización total y un renderizado extremadamente rápido, así como diseños expresivos y flexibles. Además, su catálogo de widgets incluye una amplia variedad de elementos visuales, estructurales, de plataforma e interactivos, lo que facilita el diseño rápido y la creación de interfaces de usuario atractivas y funcionales.

2. ¿Cuáles son algunas características principales de Flutter como framework de desarrollo móvil?

Flutter es un framework gratuito y de código abierto desarrollado por Google en mayo de 2017. Permite el desarrollo de proyectos de programación móvil de forma eficiente y rápida. Al ser de código abierto, fomenta la colaboración y la contribución de la comunidad de desarrolladores.

3. ¿Qué lenguaje de programación utiliza Flutter y qué ventajas ofrece este lenguaje en el desarrollo de interfaces de usuario?

Flutter emplea el lenguaje de programación Dart, desarrollado por Google y de código abierto. Dart está especialmente optimizado para la creación de interfaces de usuario, lo que lo hace ideal para el desarrollo de aplicaciones móviles con Flutter. Muchos de los puntos fuertes de Dart, como su rendimiento y facilidad de uso, se aprovechan plenamente en el desarrollo de aplicaciones con Flutter.

4. ¿Cuál es una recomendación clave para asegurar que una aplicación Flutter tenga una apariencia consistente en una variedad de dispositivos y tamaños de pantalla?

Una recomendación clave es adoptar un enfoque basado en cuadrículas al diseñar la interfaz de usuario. Dividir la pantalla de la aplicación en filas y columnas facilita la alineación de los elementos y garantiza una apariencia consistente en diferentes dispositivos y tamaños de pantalla.

Material complementario

Los siguientes recursos complementarios son sugerencias para que se pueda ampliar la información sobre el tema trabajado, como parte de su proceso de aprendizaje autónomo:

Videos de apoyo:

- <https://www.youtube.com/watch?v=wTibz0TuW7k&list=PLutrh4gv1YI8ap4JO23IN81JOSZJ3i5OO>
- <https://www.youtube.com/watch?v=-pWSQYpkkjk>
- <https://www.youtube.com/watch?v=zNmDOXbTugE&list=PLCKuOXG0bPi0sIn-nDsi7ma9OV6MEMkxj>

Links de apoyo:

- <https://docs.flutter.dev/>
- <https://medium.com/@diegolopezcamacho/comenzando-con-flutter-una-qu%C3%ADa-paso-a-paso-para-novatos-7b8af41175cf>
- <https://esflutter.dev/learn/>

Bibliografía

- » AWS. (s.f). ¿Qué es Flutter?. Recuperado de <https://aws.amazon.com/es/what-is/flutter/>
- » Méndez, M. (2019). Flutter que es y cómo funciona. Recuperado de https://codigofacilito.com/articulos/articulo_23_10_2019_21_14_20?gad_source=1&gclid=CjwKCAjwr7ayBhAPEiwA6ElGxMwaboHVDsd3WWReHsbGKSnDbk2yAeC9dYIhIXOCg3ZkkukIRAE6hoCHAUQAvD_BwE
- » Cristancho, F. (2022). ¿Qué es Flutter?. Recuperado de <https://talently.tech/blog/que-es-flutter/>
- » Jain, P. (s.f). Creación de una interfaz de usuario intuitiva con Flutter: mejores prácticas y consejos. Recuperado de <https://leadgenapp.io/es/crear-una-interfaz-de-usuario-intuitiva-con-consejos-de-mejores-pr%C3%A1cticas-de-flutter/>
- » Téllez, J. (2024). Flutter — Widgets de estilo. Recuperado de <https://medium.com/@jaimetellezb/flutter-widgets-de-estilo-023bb3d7215f>
- » Escacena, J. (2022). Animaciones en Flutter: 4 preguntas para no perderse. Recuperado de <https://www.paradigmadigital.com/techbiz/animaciones-flutter-4-preguntas/>
- » Chiva, J. (s.f). Gestión de Estado en Flutter: Una Guía Fundamental para Principiantes. Recuperado de <https://www.javierchiva.com/blog/gestion-de-estado-en-flutter/#:~:text=Estado%20en%20Flutter-,%C2%BFQu%C3%A9%20es%20la%20Gesti%C3%B3n%20de%20Estado%20en%20Flutter%3F,datos%20dentro%20de%20la%20aplicaci%C3%B3n.>
- » Herrera, D. (2022). Gestión de estados en Flutter ¿Qué es? ¿Cómo puedo aplicarlo?. Recuperado de <https://medium.com/bancolombia-tech/gesti%C3%B3n-de-estados-en-flutter-qu%C3%A9-es-c%C3%B3mo-puedo-aplicarlo-5f2a7b168c62>
- » Alejandro, M. (2018). Flutter: validación de formularios. Recuperado de <https://medium.com/@flutterman/flutter-validaci%C3%B3n-de-formularios-f18ae3e8c60a>
- » Rhm, Faiz. (2023). Comprensión de la arquitectura MVC en Flutter: una guía completa con ejemplos. Recuperado de https://medium.com/@Faiz_Rhm/understanding-mvc-architecture-in-flutter-a-comprehensive-guide-with-examples-5d1a372c7eaf



UNEMI
UNIVERSIDAD ESTATAL DE MILAGRO

Compendio del autor

COMPUTACIÓN MÓVIL

UNIDAD 4
Aplicaciones móviles avanzadas

ÍNDICE

Contenido

Unidad 4: DESARROLLO DE APLICACIONES MÓVILES	3
<i>Tema 1: Frameworks de desarrollo móvil</i>	<i>3</i>
<i>Objetivo:</i>	<i>3</i>
<i>Introducción:</i>	<i>3</i>
Información de los subtemas.....	4
<i>Subtema 1: Introducción a Microservicios</i>	<i>4</i>
<i>Subtema 2: Elementos de un Microservicio</i>	<i>8</i>
<i>Subtema 3: Desarrollo de Servicios en Nodejs.....</i>	<i>10</i>
<i>Subtema 4: Escenarios de ejemplo utilizando los Servicios.....</i>	<i>16</i>
Preguntas de Comprensión de la Unidad	23
Material complementario	25
Bibliografía.....	26

Unidad 4: Aplicaciones móviles avanzadas

Tema 1: Frameworks de desarrollo móvil

Objetivo:

Analizar, comprender y revisar información relacionada a servicios web componentes y su forma de implementación y utilización en el campo de desarrollo móvil.

Introducción:

En la actualidad, las aplicaciones móviles han transformado la forma en que interactuamos con la tecnología y accedemos a la información. Desde redes sociales y entretenimiento hasta comercio electrónico y productividad, las aplicaciones móviles han permeado todos los aspectos de nuestra vida cotidiana. Un componente crucial que potencia estas aplicaciones es su capacidad para consumir servicios web. Los servicios web permiten que las aplicaciones móviles se comuniquen con servidores remotos para obtener datos, realizar transacciones y acceder a una amplia gama de funcionalidades en tiempo real. Esta integración no solo mejora la experiencia del usuario al proporcionar información actualizada y relevante, sino que también permite a los desarrolladores construir aplicaciones más eficientes, escalables y conectadas. A medida que la tecnología sigue avanzando, la interacción entre aplicaciones móviles y servicios web continuará siendo un pilar fundamental en el desarrollo de soluciones innovadoras y dinámicas.

Información de los subtemas

Subtema 1: Introducción a Microservicios

¿Qué son los microservicios?

Los microservicios son tanto un estilo arquitectónico como una forma de desarrollar software. Con los microservicios, las aplicaciones se dividen en componentes más pequeños e independientes. A diferencia del enfoque tradicional y monolítico, en el que todo se compila en una sola unidad, los microservicios son componentes autónomos que trabajan juntos para realizar las mismas tareas. Cada uno de estos componentes es un microservicio. Este enfoque de desarrollo de software valora la granularidad, la simplicidad y la capacidad de reutilizar procesos similares en diversas aplicaciones. (Red Hat, 2023)

En resumen, el objetivo es entregar software de calidad de manera más rápida. Aunque los microservicios pueden facilitar esto, también es crucial considerar otros aspectos. No basta con dividir las aplicaciones en microservicios; es esencial administrarlos, coordinarlos y manejar los datos que generan y modifican. (Red Hat, 2023)

Los microservicios representan un enfoque arquitectónico y organizativo para el desarrollo de software, en el cual este se compone de pequeños servicios independientes que se comunican mediante API bien definidas. Estos servicios son gestionados por equipos pequeños e independientes. (AWS, 2023)

Las arquitecturas de microservicios facilitan la escalabilidad de las aplicaciones y aceleran su desarrollo, permitiendo una mayor innovación y reduciendo el tiempo necesario para llevar nuevas características al mercado. (AWS, 2023)

Arquitectura de microservicios vs arquitectura monolítica

Con las arquitecturas monolíticas, todos los procesos están estrechamente vinculados y se ejecutan como un único servicio. Esto implica que, si un proceso de la aplicación experimenta un aumento en la demanda, es necesario escalar toda la arquitectura. A medida que la base de código crece, añadir o mejorar características en una aplicación monolítica se vuelve más complejo. Esta complejidad limita la capacidad de experimentar y dificulta la implementación de nuevas ideas. Además, las arquitecturas monolíticas aumentan el riesgo para la disponibilidad de la aplicación, ya que la interdependencia de muchos procesos incrementa el impacto de un fallo en uno de ellos. (AWS, 2023)

Con una arquitectura de microservicios, una aplicación se construye a partir de componentes independientes que ejecutan cada proceso como un servicio. Estos servicios se comunican a través de interfaces bien definidas mediante API ligeras. Los servicios se desarrollan en función de las capacidades empresariales y cada uno realiza una sola función. Al ejecutarse de forma independiente, cada servicio puede actualizarse, implementarse y escalarse para satisfacer la demanda de funciones específicas de la aplicación. (AWS, 2023)

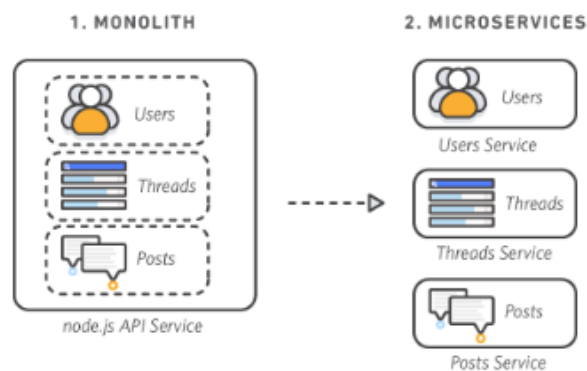


Figura 1: Aplicación monolítica y su referencia en microservicios. Fuente: AWS. 2023. Recuperado de <https://aws.amazon.com/es/microservices/>

Características de los microservicios

Autonomía

Cada componente de servicio en una arquitectura de microservicios puede desarrollarse, implementarse, operarse y escalarse de manera independiente, sin afectar el funcionamiento de otros servicios. Los servicios no necesitan compartir su código o implementaciones con otros servicios. Cualquier comunicación entre componentes individuales se realiza a través de API bien definidas. (AWS, 2023)

Especialización

Cada servicio está diseñado para un conjunto específico de capacidades y se centra en resolver un problema particular. Si con el tiempo los desarrolladores añaden más código a un servicio y este se vuelve complejo, se puede dividir en servicios más pequeños. (AWS, 2023)

Beneficios de microservicios

Agilidad

Los microservicios promueven la organización de equipos pequeños e independientes que son responsables de sus propios servicios. Estos equipos trabajan en un contexto específico y bien entendido, lo que les permite operar de manera más independiente y ágil. Esto reduce los tiempos de ciclo de desarrollo y conduce a un aumento significativo en el rendimiento de la organización. (AWS, 2023)

Escalado flexible

Los microservicios posibilitan que cada servicio se pueda escalar de manera independiente para satisfacer la demanda específica de la característica de la aplicación que respalda. Esto permite a los equipos ajustarse a las necesidades de la infraestructura, calcular con precisión el costo de una característica y mantener la disponibilidad en caso de que un servicio experimente un aumento en la demanda. (AWS, 2023)

Implementación sencilla

Los microservicios posibilitan la integración y entrega continuas, lo que simplifica la prueba de nuevas ideas y permite revertirlas si algo no funciona como se esperaba. El bajo costo de los errores favorece la experimentación, facilita la actualización del código y acelera el tiempo necesario para llevar al mercado nuevas características. (AWS, 2023)

Libertad tecnológica

En las arquitecturas de microservicios, no se adopta un enfoque de "diseño único". Los equipos tienen la libertad de seleccionar la herramienta más adecuada para abordar sus problemas particulares. Por ende, los equipos encargados de desarrollar microservicios pueden optar por la herramienta más idónea para cada tarea. (AWS, 2023)

Código reutilizable

La fragmentación del software en módulos pequeños y claramente definidos permite a los equipos utilizar funciones para diversos propósitos. Un servicio diseñado para una función específica puede ser empleado como componente básico para otra característica. Esto posibilita que una aplicación se inicie de forma independiente, ya que los desarrolladores pueden agregar nuevas capacidades sin necesidad de escribir código desde cero. (AWS, 2023)

Resistencia

La independencia de los servicios incrementa la capacidad de una aplicación para resistir errores. En una arquitectura monolítica, un error en un solo componente puede desencadenar un fallo en toda la aplicación. Sin embargo, con los microservicios, si surge un error en un servicio en particular, las aplicaciones pueden gestionarlo degradando la funcionalidad sin bloquear el funcionamiento completo de la aplicación. (AWS, 2023)

Subtema 2: Elementos de un Microservicio

Un microservicio está conformado por varios componentes que trabajan juntos para proporcionar una funcionalidad específica. Aunque la implementación de un microservicio puede variar según los requisitos y la arquitectura específica de un sistema, generalmente incluye los siguientes elementos:

Lógica de negocio: Esta es la parte central del microservicio que implementa la funcionalidad específica que ofrece el servicio. Incluye el código que realiza las operaciones y procesos necesarios para cumplir con el propósito del microservicio.

API: El microservicio expone una API (Interfaz de Programación de Aplicaciones) bien definida que define cómo otros servicios o aplicaciones pueden interactuar con él. Esta API proporciona una forma estandarizada de enviar solicitudes al microservicio y recibir respuestas.

Persistencia de datos: En muchos casos, un microservicio necesita almacenar y recuperar datos para llevar a cabo su funcionalidad. Puede tener su propia base de datos o acceder a una base de datos compartida con otros microservicios, dependiendo de los requisitos específicos.

Configuración: Los microservicios suelen tener configuraciones propias que controlan su comportamiento, como la configuración de conexión a la base de datos, la configuración de seguridad, etc.

Gestión de errores y registro: Los microservicios suelen incluir mecanismos para manejar errores de forma adecuada y registrar información relevante sobre las operaciones realizadas para fines de seguimiento y diagnóstico.

Capa de transporte: El microservicio necesita una forma de comunicarse con otros componentes del sistema. Esto puede incluir la gestión de protocolos de red, como HTTP, TCP, etc.

Contenedor y entorno de ejecución: Finalmente, el microservicio se ejecuta en un entorno de ejecución, que puede ser un contenedor de Docker, una máquina virtual, etc. Este entorno proporciona los recursos necesarios para ejecutar el microservicio, como memoria, CPU, etc.

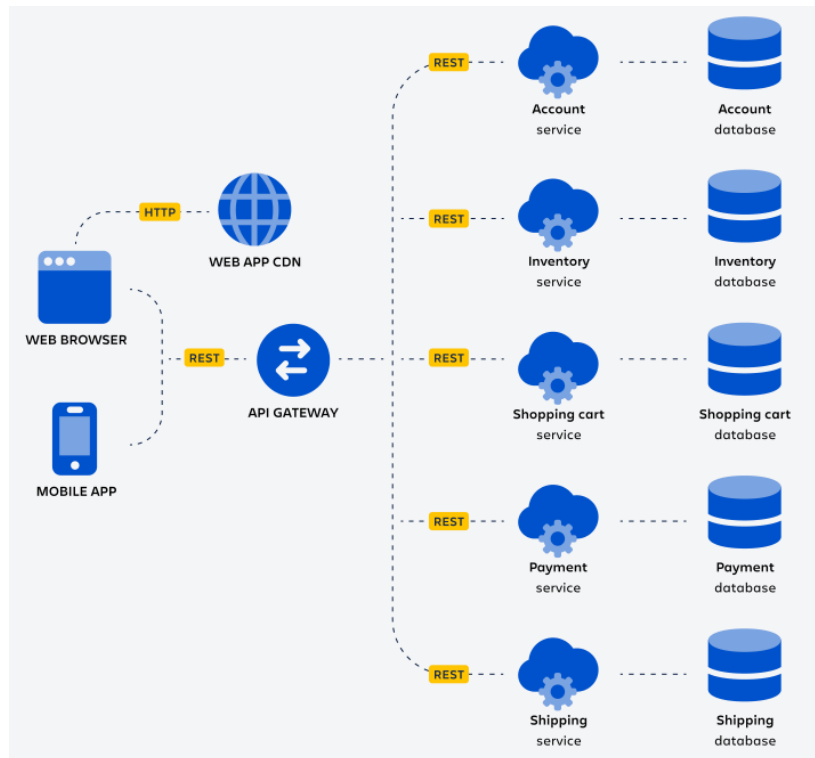


Figura 2: Componentes de una arquitectura de microservicios. Fuente: Atlassian. S.f. Recuperado de <https://www.atlassian.com/es/microservices/microservices-architecture>

Subtema 3: Desarrollo de Servicios en Nodejs

Node.js

Es un entorno de código abierto y multiplataforma que ejecuta JavaScript fuera del navegador. Este entorno de ejecución se enfoca en eventos asíncronos, permitiendo que los eventos se ejecuten independientemente de otros. Facilita la construcción de aplicaciones de red escalables, capaces de gestionar numerosas conexiones simultáneamente sin tener que procesar el código línea por línea ni abrir múltiples procesos. (HENRY, 2021)

Node.js fue creado por los desarrolladores originales de JavaScript con el objetivo de ejecutar este lenguaje fuera del navegador. Para lograrlo, utilizaron el motor V8 de Chrome, que se encarga de convertir el código JavaScript a código máquina en tiempo real, un proceso conocido como JIT (just-in-time). Esto es característico de los lenguajes interpretados como JavaScript, a diferencia de los lenguajes compilados, que deben ser compilados antes de su ejecución. (HENRY, 2021)

Node.js no solo permite crear servidores web, sino que también los hace más ágiles y capaces de interactuar con otros lenguajes de secuencias, como Python. Por esta razón, los desarrolladores lo utilizan principalmente en aplicaciones de red que requieren alta velocidad o en proyectos de gran escala donde es crucial que los procesos sean ágiles. (HENRY, 2021)

Node.js es un entorno de ejecución de código abierto, multiplataforma y de un solo hilo, diseñado para desarrollar aplicaciones de red y del lado del servidor que sean rápidas y escalables. Funciona sobre el motor de ejecución de JavaScript V8 y emplea una arquitectura de E/S no bloqueante basada en eventos, lo que lo convierte en una opción eficiente y adecuada para aplicaciones en tiempo real. (Kinsta, 2021)

El entorno de ejecución utiliza internamente Chrome V8, el motor de ejecución de JavaScript, y también está escrito en C++. Esto amplía los casos de uso de Node.js, permitiendo el acceso a funcionalidades internas del sistema, como la red. (Kinsta, 2021)

Arquitectura de Node.js

Node.js utiliza la arquitectura "Single Threaded Event Loop" para gestionar múltiples clientes simultáneamente. Para comprender cómo se diferencia de otros entornos de ejecución, es necesario entender cómo se manejan los clientes concurrentes en lenguajes como Java. (Kinsta, 2021)

En un modelo de solicitud-respuesta multihilo, varios clientes envían solicitudes y el servidor procesa cada una antes de devolver una respuesta. Sin embargo, se utilizan múltiples hilos para manejar las llamadas concurrentes. Estos hilos se asignan desde un pool de hilos, y cada vez que llega una solicitud, se asigna un hilo individual para gestionarla. (Kinsta, 2021)

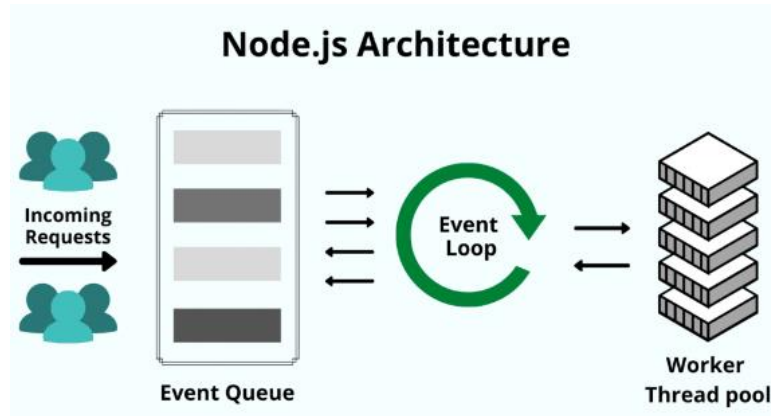


Figura 3: Procesamiento de peticiones entrantes en Node.js utilizando el bucle de eventos. Fuente: Kinsta. 2021. Recuperado de <https://kinsta.com/es/base-de-conocimiento/que-es-node-js/>

Características de Node.js

- Node.js se caracteriza por su accesibilidad, siendo una excelente opción para quienes se aventuran en el desarrollo web. Gracias a su amplia disponibilidad de recursos educativos y una comunidad activa, iniciar proyectos con Node.js resulta sencillo. (Kinsta, 2021)
- La capacidad de escalabilidad de Node.js es notable, ya que su diseño de un solo hilo le permite gestionar múltiples conexiones simultáneas con un rendimiento óptimo. (Kinsta, 2021)
- La eficiencia y rapidez de Node.js se ven potenciadas por su ejecución de hilos sin bloqueo, lo que garantiza una respuesta veloz en el procesamiento de datos. (Kinsta, 2021)
- El ecosistema de Node.js cuenta con una amplia variedad de paquetes de código abierto que facilitan el desarrollo de proyectos, con una vasta cantidad de opciones disponibles en el repositorio NPM. (Kinsta, 2021)
- Node.js cuenta con una base sólida al estar desarrollado en C y C++, lo que le otorga velocidad y funcionalidades adicionales, como la compatibilidad con redes. (Kinsta, 2021)

- Gracias a su soporte multiplataforma, Node.js permite la creación de diversos tipos de aplicaciones, incluyendo sitios web SaaS, aplicaciones de escritorio y móviles, utilizando un único conjunto de herramientas. (Kinsta, 2021)
- La facilidad de mantenimiento es otra ventaja de Node.js, ya que tanto el frontend como el backend pueden ser desarrollados y gestionados utilizando JavaScript como único lenguaje de programación. (Kinsta, 2021)

Aplicaciones de Node.js

Node.js encuentra aplicación en una amplia gama de escenarios. A continuación, exploraremos algunos casos de uso comunes donde Node.js resulta ser una elección acertada. (Kinsta, 2021)

- Chats en tiempo real
- Las aplicaciones de Internet de las Cosas
- Streaming de datos
- Aplicaciones complejas de una sola página (SPA)
- Aplicaciones basadas en REST API

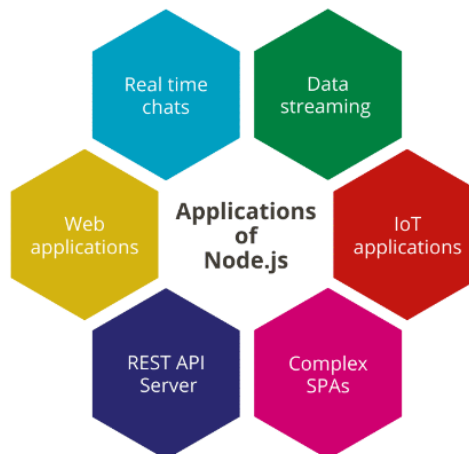


Figura 4: Aplicaciones de Node.js Fuente: Kinsta. 2021. Recuperado de <https://kinsta.com/es/base-de-conocimiento/que-es-node-js/>

Crear un servicio web con Node.js

Para generar un servicio web con Node.js es importante que primero tengamos instalado Node.js en nuestro computador (en el caso de Windows se debe descargar el archivo ejecutable para instalarlo). (Jordán, s.f)

Adicional para la creación de los servicios web podemos utilizar un framework de Node.js que es Express que está basado en JavaScript. Este framework cuenta con algunas características como son:

- Redacción de controladores de solicitudes utilizando diferentes verbos HTTP en diversas rutas URL. (Jordán, s.f)
- Incorporación de motores de renderizado de vistas para producir respuestas al insertar datos en plantillas. (Jordán, s.f)
- Configuración de opciones de aplicaciones web, como el puerto a utilizar para la conexión y la ubicación de las plantillas empleadas para generar respuestas. (Jordán, s.f)
- Adición de procesamiento adicional de middleware en cualquier punto dentro del flujo de manejo de solicitudes. (Jordán, s.f)

Para ocupar Express debemos instalarlo por medio del comando “npm install express” en el directorio de nuestro proyecto. Como lo vemos en la imagen. (Jordán, s.f)

```
D:\WWW\Test\NodeJS>mkdir apirest
D:\WWW\Test\NodeJS>cd apirest
D:\WWW\Test\NodeJS\apirest>npm install express
npm WARN saveError ENOENT: no such file or directory, open 'D:\WWW\Test\NodeJS\apirest\package.json'
npm notice created a lockfile as package-lock.json. You should commit this file.
npm WARN enoent ENOENT: no such file or directory, open 'D:\WWW\Test\NodeJS\apirest\package.json'
npm WARN apirest No description
npm WARN apirest No repository field.
npm WARN apirest No README data
npm WARN apirest No license field.

+ express@4.16.4
added 48 packages from 36 contributors and audited 121 packages in 10.152s
found 0 vulnerabilities
```

Figura 5: Instalación de express. Fuente: Jordán, L. s.f. Recuperado de <https://luisjordan.net/node-js/web-services-con-node-js-express-creacion-de-api-rest/>

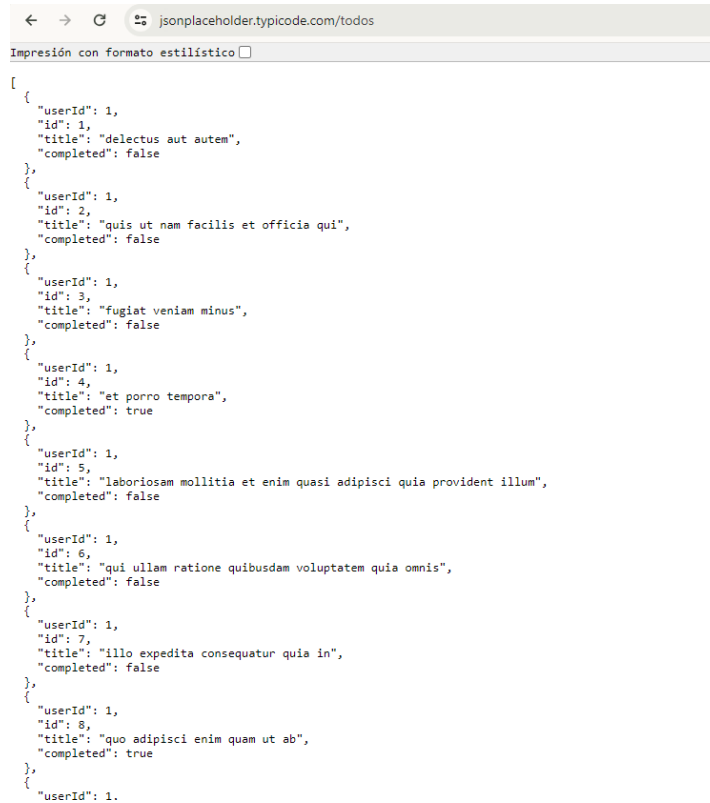
Para este ejemplo también utilizaremos un módulo denominado Request que nos permitirá recuperar y tratar datos obtenidos de una URL (en este caso los datos los obtendremos de una URL ya definida). Lo instalamos por medio del comando “npm install request”. (Jordán, s.f)

```
D:\WWW\Test\NodeJS\apirest>npm install request
npm WARN saveError ENOENT: no such file or directory, open 'D:\WWW\Test\NodeJS\apirest\package.json'
npm WARN enoent ENOENT: no such file or directory, open 'D:\WWW\Test\NodeJS\apirest\package.json'
npm WARN apirest No description
npm WARN apirest No repository field.
npm WARN apirest No README data
npm WARN apirest No license field.

+ request@2.88.0
added 42 packages from 51 contributors and audited 335 packages in 3.754s
found 0 vulnerabilities
```

Figura 6: Instalación de modulo Request. Fuente: Jordán, L. s.f. Recuperado de <https://luisjordan.net/node-js/web-services-con-node-js-express-creacion-de-api-rest/>

Para este ejemplo la URL con la información que vamos a ocupar en el código será “https://jsonplaceholder.typicode.com/todos” que es un conjunto de información en formato JSON. (Jordán, s.f)



```
[
  {
    "userId": 1,
    "id": 1,
    "title": "delectus aut autem",
    "completed": false
  },
  {
    "userId": 1,
    "id": 2,
    "title": "quis ut nam facilis et officia qui",
    "completed": false
  },
  {
    "userId": 1,
    "id": 3,
    "title": "fugiat veniam minus",
    "completed": false
  },
  {
    "userId": 1,
    "id": 4,
    "title": "et porro tempora",
    "completed": true
  },
  {
    "userId": 1,
    "id": 5,
    "title": "laboriosam mollitia et enim quasi adipisci quia provident illum",
    "completed": false
  },
  {
    "userId": 1,
    "id": 6,
    "title": "qui ullam ratione quibusdam voluptatem quia omnis",
    "completed": false
  },
  {
    "userId": 1,
    "id": 7,
    "title": "illo expedita consequatur quia in",
    "completed": false
  },
  {
    "userId": 1,
    "id": 8,
    "title": "quo adipisci enim quam ut ab",
    "completed": true
  },
  {
    "userId": 1
  }
]
```

Figura 7: URL con datos ocupados. Recuperado de <https://jsonplaceholder.typicode.com/todos>

Ahora si creamos nuestro archivo donde colocaremos el código fuente sobre los servicios que vamos a generar, este archivo lo llamaremos `application.js` y lo crearemos en el mismo nivel que el directorio `node_modules` y el script `package-lock.json`, colocaremos el siguiente código. (Jordán, s.f)

```
var express = require("express");
var app = express();
var bodyParser = require('body-parser');
var request = require("request");

// URL con contenido JSON demostrativo.
var url = "https://jsonplaceholder.typicode.com/todos/"

// Soporte para bodies codificados en jsonsupport.
app.use(bodyParser.json());
// Soporte para bodies codificados
app.use(bodyParser.urlencoded({ extended: true }));

// Ejemplo: GET http://localhost:8080/users
// Consumimos datos Fake de la URL: https://jsonplaceholder.typicode.com/todos
app.get('/users', function(req, res) {
  request({
    url: url,
    json: false
  }, function (error, response, body) {

    if (!error && response.statusCode === 200) {
      // Pintamos la respuesta JSON en navegador.
      res.send(body)
    }
  })
});
```

Figura 8: Código fuente del archivo `application.js` Fuente: Jordán, L. s.f. Recuperado de <https://luisjordan.net/node-js/web-services-con-node-js-express-creacion-de-api-rest/>

```
//Ejemplo: GET http://localhost:8080/items/3
app.get('/users/:id', function(req, res) {

    var itemId = req.params.id;

    request({
        url: url+itemId,
        json: false
    }, function (error, response, body) {

        if (!error && response.statusCode === 200) {
            // Pintamos la respuesta JSON en navegador.
            res.send(body)
        }
    })

})

var server = app.listen(8888, function () {
    console.log('Server is running..');
});
```

Figura 9: Segunda parte del código fuente de application.js Fuente: Jordán, L. s.f. Recuperado de <https://luisjordan.net/node-js/web-services-con-node-js-express-creacion-de-api-rest/>

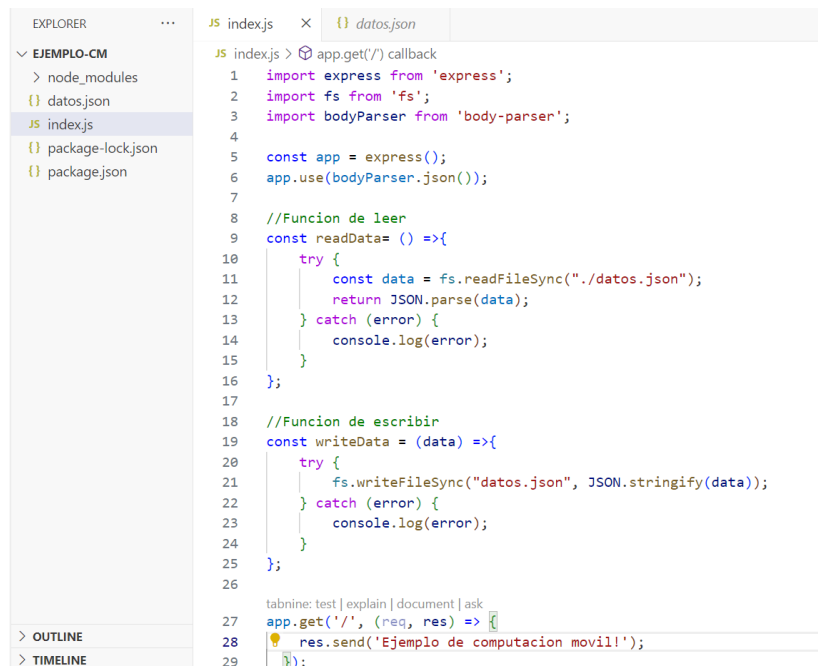
Por último, para ejecutar nuestra aplicación utilizamos el siguiente comando en la consola “node application.js”, en el navegador deberemos ingresar la siguiente URL para ver su funcionamiento “http://localhost:8888/users/”. (Jordán, s.f)

Subtema 4: Escenarios de ejemplo utilizando los Servicios

Generación del Servicio web con Node.js y Express

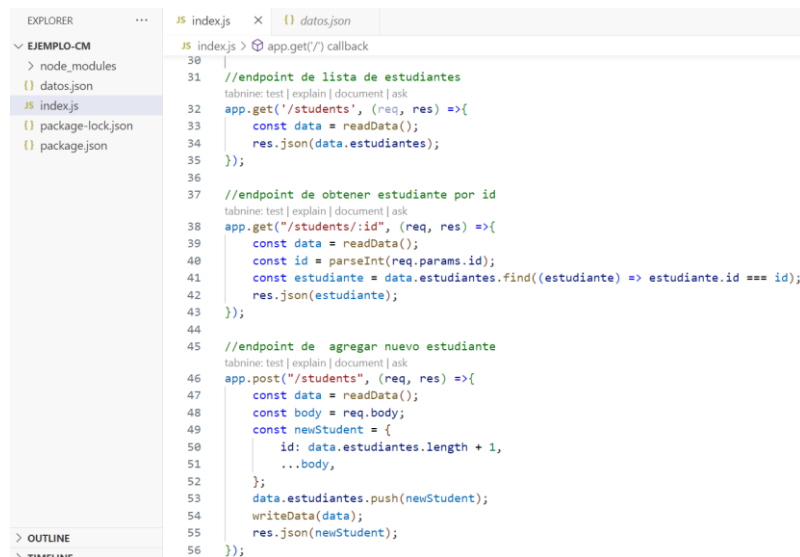
Para la generación del servicio web y su respectiva API ocupamos lo que es el framework de Express de JavaScript que se ejecuta en el entorno de Node.js, para lo cual se genero los endpoints relacionados a un CRUD, en este caso ocupamos un archivo con extensión .json que contiene información de estudiantes, esta información hará referencia a la parte de persistencia de datos.

A continuación, se presenta el código generado en Node.js relacionado a los endpoints mencionados.



```
1  import express from 'express';
2  import fs from 'fs';
3  import bodyParser from 'body-parser';
4
5  const app = express();
6  app.use(bodyParser.json());
7
8  //Funcion de leer
9  const readData= () =>{
10     try {
11         const data = fs.readFileSync("./datos.json");
12         return JSON.parse(data);
13     } catch (error) {
14         console.log(error);
15     }
16 };
17
18 //Funcion de escribir
19 const writeData = (data) =>{
20     try {
21         fs.writeFileSync("datos.json", JSON.stringify(data));
22     } catch (error) {
23         console.log(error);
24     }
25 };
26
27 app.get('/', (req, res) => {
28     res.send('Ejemplo de computacion movill');
29 });
```

Figura 10: Archivo index.js con la definición de los endpoints Parte 1.

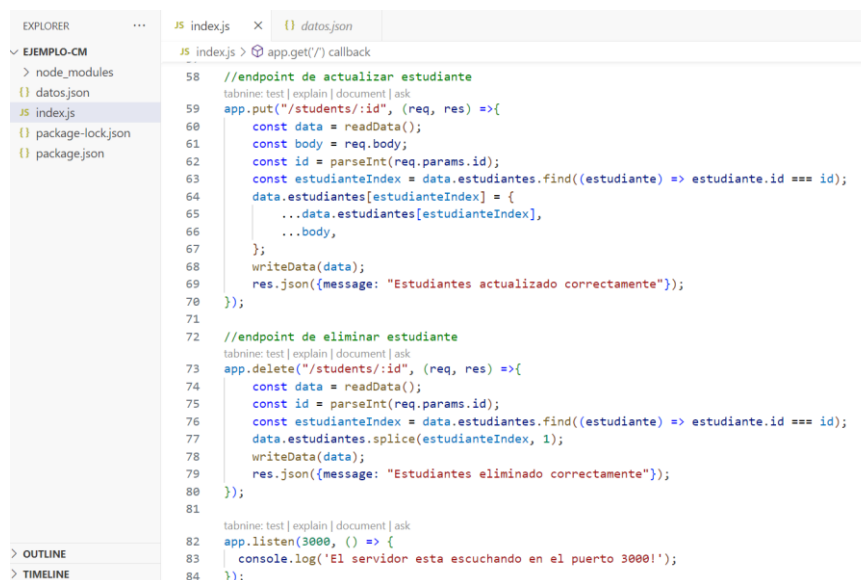


```

EXPLORER
  EJEMPLO-CM
    > node_modules
    {} datos.json
    JS index.js
    {} package-lock.json
    {} package.json

JS index.js
  JS index.js > app.get("/") callback
  30
  31 //endpoint de lista de estudiantes
  32 tabnine: test | explain | document | ask
  33 app.get("/students", (req, res) =>{
  34   const data = readData();
  35   res.json(data.estudiantes);
  36 });
  37
  38 //endpoint de obtener estudiante por id
  39 tabnine: test | explain | document | ask
  40 app.get("/students/:id", (req, res) =>{
  41   const data = readData();
  42   const id = parseInt(req.params.id);
  43   const estudiante = data.estudiantes.find((estudiante) => estudiante.id === id);
  44   res.json(estudiante);
  45 });
  46
  47 //endpoint de agregar nuevo estudiante
  48 tabnine: test | explain | document | ask
  49 app.post("/students", (req, res) =>{
  50   const data = readData();
  51   const body = req.body;
  52   const newStudent = {
  53     id: data.estudiantes.length + 1,
  54     ...body,
  55   };
  56   data.estudiantes.push(newStudent);
  57   writeData(data);
  58   res.json(newStudent);
  59 });
  60
  61
  62
  63
  64
  65
  66
  67
  68
  69
  70
  71
  72
  73
  74
  75
  76
  77
  78
  79
  80
  81
  82
  83
  84
  85
  86
  87
  88
  89
  90
  91
  92
  93
  94
  95
  96
  97
  98
  99
  100
  
```

Figura 11: Archivo index.js con la definición de los endpoints Parte 2.



```

EXPLORER
  EJEMPLO-CM
    > node_modules
    {} datos.json
    JS index.js
    {} package-lock.json
    {} package.json

JS index.js
  JS index.js > app.get("/") callback
  58
  59 //endpoint de actualizar estudiante
  60 tabnine: test | explain | document | ask
  61 app.put("/students/:id", (req, res) =>{
  62   const data = readData();
  63   const body = req.body;
  64   const id = parseInt(req.params.id);
  65   const estudianteIndex = data.estudiantes.find((estudiante) => estudiante.id === id);
  66   data.estudiantes[estudianteIndex] = {
  67     ...data.estudiantes[estudianteIndex],
  68     ...body,
  69   };
  70   writeData(data);
  71   res.json({message: "Estudiantes actualizado correctamente"});
  72 });
  73
  74 //endpoint de eliminar estudiante
  75 tabnine: test | explain | document | ask
  76 app.delete("/students/:id", (req, res) =>{
  77   const data = readData();
  78   const id = parseInt(req.params.id);
  79   const estudianteIndex = data.estudiantes.find((estudiante) => estudiante.id === id);
  80   data.estudiantes.splice(estudianteIndex, 1);
  81   writeData(data);
  82   res.json({message: "Estudiantes eliminado correctamente"});
  83 });
  84
  85
  86
  87
  88
  89
  90
  91
  92
  93
  94
  95
  96
  97
  98
  99
  100
  101
  102
  103
  104
  105
  106
  107
  108
  109
  110
  111
  112
  113
  114
  115
  116
  117
  118
  119
  120
  121
  122
  123
  124
  125
  126
  127
  128
  129
  130
  131
  132
  133
  134
  135
  136
  137
  138
  139
  140
  141
  142
  143
  144
  145
  146
  147
  148
  149
  150
  151
  152
  153
  154
  155
  156
  157
  158
  159
  160
  161
  162
  163
  164
  165
  166
  167
  168
  169
  170
  171
  172
  173
  174
  175
  176
  177
  178
  179
  180
  181
  182
  183
  184
  185
  186
  187
  188
  189
  190
  191
  192
  193
  194
  195
  196
  197
  198
  199
  200
  
```

Figura 12: Archivo index.js con la definición de los endpoints Parte 3.

The screenshot shows a code editor with a file explorer on the left and a code editor on the right. The file explorer shows a project named 'EJEMPLO-CM' with files: 'node_modules', 'datos.json', 'index.js', 'package-lock.json', and 'package.json'. The 'datos.json' file is selected and its content is displayed in the code editor. The JSON content is as follows:

```

1  {
2    "estudiantes": [
3      {
4        "id": 1,
5        "nombre": "Juan",
6        "apellido": "Perez",
7        "edad": 20,
8        "nivel": "Noveno",
9        "cedula": "1804597412"
10     },
11     {
12       "id": 2,
13       "nombre": "Maria",
14       "apellido": "Alvear",
15       "edad": 29,
16       "nivel": "Octavo",
17       "cedula": "0804585102"
18     },
19     {
20       "id": 3,
21       "nombre": "Luisa",
22       "apellido": "Jimenez",
23       "edad": 33,
24       "nivel": "Decimo",
25       "cedula": "0301487451"
26     },
27     {
28       "id": 4,
29       "nombre": "Mauricio",
30       "apellido": "Lopez",
31       "edad": 24,
32       "nivel": "Sexto",
33       "cedula": "1045897410"
34     }
35   ]
36 }
  
```

Figura 13: Archivo de datos .json ocupado como persistencia de datos del servicio

The screenshot shows a web browser window with the address bar displaying 'localhost:3000/students'. Below the address bar, there is a checkbox labeled 'Impresión con formato estilístico' which is checked. The main content of the browser displays the JSON response from the API, which is identical to the one shown in Figure 13, but with Spanish labels for the fields:

```

{
  "identificación": 1,
  "nombre": "Juan",
  "apellido": "Pérez",
  "edad": 20,
  "nivel": "Noveno",
  "cédula": "1804597412"
},
{
  "identificación": 2,
  "nombre": "María",
  "apellido": "Alvear",
  "edad": 29,
  "nivel": "Octavo",
  "cédula": "0804585102"
},
{
  "identificación": 3,
  "nombre": "Luisa",
  "apellido": "Jiménez",
  "edad": 33,
  "nivel": "Décimo",
  "cédula": "0301487451"
},
{
  "identificación": 4,
  "nombre": "Mauricio",
  "apellido": "López",
  "edad": 24,
  "nivel": "Sexto",
  "cédula": "1045897410"
}
  
```

Figura 14: Resultado de llamar a la URL definida en el endpoint de lista de estudiantes

Consumo de servicio en Android Studio

Para el consumo de servicios en Android Studio vamos a emplear la librería Retrofit con la cual podremos obtener la información desde el servicio que creamos en Nopde.js.

Los primero será agregar en el archivo Gradle la librería de Retrofit en este caso en la versión que tenemos de Android Studio lo debemos realizar en dos archivos.



Figura 15: Agregamos la librería Retrofit en el archivo libs.versions.toml

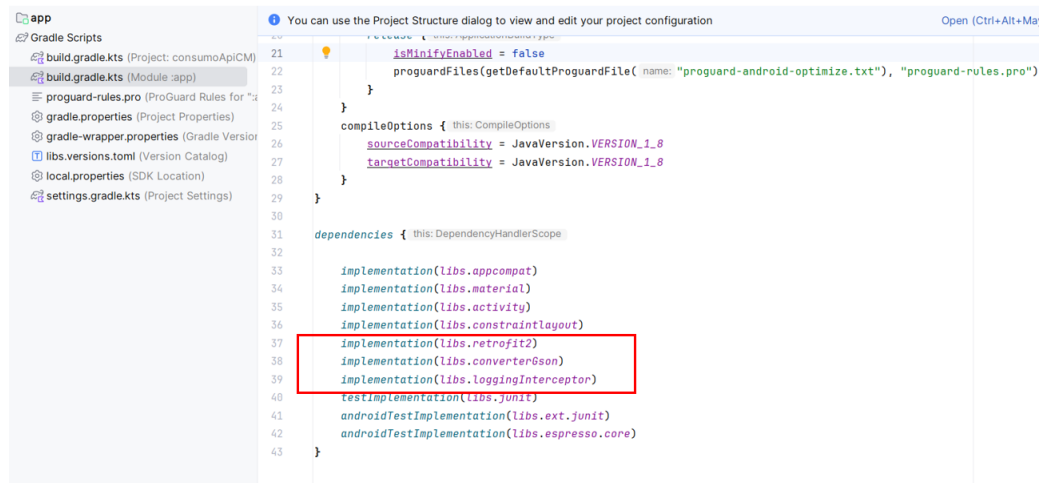
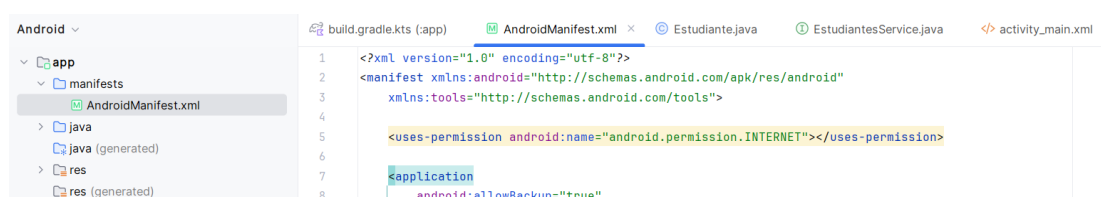


Figura 16: Agregamos la implementación de la librería retrofit en el archivo gradle

Paso seguido agregamos los permisos de internet en el archivo AndroidManifest.xml



Siguiente paso creamos una clase con los atributos que cuenta el archivo de datos json del servicio web, en este caso la clase será estudiante y lo que haremos es agregar lo métodos get y set correspondientes.

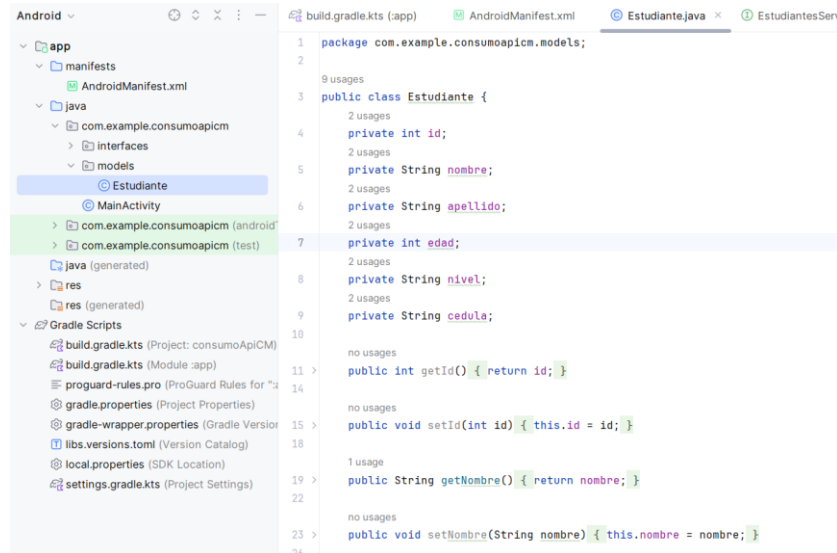


Figura 17: Clase estudiante con sus métodos set y get

Después lo que hacemos es crear una Interface la cual vamos a ocupar para realizar la gestión de cada una de nuestras peticiones a la API que vamos a consumir.

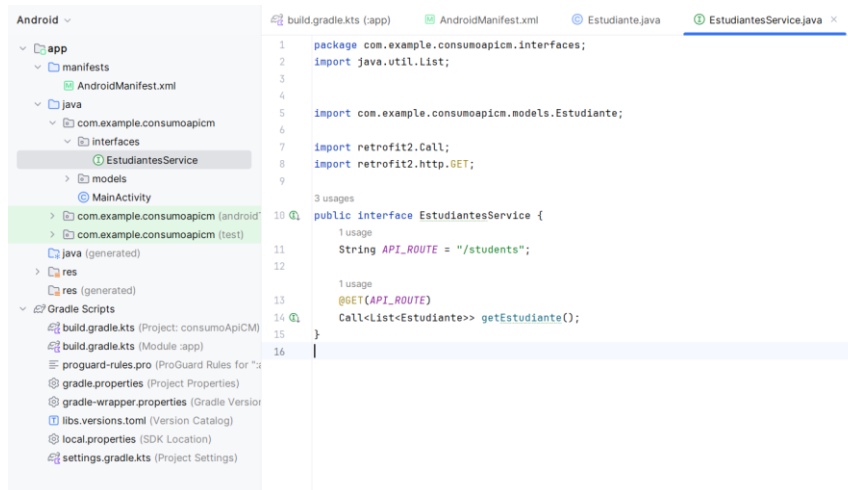


Figura 18: Interface de estudiante para usarlo en la gestión de cada petición a la API

Ahora en el archivo XML de la Activity colocamos el siguiente código, simplemente agregamos un elemento ListView.

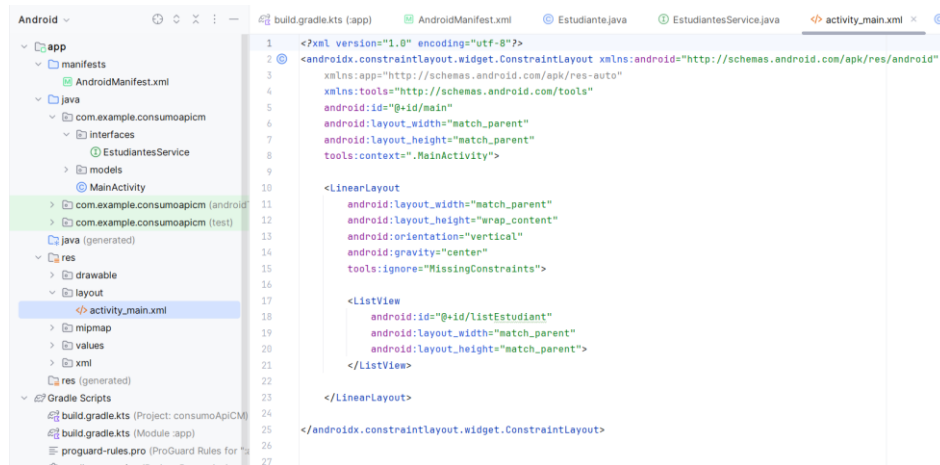


Figura 19: Estructura del código XML

Ahora en el archivo Java de la Activity vamos a enlazar el elemento ListView con el elemento de la interfaz gráfica, adicional definimos un arrayList para el conjunto de datos que vamos a recibir y también implementamos un ArrayAdapter para definir los elementos que vamos a recibir y adaptarlos al ListView.

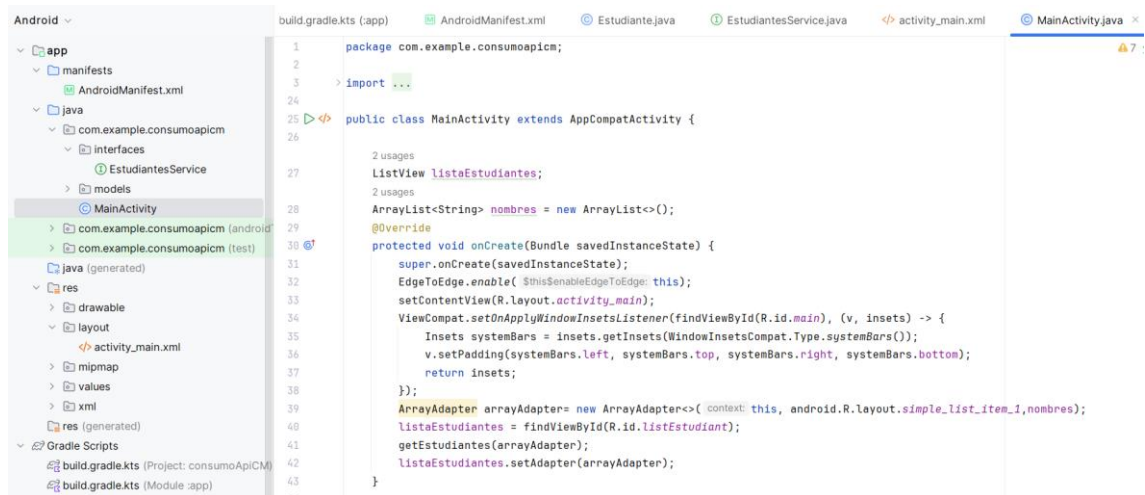


Figura 20: Creación del ListView y del ArrayAdapter

Por último en el mismo archivo Java de la Activity generamos un método en el cual vamos a realizar el consumo de la API mediante la librería Retrofit.

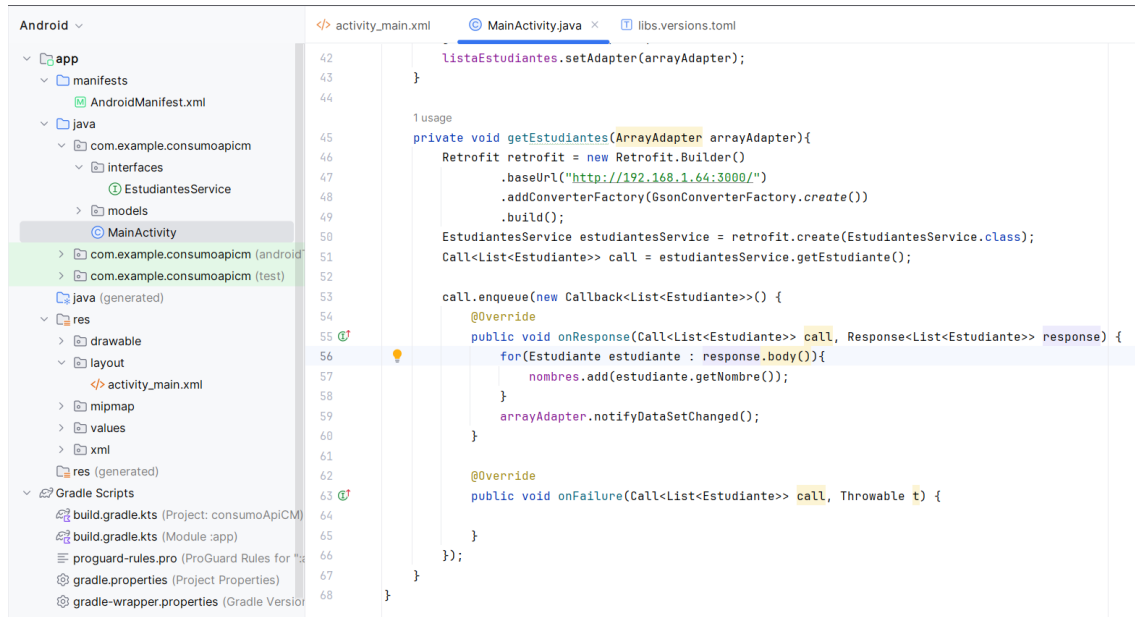


Figura 21 Implementación de la librería Retrofit para el consumo de la API

Por último, con el servicio de Node.js levantado y funcionando ejecutamos la aplicación móvil en donde presentamos los nombres de los estudiantes presentes en el archivo json del servicio.

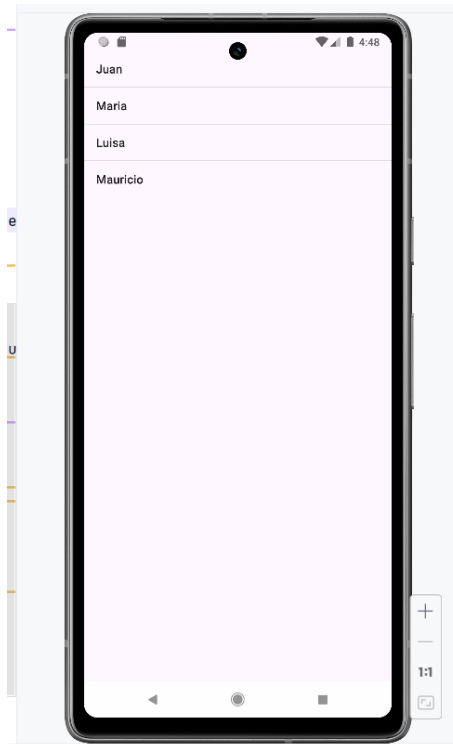


Figura 22: Información obtenida desde la API

Preguntas de Comprensión de la Unidad

- **¿Cuál es la diferencia entre los microservicios y el enfoque monolítico en el desarrollo de software?**

Los microservicios son un estilo arquitectónico y una forma de desarrollar software en el que las aplicaciones se dividen en componentes más pequeños e independientes. A diferencia del enfoque tradicional y monolítico, donde todo se compila en una sola unidad, los microservicios son componentes autónomos que trabajan juntos para realizar las mismas tareas. Este enfoque valora la granularidad, la simplicidad y la capacidad de reutilizar procesos similares en diversas aplicaciones.

- **¿Cómo y por qué fue creado Node.js?**

Node.js fue creado por los desarrolladores originales de JavaScript con el objetivo de ejecutar este lenguaje fuera del navegador. Para lograrlo, utilizaron el motor V8 de Chrome, que se encarga de convertir el código JavaScript a código máquina en tiempo real, un proceso conocido como JIT (just-in-time). Esto es característico de los lenguajes interpretados como JavaScript, a diferencia de los lenguajes compilados, que deben ser compilados antes de su ejecución.

- **¿Cómo se manejan los componentes de servicio en una arquitectura de microservicios?**

Cada componente de servicio en una arquitectura de microservicios puede desarrollarse, implementarse, operarse y escalarse de manera independiente, sin afectar el funcionamiento de otros servicios. Los servicios no necesitan compartir su código o implementaciones con otros servicios. Cualquier comunicación entre componentes individuales se realiza a través de API bien definidas.

- **¿Qué beneficios ofrecen las arquitecturas de microservicios en el desarrollo de aplicaciones?**

Las arquitecturas de microservicios facilitan la escalabilidad de las aplicaciones y aceleran su desarrollo, permitiendo una mayor innovación y reduciendo el tiempo necesario para llevar nuevas características al mercado.

- **¿Cómo se estructuran los microservicios en el desarrollo de software?**

Los microservicios representan un enfoque arquitectónico y organizativo para el desarrollo de software, en el cual este se compone de pequeños servicios independientes que se comunican mediante API bien definidas. Estos servicios son gestionados por equipos pequeños e independientes.

Material complementario

Los siguientes recursos complementarios son sugerencias para que se pueda ampliar la información sobre el tema trabajado, como parte de su proceso de aprendizaje autónomo:

Videos de apoyo:

- <https://www.youtube.com/watch?v=BlmKbdy-ubM>
- <https://www.youtube.com/watch?v=kUzmksdC6ho>
- https://www.youtube.com/watch?v=X5fsU0Oxljg&list=PLlygiKpYTC_6XbaH_E39-cBEfglosadAg

Links de apoyo:

- <https://programacionymas.com/blog/consumir-una-api-usando-retrofit>
- <https://ed.team/blog/como-consumir-una-api-rest-desde-android>
- <https://developer.android.com/develop>

Bibliografía

- » Red Hat. (2023). ¿Qué son y para qué sirven los microservicios?. Recuperado de <https://www.redhat.com/es/topics/microservices>
- » AWS. (2023). ¿Qué son los microservicios?. Recuperado de <https://aws.amazon.com/es/microservices/>
- » Atlassian. (s.f). Arquitectura de microservicios. Recuperado de <https://www.atlassian.com/es/microservices/microservices-architecture>
- » HENRY. (2021). ¿Qué es Node.js y para qué se utiliza?. Recuperado de <https://blog.soyhenry.com/que-es-node-js-y-para-que-se-utiliza/>
- » Kinsta. (2021). Qué es Node.js y por qué debería usarlo. Recuperado de <https://kinsta.com/es/base-de-conocimiento/que-es-node-js/>
- » Jordán, L. (s.f). Web services con Node JS + Express: Creación de Api Rest. Recuperado de <https://luisjordan.net/node-js/web-services-con-node-js-express-creacion-de-api-rest/>
- » Garcia, A. (2018). ¿Cómo consumir una API Rest desde android?. Recuperado de <https://ed.team/blog/como-consumir-una-api-rest-desde-android>



UNEMI
UNIVERSIDAD ESTATAL DE MILAGRO

Compendio del autor

COMPUTACIÓN MÓVIL

UNIDAD 4
Aplicaciones móviles avanzadas

ÍNDICE

Contenido

Unidad 4: DESARROLLO DE APLICACIONES MÓVILES	3
<i>Tema 2: Aplicaciones híbridas (PWA).....</i>	<i>3</i>
<i>Objetivo:</i>	<i>3</i>
<i>Introducción:</i>	<i>3</i>
Información de los subtemas.....	4
<i>Subtema 1: Introducción Aplicaciones Híbridas PWA.....</i>	<i>4</i>
<i>Subtema 2: Desarrollo y Diseño Responsive Cascading Style Sheets</i>	<i>9</i>
<i>Subtema 3: Construcción de Aplicaciones Móviles Híbridas.....</i>	<i>14</i>
<i>Subtema 4: Publicación de APP en Google Play Store</i>	<i>20</i>
Preguntas de Comprensión de la Unidad	23
Material complementario	25
Bibliografía.....	26

Unidad 4: Aplicaciones móviles avanzadas

Tema 2: Aplicaciones híbridas (PWA)

Objetivo:

Revisar, comprender y analizar aspectos, conceptos y características relacionadas al desarrollo de aplicaciones móviles híbridas, así como la publicación de aplicaciones.

Introducción:

Las aplicaciones móviles híbridas combinan lo mejor de dos mundos: la versatilidad y la facilidad de desarrollo de las aplicaciones web con la capacidad de acceder a las funcionalidades nativas de los dispositivos móviles. Estas aplicaciones están escritas con tecnologías web estándar como HTML, CSS y JavaScript, pero se ejecutan dentro de un contenedor nativo que permite acceder a características del dispositivo, como la cámara, el GPS y las notificaciones push. Este enfoque permite a los desarrolladores crear una sola aplicación que funcione en múltiples plataformas, como iOS y Android, lo que reduce los costos y el tiempo de desarrollo.

Información de los subtemas

Subtema 1: Introducción Aplicaciones Híbridas PWA

Aplicación Híbrida

Las aplicaciones híbridas son un tipo de software móvil que integra elementos de las aplicaciones nativas y web. Estas aplicaciones se desarrollan utilizando tecnologías web como HTML5, CSS3 y JavaScript para su estructura. (IMMUNE, 2023)

Básicamente, una aplicación híbrida se ejecuta dentro de un navegador web, pero se presenta como una aplicación nativa en los dispositivos móviles. Este enfoque permite a los desarrolladores disminuir costos y tiempos de desarrollo al reutilizar el mismo código para múltiples plataformas. (IMMUNE, 2023)

De esta manera, las aplicaciones híbridas están concebidas para ofrecer una experiencia uniforme en todos los dispositivos móviles, sin importar el sistema operativo o la marca del dispositivo. Así, las aplicaciones híbridas permiten a empresas y organizaciones lanzar sus productos en diversas plataformas de forma rápida y eficiente, sin comprometer la calidad del producto. (IMMUNE, 2023)

Una aplicación móvil híbrida integra elementos de una aplicación nativa (desarrollada para una plataforma específica como Android o iOS) y una aplicación web (accesible a través de un navegador en internet). Se construye utilizando lenguajes y tecnologías populares de desarrollo front-end como HTML5, JavaScript y CSS, lo que facilita la funcionalidad multiplataforma. (Appsflyer, s.f)

Esto significa que los desarrolladores no necesitan crear código separado para Android e iOS. Pueden escribir el código para una aplicación móvil una vez y adaptarlo para múltiples plataformas simultáneamente. Cuando los usuarios descargan una aplicación híbrida desde una tienda de aplicaciones e instalan localmente, su envoltura nativa se conecta a las capacidades de la plataforma móvil a través de un navegador integrado en la aplicación. (Appsflyer, s.f)

Ventajas

Mayor alcance

Las empresas con presupuestos limitados suelen lanzar aplicaciones en una sola plataforma inicialmente, lo que restringe su alcance, ya que los usuarios de otras plataformas deben esperar para poder descargarla. Sin embargo, una vez que una aplicación híbrida está lista, puede estar disponible en las tiendas de aplicaciones tanto de Android como de iOS, permitiendo que cualquier usuario interesado la utilice y ampliando así la audiencia. (Appsflyer, s.f)

Más fácil de escalar

El lanzamiento diferido y desincronizado de funciones en distintas plataformas puede generar frustraciones entre los usuarios. Las aplicaciones híbridas eliminan este problema al ser fácilmente escalables, permitiendo la incorporación y el despliegue de nuevas funcionalidades en todas las plataformas de manera simultánea. (Appsflyer, s.f)

Costos más bajos

Las aplicaciones híbridas requieren solo una base de código, lo que permite a los desarrolladores crearlas más rápidamente en comparación con desarrollar dos aplicaciones nativas por separado. La reducción en las horas de desarrollo facturables también hace que el desarrollo de aplicaciones híbridas sea más rentable que el de aplicaciones nativas. (Appsflyer, s.f)

Bajo mantenimiento

Las aplicaciones híbridas permiten publicar parches y corregir errores en todas las plataformas y dispositivos simultáneamente. Con una aplicación nativa, tendrías que solucionar problemas primero en la plataforma iOS y luego nuevamente en Android, duplicando el esfuerzo. (Appsflyer, s.f)

Acceso a todas las funciones del dispositivo

Las aplicaciones híbridas, al igual que las aplicaciones nativas, pueden acceder a las funciones del dispositivo. Esto mejora significativamente el rendimiento de la aplicación y la experiencia del usuario en comparación con las aplicaciones web. (Appsflyer, s.f)

Desventajas

Rendimiento lento

Aunque los lenguajes de programación para aplicaciones híbridas han mejorado considerablemente, siguen siendo generalmente más lentos que las aplicaciones nativas, que se desarrollan utilizando los lenguajes de programación específicos de Apple o Google. Una aplicación híbrida se ejecuta en un componente similar a un navegador (llamado vista web) y su rendimiento depende en gran medida de la calidad de esta vista web para mostrar la interfaz de usuario y ejecutar el código JavaScript. (Appsflyer, s.f)

Complejidad de la prueba

Aunque las aplicaciones híbridas comparten una parte significativa de su código entre plataformas, es posible que ciertas partes del código sean nativas. Esto puede incrementar la complejidad de tus pruebas, especialmente dependiendo de la naturaleza específica de tu aplicación. (Appsflyer, s.f)

Inconsistencias en la UI/UX

La coherencia en la experiencia del usuario (UX) y en la interfaz de usuario (UI) de una aplicación híbrida puede variar significativamente y suele depender en gran medida de las decisiones de los desarrolladores. La mayor flexibilidad ofrecida por el desarrollo de aplicaciones híbridas puede llevar a errores si los desarrolladores no tienen un buen entendimiento del proceso. Además, problemas como una conexión a internet deficiente pueden resultar en una experiencia de usuario inconsistente si los desarrolladores no implementan correctamente el diseño web progresivo. Es importante tener en cuenta que, para cumplir con las directrices de interacción en Android e iOS o para acceder a APIs específicas de la plataforma, los desarrolladores aún deben escribir código nativo en ciertos casos. (Appsflyer, s.f)

Factores a considerar para elegir un tipo de aplicación

Público objetivo

Cuando se trata de consumidores, la elección entre aplicaciones nativas o híbridas es más adecuada. Sin embargo, si la aplicación está destinada a un uso interno dentro de una organización específica, las aplicaciones web serían la opción más apropiada. (Appsflyer, s.f)

Plataforma

Las aplicaciones nativas están diseñadas para un sistema operativo específico, como Android o iOS, mientras que las aplicaciones web son accesibles a través de un navegador en cualquier dispositivo. Si se busca flexibilidad, las aplicaciones híbridas y multiplataforma pueden ser más adecuadas. (Appsflyer, s.f)

Función principal

Las diversas categorías de aplicaciones ofrecen distintas capacidades. Por ejemplo, las aplicaciones nativas pueden acceder directamente a las funciones específicas del dispositivo, como la cámara y el micrófono, mientras que las aplicaciones web no tienen esta capacidad. (Appsflyer, s.f)

Recursos de desarrollo

Las aplicaciones nativas suelen necesitar habilidades especializadas y un equipo de desarrollo más grande en comparación con las aplicaciones web. Las aplicaciones híbridas y multiplataforma también requieren un nivel intermedio de recursos de desarrollo. (Appsflyer, s.f)

Mantenimiento

Las aplicaciones nativas requieren un mantenimiento más continuo ya que deben actualizarse para cada plataforma. En contraste, las aplicaciones web pueden actualizarse en un solo lugar y los cambios se reflejarán en todos los dispositivos. Las aplicaciones híbridas y multiplataforma se encuentran en un punto intermedio, necesitando niveles moderados de mantenimiento. (Appsflyer, s.f)

Costo

El desarrollo de aplicaciones nativas tiende a ser más costoso que el desarrollo de aplicaciones web debido a la necesidad de habilidades especializadas y la creación para múltiples plataformas. Aunque las aplicaciones híbridas y multiplataforma ofrecen flexibilidad, sus costos de desarrollo suelen ubicarse en un punto intermedio. (Appsflyer, s.f)

Cuando utilizar una aplicación híbrida

- Si buscas compatibilidad con múltiples plataformas, desarrollar una aplicación híbrida es una opción obvia. Estas aplicaciones son independientes de la plataforma y pueden funcionar tanto en iOS como en Android, gracias a una única base de código. (Appsflyer, s.f)
- Si no cuentas con conocimientos especializados en codificación, una aplicación híbrida puede ser una opción más accesible. Requiere menos dominio de lenguajes de programación complejos y facilita el proceso de contratación de desarrolladores. (Appsflyer, s.f)
- Las aplicaciones híbridas pueden ser suficientes si no necesitas características avanzadas específicas de las aplicaciones nativas. Además, simplifican la integración de funciones básicas que requieren iteraciones constantes, como pruebas divididas, ajustes y notificaciones. (Appsflyer, s.f)
- Las aplicaciones híbridas eliminan la necesidad de crear APIs adicionales, lo que puede resultar en un ahorro significativo de costos. Desarrollar una API para respaldar una aplicación nativa puede ser costoso, y las aplicaciones híbridas ofrecen una solución rentable al eliminar esta necesidad. (Appsflyer, s.f)

Subtema 2: Desarrollo y Diseño Responsive Cascading Style Sheets

Diseño Responsive

Actualmente, el uso de una amplia gama de dispositivos móviles ha aumentado significativamente, incluyendo no solo teléfonos inteligentes, sino también tabletas, relojes inteligentes, lectores de ebooks y diversos tipos de dispositivos con capacidad de conexión a Internet. (MANZ.DEV, s.f)

Es cada vez más común acceder a Internet desde distintos dispositivos, cada uno con diferentes pantallas y resoluciones, tamaños y formas, lo que provoca que las páginas web se consuman de diversas maneras, generando nuevas necesidades, problemas y soluciones en el proceso. (MANZ.DEV, s.f)

Por lo tanto, hoy en día, al diseñar un sitio web, es esencial que se visualice correctamente en diversas resoluciones, lo cual no es una tarea sencilla. Anteriormente, se solía crear una versión diferente del sitio web según el dispositivo o navegador del usuario, pero esto se descartó por ser impráctico. (MANZ.DEV, s.f)

Afortunadamente, esos tiempos han quedado atrás y la norma actual es diseñar un único sitio web que se adapte visualmente al dispositivo utilizado. (MANZ.DEV, s.f)

Actualmente, esto se conoce como Diseño Web Responsivo (o RWD), un enfoque en el que los diseños web se ajustan al tamaño y formato de la pantalla donde se visualiza el contenido, a diferencia de los diseños tradicionales que estaban hechos para un tamaño o formato específico y carecían de esta capacidad de adaptación. (MANZ.DEV, s.f)

El diseño responsivo es una estrategia de diseño web que ajusta el contenido para adaptarse a diversos tamaños de pantalla y ventanas en una variedad de dispositivos. (Duò, 2022)

El diseño web responsivo se enfoca en el entorno del usuario dentro de un sitio web. Este entorno varía según el dispositivo que el usuario tenga conectado a internet. (Boneu, 2020)

Antes de que el diseño web responsivo ganara popularidad, muchas empresas gestionaban un sitio web completamente separado, que recibía tráfico redirigido según el agente de usuario (user-agent). Sin embargo, en el diseño web responsivo, el servidor siempre envía el mismo código HTML a todos los dispositivos, y se utiliza CSS para modificar la representación de la página en cada dispositivo. (Boneu, 2020)

Responsive Web Design y Mobile First Web

Aunque están relacionados, es importante distinguir entre Responsive Web Design (Diseño Web Adaptativo) y Mobile First Web. El Diseño Web Adaptativo se refiere a la capacidad de un sitio web para adaptarse a diferentes tamaños y dispositivos, haciendo que el sitio sea más accesible y fácil de usar sin importar el dispositivo. Por otro lado, el concepto de Mobile First implica comenzar el diseño de un sitio web desde la resolución más pequeña y luego expandir y ajustar el contenido y el diseño a resoluciones mayores. (Guerras, s.f)

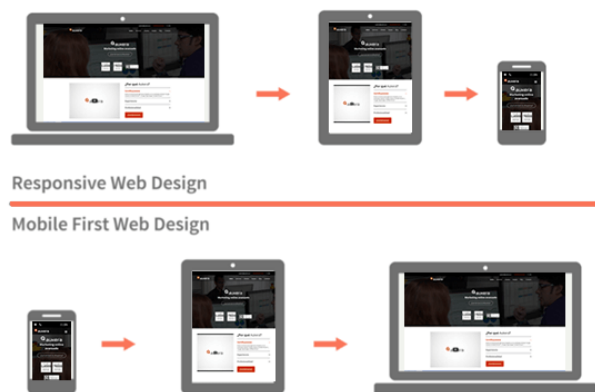


Figura 1: Diseño responsive, Mobile First. Fuente: Guerras, s.f. Recuperado de <https://aukera.es/blog/diseño-responsivo/>

Frameworks CSS para diseño web responsivo

Si crees que escribir todo el CSS desde cero para cada proyecto no es lo más eficiente, es hora de considerar el uso de un framework CSS. (Prieto, 2021)

Ventajas

- **Ahorrar tiempo:** Reescribir el mismo código continuamente es repetitivo e improductivo, como mencionamos en nuestras buenas prácticas para escribir CSS. Con un framework, ya tienes una base sobre la cual empezar a diseñar. (Prieto, 2021)
- **Compatibilidad con navegadores:** La mayoría de los frameworks son utilizados por muchos usuarios, quienes reportan los errores que

encuentran en diferentes navegadores. Esto permite que los frameworks mejoren su compatibilidad con el tiempo, eliminando la preocupación de si tu diseño se ve bien en Safari, Chrome, Firefox, entre otros. (Prieto, 2021)

- **Mobile first:** La mayoría de los frameworks están diseñados para ser responsivos y se basan en la metodología Mobile First. Esto asegura una visualización óptima en dispositivos con menor resolución sin sobrecargar con estilos y propiedades innecesarias. (Prieto, 2021)
- **Escalabilidad:** Utilizar un framework CSS permite que tu diseño tenga la capacidad de crecer en el futuro. Si añades una nueva sección, página o elemento, no necesitarás escribir nuevas líneas de CSS, sino reutilizar los estilos ya predefinidos y centrarte en el HTML y en los detalles específicos de esa página. (Prieto, 2021)

Desventajas

- **Puede limitar la creatividad de tu diseño:** Empezar con una estructura de rejilla o con márgenes y formas predefinidos puede hacer que tu diseño se vea condicionado por estas limitaciones, reduciendo el nivel de creatividad en tu proyecto en comparación con empezar desde cero. (Prieto, 2021)
- **Líneas de código innecesarias:** Es probable que no utilices la mitad de las especificaciones y clases escritas en el CSS del framework. Sin embargo, los usuarios descargan los archivos completos, lo que aumenta el peso de la web y ralentiza su carga. (Prieto, 2021)
- **Tiempo de aprendizaje:** Aunque la curva de aprendizaje suele ser rápida, familiarizarse con el framework requiere tiempo, y no será hasta después de algunos proyectos que conocerás la mayor parte del código. (Prieto, 2021)

Algunos frameworks CSS que se pueden mencionar son los siguientes:

- Bootstrap
- Foundation
- Pure CSS
- Milligram

Bootstrap

Es probablemente el más utilizado y solicitado en las ofertas de empleo para diseñadores y maquetadores web. Entre sus ventajas se encuentra una base muy sólida de usuarios y profesionales que lo usan actualmente, así como la incorporación de reglas para prácticamente todos los estilos que pueden diseñarse en una web. (Prieto, 2021)

Como inconveniente, si no se usa con cuidado y no se filtra adecuadamente qué módulos cargar, se puede obligar al usuario a descargar muchos datos innecesarios. Por ello, antes de ponerlo en producción, es recomendable conocer bien cómo funciona este framework y qué elementos se van a utilizar. (Prieto, 2021)

Foundation

Un framework que ha estado presente en la web desde 2011 y ha ido mejorando y añadiendo nuevos elementos en cada versión. De hecho, en su versión actual, al igual que Bootstrap, ya es posible utilizar Flexbox en su grid. Es una elección confiable. (Prieto, 2021)

Además de su uso en sitios web, Foundation ofrece otras dos distribuciones de su framework: una para trabajar con correos electrónicos y otra para trabajar con aplicaciones, lo que amplía las oportunidades de aprendizaje y abre otras posibilidades de diseño. (Prieto, 2021)

Pure CSS

Para aquellos que buscan un framework liviano que permita una carga rápida del sitio web, Pure CSS es la elección ideal. Es perfecto para iniciar un proyecto de manera rápida, sin complicaciones y con cierto grado de confiabilidad. Comprimido, ocupa menos de 4 KB, lo cual es realmente impresionante. (Prieto, 2021)

Sin embargo, es cierto que los elementos pueden no ser los más estéticos. Personalmente, no estoy convencido del todo por los estilos de los botones y los formularios, pero como se dice, para gustos están los colores. Si buscas algo rápido y sencillo, es una opción a considerar. (Prieto, 2021)

Milligram

Con solo 2 KB de tamaño cuando se sirve comprimido, ofrece estilos muy básicos pero con un diseño cuidado y muy plano. Además de esto, aborda uno de los problemas más criticados de los grandes frameworks al permitir cambiar entre una cuadrícula de 12 columnas a 10 columnas, e incluso combinar ambas. Además, todas las medidas están en rem. (Prieto, 2021)

Sin embargo, como punto en contra, su compatibilidad solo garantiza la última versión de los navegadores más utilizados, lo que significa que es posible encontrar comportamientos inesperados en navegadores más antiguos. Pero, para mantener ese tamaño, esa falta de compatibilidad está más que justificada. Es realmente fantástico. (Prieto, 2021)

Subtema 3: Construcción de Aplicaciones Móviles Híbridas

Las aplicaciones móviles híbridas son aquellas diseñadas para funcionar en diversos dispositivos y sistemas operativos. Esto permite su uso en diferentes móviles, tablets y ordenadores, sin importar la marca o el modelo del dispositivo. (Appandweb, 2021)

Estas aplicaciones ofrecen varias ventajas, como un rendimiento consistente en todas las plataformas y un costo de desarrollo menor en comparación con las aplicaciones nativas. (Appandweb, 2021)

Ahora que recapitulamos lo que es una aplicación móvil híbrida es momento de hablar sobre algunos frameworks que podemos ocupar para construirlas. (Appandweb, 2021)

ionic

Ionic es una herramienta poderosa que facilita la creación de aplicaciones nativas para iOS y Android, así como aplicaciones web progresivas para dispositivos móviles. Utiliza bibliotecas web, marcos y lenguajes familiares para brindar una experiencia de desarrollo familiar y efectiva. (Cristancho, 2022)

Este framework de código abierto ofrece una amplia gama de componentes, gestos y herramientas de interfaz de usuario optimizados específicamente para dispositivos móviles. Esto permite el desarrollo de aplicaciones rápidas y altamente interactivas que se ejecutan sin problemas en una variedad de dispositivos móviles. (Cristancho, 2022)

Ionic está diseñado para ofrecer un rendimiento óptimo en los dispositivos móviles más recientes, garantizando aplicaciones ultrarrápidas con un tamaño reducido. Además, incorpora mejores prácticas, como transiciones aceleradas por hardware, gestos táctiles optimizados y técnicas de procesamiento previo y compilación AOT para una experiencia de usuario fluida y eficiente. (Cristancho, 2022)

Ionic proporciona una amplia gama de componentes, como cabeceras, pies de página, botones, enlaces, listados, formularios, rejillas y pestañas. Cada componente está compuesto por elementos personalizados que ofrecen métodos, eventos y propiedades CSS personalizadas. Estos componentes están escritos en HTML, CSS y JavaScript, lo que simplifica la creación de interfaces

modernas y de alta calidad que funcionan de manera óptima en todos los dispositivos. (Cristancho, 2022)

Ventajas:

- Ionic es una plataforma multiplataforma que facilita el desarrollo de aplicaciones híbridas para una variedad de sistemas operativos, incluyendo Android, iOS, Windows, WebApps y Amazon FireOS. (Cristancho, 2022)
- Utiliza tecnologías como HTML5, AngularJS y TypeScript para ofrecer una experiencia de desarrollo sencilla y flexible. (Cristancho, 2022)
- Ionic ofrece una amplia gama de componentes de interfaz de usuario predefinidos que se ajustan a las interfaces nativas de diferentes sistemas, lo que permite a los desarrolladores trabajar de manera productiva y eficiente. (Cristancho, 2022)
- Ionic es un proyecto de código abierto, lo que significa que es gratuito y disponible para su uso, lo que ayuda a ahorrar tiempo y recursos. (Cristancho, 2022)

Desventajas:

- Bajo rendimiento: Aunque las aplicaciones híbridas pueden ofrecer un rendimiento ligeramente inferior en comparación con las aplicaciones nativas, la potencia cada vez mayor de los smartphones ha minimizado esta brecha. Con una optimización adecuada durante el desarrollo, es difícil distinguir entre una aplicación nativa y una híbrida en términos de rendimiento. (Cristancho, 2022)
- Limitaciones en aplicaciones de alta carga gráfica: Ionic puede no ser la opción ideal para desarrollar aplicaciones que requieran una gran potencia gráfica, como juegos. Debido a sus limitaciones en el procesamiento gráfico, las aplicaciones híbridas pueden no ofrecer el rendimiento necesario para este tipo de aplicaciones. (Cristancho, 2022)
- Posibles errores de compilación: El desarrollo de aplicaciones móviles está sujeto a problemas de compatibilidad entre diferentes versiones de sistemas operativos, bibliotecas y paquetes. Con Ionic, es posible que surjan errores de compilación que puedan afectar el funcionamiento del proyecto, lo que requiere tiempo para identificar y solucionar. (Cristancho, 2022)

React Native

React Native es un framework de JavaScript diseñado para la creación de aplicaciones nativas reales tanto en iOS como en Android. Se basa en la biblioteca JavaScript React, que se utiliza para crear componentes visuales. Sin embargo, en lugar de ejecutar estos componentes en un navegador, React Native permite que se ejecuten directamente en las plataformas móviles nativas, lo que significa que obtienes una aplicación nativa real al final del proceso. Esto hace que las aplicaciones desarrolladas con React Native sean prácticamente indistinguibles de aquellas creadas utilizando lenguajes como Objective-C o Java. (Blanes, s.f)

Características:

- **Compatibilidad Cross-Platform:** La mayoría de las APIs en React Native son compatibles con ambos sistemas, lo que permite a los desarrolladores crear aplicaciones que pueden ejecutarse simultáneamente en iOS y Android utilizando el mismo código base. (Blanes, s.f)
- **Funcionalidad nativa:** Las aplicaciones desarrolladas con React Native se comportan como aplicaciones nativas reales creadas para cada sistema utilizando sus lenguajes nativos respectivos. La combinación de React Native con JavaScript permite la creación de aplicaciones complejas de manera fluida, mejorando incluso el rendimiento de las aplicaciones nativas y sin necesidad de utilizar un WebView. (Blanes, s.f)
- **Actualizaciones instantáneas (para desarrollo y pruebas):** Con el uso de JavaScript, los desarrolladores tienen la capacidad de cargar cambios directamente en los dispositivos de los usuarios sin pasar por los procesos tediosos de las tiendas de aplicaciones. Es importante tener en cuenta que esta práctica está limitada a las versiones de desarrollo o de prueba; realizar cambios directos en el código de aplicaciones ya publicadas y en producción puede resultar ilegal y conllevar sanciones, incluida la retirada de la aplicación de las tiendas. (Blanes, s.f)
- **Curva de aprendizaje sencilla:** React Native es fácil de entender y aprender, ya que se basa en los conceptos fundamentales de JavaScript. Es intuitivo tanto para expertos en JavaScript como para aquellos sin experiencia en el lenguaje, ya que proporciona una amplia gama de componentes, incluidos los mapas y filtros que son comunes en JavaScript. (Blanes, s.f)

Flutter

Flutter es un framework de código abierto respaldado y desarrollado por Google. Es utilizado por desarrolladores de front-end y de pila completa para construir interfaces de usuario de aplicaciones que funcionan en varias plataformas utilizando un único código base. (AWS, s.f)

Este marco de desarrollo aprovecha el lenguaje de programación de código abierto Dart, también creado por Google. Dart está especialmente optimizado para la creación de interfaces de usuario, y muchas de sus características se utilizan de manera efectiva en Flutter. (AWS, s.f)

Ventajas:

- Flutter ofrece un rendimiento casi nativo al utilizar el lenguaje de programación Dart y compilarlo directamente en código máquina. Esto garantiza una ejecución rápida y eficiente en los dispositivos donde se ejecuta. (AWS, s.f)
- Además, Flutter proporciona un rendimiento rápido, constante y personalizable al utilizar la biblioteca gráfica de código abierto Skia de Google para renderizar la interfaz de usuario. Esto asegura una experiencia visual uniforme en todas las plataformas, independientemente del dispositivo utilizado para acceder a la aplicación. (AWS, s.f)
- Google se ha centrado en hacer que Flutter sea fácil de usar para los desarrolladores. Introduce herramientas como la recarga en caliente, que permite previsualizar los cambios de código en tiempo real sin perder el estado de la aplicación. Otras herramientas, como el inspector de widgets, facilitan la visualización y la solución de problemas relacionados con el diseño de la interfaz de usuario. (AWS, s.f)

PWA

Las Progressive Web Apps (PWA) no son un concepto nuevo, ya que Google lo introdujo en 2015, pero aún es desconocido para muchas personas. Estas aplicaciones son una combinación de lo mejor de las aplicaciones web y las nativas, a menudo consideradas como un punto medio o una evolución de ambas. (López, 2020)

Aunque tienen su base en páginas web, las PWA utilizan tecnologías que les permiten parecer y funcionar de manera muy similar a las aplicaciones nativas. Por ejemplo, pueden ejecutarse en segundo plano. Aunque se accede a ellas a través de un navegador, es posible anclar un acceso directo en el dispositivo, ya

sea en la pantalla de inicio o en el menú de aplicaciones. Además, no dependen de sistemas operativos específicos, ya que se ejecutan en el navegador, y pueden incorporar funcionalidades nativas del dispositivo. (López, 2020)

Características

- **Responsive:** Hoy en día, la mayoría de los sitios web cuentan con diseño responsive, lo que les permite adaptarse a diferentes dispositivos, una característica crucial dado el predominio de los smartphones. Aunque las PWA van más allá del diseño responsive básico, esta capacidad sigue siendo una de sus características principales. Estas aplicaciones deben adaptarse automáticamente a cualquier formato, navegador o dispositivo, considerando su naturaleza móvil. (López, 2020)
- **Actualizada:** Las PWA siempre muestran la última versión al usuario mediante actualizaciones automáticas y constantes, sin necesidad de que el usuario las descargue. Esto se logra gracias al uso de Service Workers y al hecho de que, al final del día, son aplicaciones web independientes de las tiendas de aplicaciones y sus procesos de revisión e instalación. (López, 2020)
- **Segura:** Las PWA utilizan el protocolo seguro HTTPS, que es esencial para la instalación del Service Worker. Esto garantiza un acceso seguro y la integridad del contenido servido, gracias a tecnologías como TLS para el cifrado web. (López, 2020)
- **Rápida:** Por lo general, las PWA tienen una velocidad optimizada tanto de carga como de navegación. Esto permite que el contenido se muestre al usuario casi instantáneamente, ya que aprovechan el almacenamiento en caché. Las interacciones, como clics o desplazamientos, también deben ser rápidas. El menor tamaño de las Progressive Web Apps en comparación con las aplicaciones nativas contribuye significativamente a esta velocidad. (López, 2020)
- **Offline:** Las PWA deben permitir el acceso, total o parcial, incluso cuando no hay conexión a Internet o esta es de baja calidad. Esto se logra mediante el uso de service workers y el almacenamiento en caché de información esencial para iniciar la aplicación. Gracias a esto, los usuarios pueden acceder a cierto contenido incluso cuando están sin conexión, lo que mejora la experiencia del usuario y evita la frustración por la falta de acceso. (López, 2020)
- **Multiplataforma:** Las PWA están diseñadas para ejecutarse en diversos dispositivos, sistemas operativos y navegadores. Esto no solo garantiza una experiencia de usuario satisfactoria y amplía el alcance del público, sino que también facilita el trabajo de los desarrolladores y reduce los costos, ya que no requieren desarrollos específicos para cada plataforma, a diferencia de las aplicaciones nativas. (López, 2020)

Tecnologías o requisitos que debe tener una PWA

Para comprender a fondo el funcionamiento de las PWA, es importante mencionar las tecnologías y métodos de trabajo que respaldan su desarrollo. Según Google, los requisitos que una Progressive Web App debe cumplir en este aspecto son cuatro (López, 2020):

Manifiesto de la aplicación: En Android y Chrome, se utiliza un archivo JSON simple, conocido como Manifiesto, que permite especificar diversas características para controlar la visualización de la aplicación después de su instalación. (López, 2020)

Service workers o trabajadores de servicio: Estos actúan como intermediarios entre el servidor o la red y el dispositivo o la aplicación. Son scripts de JavaScript que se ejecutan en el navegador y detectan eventos. Requieren el uso de HTTPS y funcionan independientemente de la aplicación, permitiendo el almacenamiento en caché de datos y el funcionamiento offline, así como el envío de notificaciones push. (López, 2020)

HTTPS: La PWA debe servir todas las solicitudes a través de HTTPS para garantizar la seguridad. Este protocolo es esencial para la instalación del service worker, ya que almacena una gran cantidad de información y requiere navegación cifrada para mantener la protección. (López, 2020)

Icono: Necesario para mostrar el acceso directo en el cajón de aplicaciones o en la pantalla de inicio del dispositivo. Está estrechamente vinculado al manifiesto de la aplicación mencionado anteriormente. (López, 2020)

Subtema 4: Publicación de APP en Google Play Store

Para poder publicar o subir una aplicación a Google Play Store se deben realizar algunos pasos como:

Crear una cuenta de desarrollador

Si estás lanzando tu primera aplicación en Google Play, el primer paso es crear una cuenta de desarrollador. Para hacerlo, necesitarás tener una cuenta de Google y proporcionar la información solicitada durante el proceso de registro. (YeePLY, s.f)

Una vez que hayas aceptado el acuerdo de distribución para desarrolladores, deberás pagar una tarifa única de registro de 25 USD. Después, podrás completar los detalles de tu cuenta. Tu nombre de desarrollador será visible para los usuarios, pero también podrás agregar información adicional de contacto, como tu sitio web, dirección de correo electrónico y dirección física. (YeePLY, s.f)

Además, si tu aplicación incluye compras dentro de la app o es de pago, deberás configurar un perfil de pagos. Este perfil te permitirá gestionar los pagos mensuales y acceder a informes de ventas a través de Play Console. (YeePLY, s.f)

Crear una nueva app

Una vez que hayas configurado tu cuenta en Play Console, es hora de publicar tu aplicación en Google Play. (YeePLY, s.f)

En el menú, elige 'Todas las aplicaciones' y selecciona 'Crear Aplicación'. Luego, elige el idioma predeterminado y añade el título de tu aplicación que se mostrará en la tienda. No te preocupes si tienes dudas, ya que podrás modificarlo más adelante. Una vez completados estos pasos, simplemente haz clic en 'Crear'. (YeePLY, s.f)

Completar los datos de la ficha de Play Store

Antes de publicar tu aplicación, es crucial completar la ficha de la app en Play Store. Esta información será visible para los usuarios y facilitará que te encuentren a través de las búsquedas. Aunque algunos campos son obligatorios y otros son opcionales, te recomendamos completarlos todos. Cuanta más información proporciones, más probable será que aparezcas en las búsquedas. (YeePLY, s.f)

- **Detalles del producto:** Este es uno de los aspectos más importantes. Debes indicar el nombre de la app (máximo 50 caracteres), una descripción breve (máximo 80 caracteres) y una descripción completa (máximo 4.000 caracteres). Es fundamental incluir la palabra clave que consideres más relevante para aumentar el tráfico. Sin embargo, evita incluir palabras clave sin sentido para atraer tráfico, ya que esto podría resultar en la suspensión de la app en Google Play. (YeePLY, s.f)
- **Recursos gráficos:** Las capturas de pantalla y los vídeos que incluyas en la descripción de tu app son clave para causar una buena impresión. Puedes agregar hasta 8 capturas de pantalla por cada tipo de dispositivo compatible. Asegúrate de que reflejen las principales funcionalidades y sean atractivas para captar la atención de los usuarios. (YeePLY, s.f)
- **Idiomas:** Aunque el idioma predeterminado es el inglés, puedes personalizarlo y añadir traducciones para tu aplicación. Si no proporcionas traducciones, los usuarios podrían ver una traducción automática. Si tu app estará disponible en varios países, es recomendable incluir traducciones para garantizar una experiencia de usuario óptima. (YeePLY, s.f)
- **Categoría:** Debes seleccionar el tipo de aplicación (app o juegos) y la categoría correspondiente. También puedes completar la clasificación del contenido, pero este paso se realiza después de subir la APK. (YeePLY, s.f)
- **Datos de contacto:** Es importante proporcionar al menos una dirección de correo electrónico para que los usuarios puedan ponerse en contacto contigo. También puedes incluir un sitio web o un número de teléfono para ofrecer más opciones de contacto. (YeePLY, s.f)
- **Política de privacidad:** Debes incluir un enlace a tu política de privacidad para informar a los usuarios sobre el uso que harás de sus datos confidenciales y los de su dispositivo. Si tu app solicita acceso a información o contenido sensible, el enlace a la política de privacidad debe estar disponible tanto en la ficha de la Play Store como dentro de la app. (YeePLY, s.f)

Subir la APK

Es hora de cargar el código fuente de tu aplicación o APK. Sin embargo, antes de hacerlo, necesitas crear una versión de la aplicación. En Play Console, puedes hacer esto en el menú "Gestión de versiones" / "Versiones de la aplicación". (YeePLY, s.f)

Tendrás la opción de elegir entre lanzar una prueba interna, una versión cerrada para un grupo selecto de testers, una versión abierta o, finalmente, crear una versión de producción que estará disponible para todos los usuarios de la aplicación. (YeePLY, s.f)

Clasificación de contenido

Es crucial que las aplicaciones en Google Play proporcionen información sobre su clasificación por edad. Tras subir el código fuente, es esencial completar el cuestionario de clasificación de contenido. De lo contrario, tu aplicación se mostrará como "Sin clasificar" y podría ser eliminada de la tienda. Para hacerlo, en Play Console, selecciona tu aplicación y luego, en el menú de la izquierda, elige "Presencia en Google Play Store" / "Clasificación de contenido". (YeePLY, s.f)

Establecer precio y distribución

El primer paso consiste en determinar si la aplicación será gratuita o de pago. Es importante tener en cuenta que siempre puedes cambiar una aplicación de pago a gratuita, pero no al revés. Además, en este punto, también puedes seleccionar en qué países deseas que esté disponible tu aplicación. (YeePLY, s.f)

Enviar la app a revisión

Antes de enviar tu aplicación para su revisión, es importante revisar toda la información que has proporcionado. Verifica que el nombre sea correcto, que la ficha de la Play Store contenga información relevante, entre otros detalles. Si toda la información está completa, verás que cada sección tendrá una marca de verificación verde. (YeePLY, s.f)

En este punto, simplemente regresa a la sección de 'Versiones de la aplicación', selecciona la versión que deseas lanzar y elige 'Revisar y Lanzar' para verificar que no haya problemas con la versión. Luego, solo queda seleccionar 'Confirmar lanzamiento' para subir la aplicación a Google Play Store. (YeePLY, s.f)

Esperar

Una vez completado este paso, deberás aguardar a que tu aplicación sea revisada para estar disponible en la tienda. Si es tu primera aplicación, este proceso de validación puede demorar un par de días. Sin embargo, una vez que tengas varias aplicaciones subidas, es posible que la espera sea de solo unas pocas horas. (YeePLY, s.f)

Preguntas de Comprensión de la Unidad

1. ¿Cuál es la diferencia clave entre Responsive Web Design y Mobile First Web?

El Responsive Web Design se centra en la capacidad de un sitio web para adaptarse a diferentes tamaños y dispositivos, priorizando la accesibilidad y usabilidad en cualquier dispositivo. En cambio, el enfoque Mobile First implica comenzar el diseño desde la resolución más pequeña y luego expandirlo a resoluciones mayores, priorizando la experiencia móvil como punto de partida.

2. ¿Cuál es la diferencia fundamental entre React Native y otras tecnologías de desarrollo móvil?

React Native permite crear aplicaciones nativas reales tanto en iOS como en Android utilizando JavaScript y la biblioteca React para crear componentes visuales. A diferencia de otras tecnologías, React Native ejecuta estos componentes directamente en las plataformas móviles nativas, lo que resulta en una aplicación nativa real al final del proceso.

3. ¿Cómo garantiza Flutter una experiencia visual consistente en todas las plataformas?

Flutter proporciona un rendimiento rápido, constante y personalizable al utilizar la biblioteca gráfica de código abierto Skia de Google para renderizar la interfaz de usuario. Esto asegura una experiencia visual uniforme en todas las plataformas, independientemente del dispositivo utilizado para acceder a la aplicación.

4. ¿Cómo contribuye el tamaño reducido de las Progressive Web Apps (PWA) a su velocidad de carga y rendimiento?

Por lo general, las PWA tienen una velocidad optimizada tanto de carga como de navegación. Esto permite que el contenido se muestre al usuario casi instantáneamente, ya que aprovechan el almacenamiento en caché. Las interacciones, como clics o desplazamientos, también deben ser rápidas. El menor tamaño de las Progressive Web Apps en comparación con las aplicaciones nativas contribuye significativamente a esta velocidad.

5. ¿Cómo se garantiza que las Progressive Web Apps (PWA) muestren siempre la última versión al usuario?

Las PWA siempre muestran la última versión al usuario mediante actualizaciones automáticas y constantes, sin necesidad de que el usuario las descargue. Esto se logra gracias al uso de Service Workers y al hecho de que, al final del día, son aplicaciones web independientes de las tiendas de aplicaciones y sus procesos de revisión e instalación.

Material complementario

Los siguientes recursos complementarios son sugerencias para que se pueda ampliar la información sobre el tema trabajado, como parte de su proceso de aprendizaje autónomo:

Videos de apoyo:

- <https://www.youtube.com/watch?v=7bhlQK26Brw>
- <https://www.youtube.com/watch?v=VJgVuZ0YGag>
- https://www.youtube.com/watch?v=fY_O1Fb-e5Y

Links de apoyo:

- <https://ionicframework.com/>
- <https://getbootstrap.com/docs/5.0/getting-started/introduction/>
- https://www.youtube.com/watch?v=SQLDK1nPk_w

Bibliografía

- » Appsflyer. (s.f). Aplicación híbrida. Recuperado de <https://www.appsflyer.com/es/glossary/hybrid-app/>
- » IMMUNE. (2023). Aplicaciones híbridas, ¿cómo funcionan y por qué son el futuro del desarrollo móvil?. Recuperado de <https://immune.institute/blog/aplicaciones-hibridas-desarrollo-mobile/>
- » Duò, M. (2022). La Guía para Principiantes del Diseño Web Responsivo (muestras de códigos y ejemplos de diseño). Recuperado de <https://kinsta.com/es/blog/diseno-de-paginas-web-sensibles/>
- » MANZ.DEV. (s.f). ¿Qué es Responsive Design?. Recuperado de <https://lenguajecss.com/css/responsive-web-design/que-es/>
- » Boneu, M. (2020). Diseño Web Responsivo — Cómo hacer que un sitio Web se vea bien en Teléfonos y Tablet. Recuperado de <https://www.freecodecamp.org/espanol/news/disenio-web-responsive-como-hacer-que-un-sitio-web-se-vea-bien-en-telefonos-y-tabletas/>
- » Guerras, A. (s.f). Diseño responsive: ¿cómo configurarlo correctamente?. Recuperado de <https://aukera.es/blog/disenio-responsive/>
- » Prieto, R. (2021). Mejores Frameworks CSS para diseño web responsive. Recuperado de <https://www.silocreativo.com/mejores-frameworks-css/>
- » Cristancho, F. (2022). ¿Qué es Ionic?. Recuperado de <https://talently.tech/blog/que-es-ionic/>
- » Blanes, J. (s.f). ¿Qué es React Native?. Recuperado de <https://www2.deloitte.com/es/es/pages/technology/articles/que-es-react-native.html>
- » AWS. (s.f). ¿Qué es Flutter?. Recuperado de <https://aws.amazon.com/es/what-is/flutter/>
- » López, S. (2020). Qué es PWA: características, tecnologías, ventajas y desventajas. Recuperado de <https://digital55.com/blog/que-es-pwa-ventajas-desventajas/>
- » YeePLY. (s.f). Guía para subir tu app a Google Play Store y triunfar. Recuperado de <https://www.yeeply.com/blog/desarrollo-de-apps/guia-subir-app-google-play-store/>